

Cow book: Chapter 12 (12.5)

Chapter 1 – DBMS

ITERATORS AND JOINS

Big picture overview

SQL Query

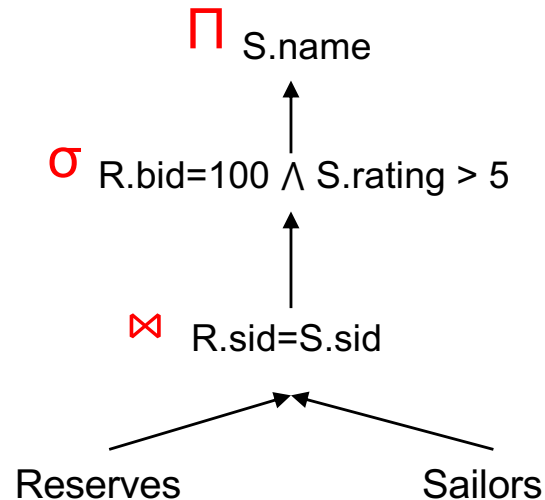
```
SELECT S.name
FROM Reserves R, Sailors S
WHERE R.sid = S.sid
AND R.bid = 100
AND S.rating > 5
```

Query Parser

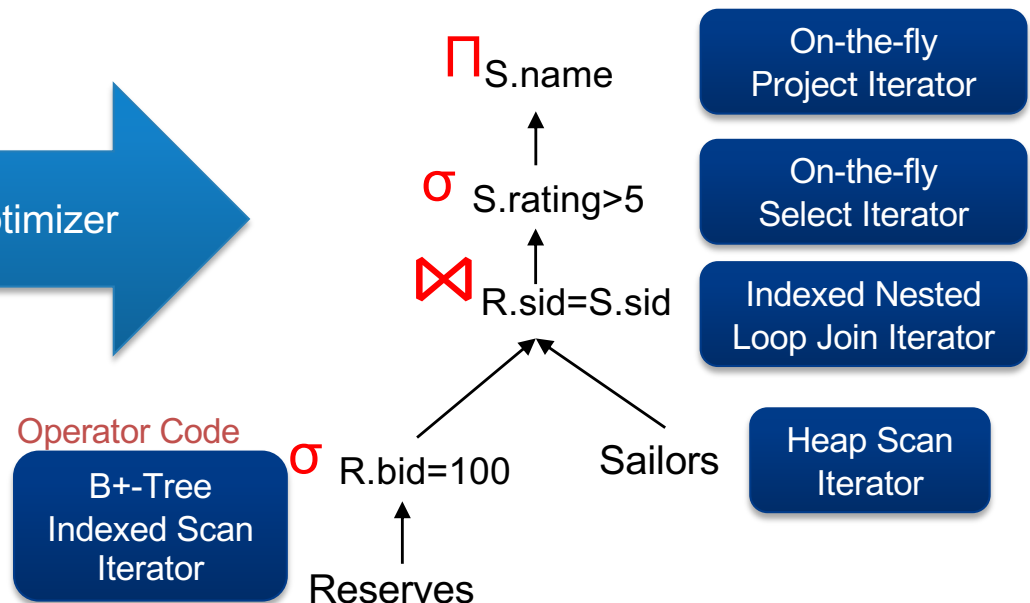
Relational Algebra

```
 $\Pi_{S.name}(\sigma_{bid=100 \wedge rating > 5}(Reserves \bowtie_{R.sid=S.sid} Sailors))$ 
```

(Logical) Query Plan:



Optimized (Physical) Query Plan:



Schema for examples

Cost notation

- $[R]$: the number of **pages** to store R
- p_R : number of **records per page** of R
- $|R|$: the **cardinality** (*number of records*) of R
 - $|R| = p_R * [R]$

Reserves (sid : int, bid : int, day : date, $rname$: string)

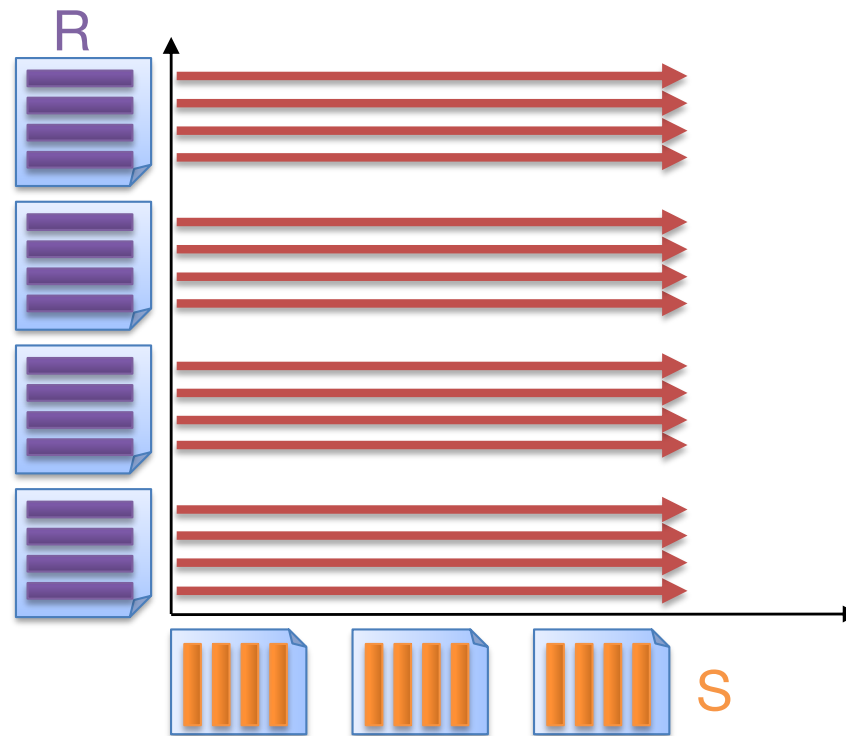
- $[R]=1000$, $p_R=100$, $|R| = 100,000$

Sailors (sid : int, $sname$: string, $rating$: int, age : real)

- $[S]=500$, $p_S=80$, $|S| = 40,000$

Simple nested loops join

Join(R,S): foreach record r in R do
 foreach record s in S do
 if $\theta(r_i, s_j)$ then add $\langle r, s \rangle$ to result



$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$

Quick quizz

What is the I/O cost of Join(R,S)?

(a) $[R] * [S]$

(b) $[R] * [S] + [R]$

(c) $(p_R * [R]) * [S] + [R]$

(d) $[R] * [S] + p_R$

Quick quizz

What is the I/O cost of Join(R,S)?

(a) $[R] * [S]$

(b) $[R] * [S] + [R]$

(c) $(p_R * [R]) * [S] + [R]$

(d) $[R] * [S] + p_R$

Quick quizz

What is the I/O cost of Join(R,S)?

(a) $[R] * [S]$

(b) $[R] * [S] + [R]$ (page of R at a time !)

(c) $(p_R * [R]) * [S] + [R]$

(d) $[R] * [S] + p_R$

Quick quizz

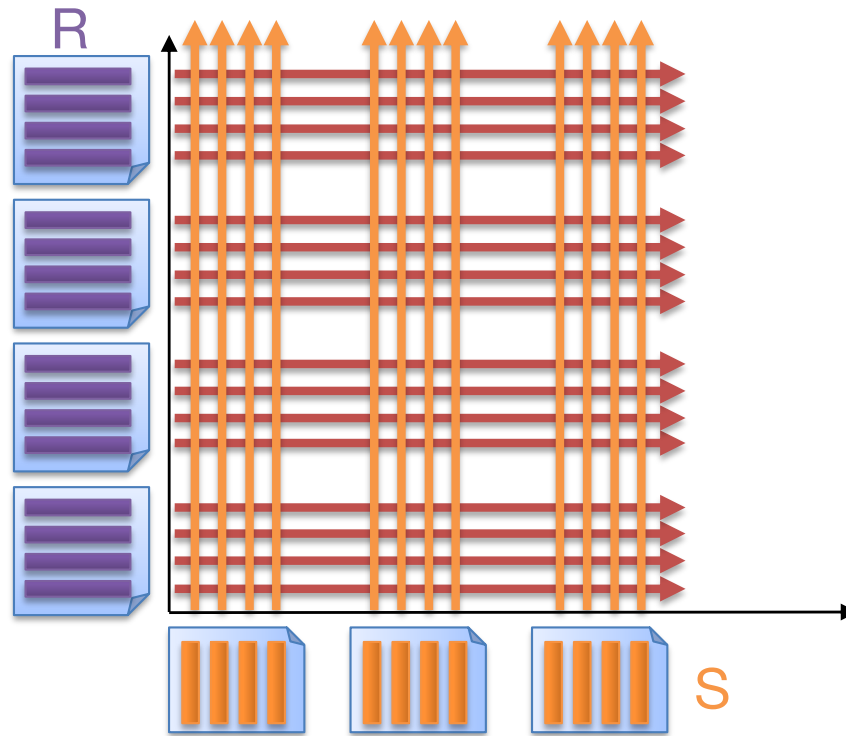
- What if smaller relation (S) was “outer”?
- What about when p_R is small (say one record per page)
- What if S fits in memory completely ?

Quick quizz

- What if smaller relation (S) was “outer”?
 $[R] * [S] + [S]$
- What about when p_R is small (say one record per page)
we might want R to be outer relation
- What if S fits in memory completely ?
 $[S] + [R]$

Important special case

Join(R,S): foreach **record** r in R do
 foreach **record** s in S do
 if $r_i.a == s_j.a$ then add $\langle r, s \rangle$ to result



$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$

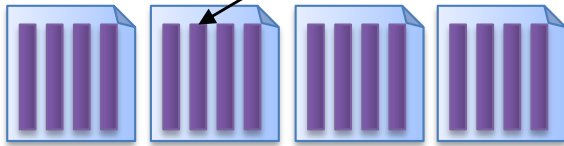
Index nested loops join

Join(R,S): **foreach tuple** r in R do

foreach tuple s in S where $r_i == s_i$ do

add $\langle r, s \rangle$ to result

R



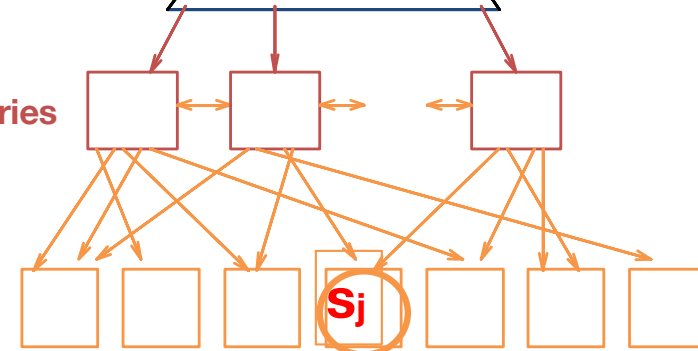
lookup(r_i)

INDEX on S

Data entries

S :

Data Records



Index nested loops join

Reserves (sid: int, bid: int, day: date, *rname*: string)

- $[R]=1000$, $p_R=100$, $|R| = 100,000$

Sailors (sid: int, *sname*: string, *rating*: int, *age*: real)

- $[S]=500$, $p_S=80$, $|S| = 40,000$



Index on this!

Index nested loops join

Join(R,S): **foreach tuple r** in R **do**
 foreach tuple s in S where $r_i == s_j$ **do**
 add <r, s> to result

$$\text{Cost} = [R] + ([R] * p_R) * \underline{\text{cost to find matching S tuples}}$$

- If index uses **Alt. 1** $\rightarrow$ cost to traverse tree from root to leaf.
(e.g., 2-4 IOs)
- For **Alt. 2** or **3**:
 1. Cost to lookup RID(s); typically 2-4 IOs for B+Tree.
 2. Cost to retrieve records by RID(s); depends on clustering...
 - **Clustered index**: 1 I/O per page of matching S tuples.
 - **Unclustered**: up to 1 I/O per matching S tuple (w/o sorting)

Why?

Quick quizz

Assume that on S we have an Alternative 2 B+ Tree of Height 3.
What is the I/O cost of Join(R,S)?

(a) $[R] + ([R] * p_R) * (4 + 1)$

(b) $[R] + ([S] * p_R) * (4 + 1)$

(c) $(p_R * [R]) * [S] + [R]$

(d) Not enough information to answer

$$\begin{aligned} [R] &= 1,000, p_R = 100, |R| = 100,000 \\ [S] &= 500, p_S = 80, |S| = 40,000 \end{aligned}$$

Quick quizz

Assume that on S we have an Alternative 2 B+ Tree of Height 3.
What is the I/O cost of Join(R,S)?

(a) $[R] + ([R] * p_R) * (4 + 1)$ Why?

(b) $[R] + ([S] * p_R) * (4 + 1)$

(c) $(p_R * [R]) * [S] + [R]$

(d) Not enough information to answer

$$\begin{aligned} [R] &= 1,000, p_R = 100, |R| = 100,000 \\ [S] &= 500, p_S = 80, |S| = 40,000 \end{aligned}$$

Index nested loops join

Reserves (sid: int, bid: int, day: date, *rname*: string)

- $[R]=1000$, $p_R=100$, $|R| = 100,000$

Sailors (sid: int, *sname*: string, *rating*: int, *age*: real)

- $[S]=500$, $p_S=80$, $|S| = 40,000$

Index on this!

Join(S,R)

(a) Always Less Cost

(b) Always More Cost

(c) It Depends

Index nested loops join

Reserves (sid: int, bid: int, day: date, *rname*: string)

- $[R]=1000$, $p_R=100$, $|R| = 100,000$

Sailors (sid: int, *sname*: string, *rating*: int, *age*: real)

- $[S]=500$, $p_S=80$, $|S| = 40,000$

Index on this!

Join(S,R)

(a) Always Less Cost

(b) Always More Cost

(c) It Depends

Index nested loops join

$$\text{Cost}(S,R) = [S] + ([S] * p_s) * (\text{Search} + \text{RetrievalR})$$

$$\text{Cost}(R,S) = [R] + ([R] * p_r) * (\text{Search} + \text{RetrievalS})$$

RetrievalR = 1

- $R \bowtie S$: $1000 + (1,000 * 100) * 5 = 501,000$
- $S \bowtie R$: $500 + (500 * 80) * 5 = \mathbf{200,500}$

RetrievalR = 9

- $R \bowtie S$: $1000 + (1,000 * 100) * 5 = 501,000$
- $S \bowtie R$: $500 + (500 * 80) * 13 = 520,500$

$$\begin{aligned} [R] &= 1,000, p_R = 100, |R| = 100,000 \\ [S] &= 500, p_S = 80, |S| = 40,000 \end{aligned}$$

Sort-merge join

$$\mathbf{R} \bowtie_{\text{sid}} \mathbf{S} = \sigma_{\text{sid}}(\mathbf{R} \times \mathbf{S})$$

- Requires equality predicate:
 - Works for **equi-joins** & **natural joins**
- Two stages:
 - **Sort** tuples in $\mathbf{R}$ and $\mathbf{S}$ by join key
 - all tuples with same key in consecutive order
 - input might already be sorted ... why?
 - **Merge** scan the sorted partitions and emit tuples that match

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```



sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty



sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103
28	yuppy	104

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103
28	yuppy	104

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103
28	yuppy	104

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103
28	yuppy	104
31	lubber	101

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103
28	yuppy	104
31	lubber	101
31	lubber	102

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103
28	yuppy	104
31	lubber	101
31	lubber	102
31	lubber2	101

Sort-merge join

```
while not done {  
  while (r < s) { advance r }  
  while (r > s) { advance s }  
  // assert r == s  
  mark s // save start of "block"  
  while (r == s) {  
    // Outer loop over r  
    while (r == s) {  
      // Inner loop over s  
      yield <r, s>  
      advance s  
    }  
    reset s to mark  
    advance r  
  }  
}
```

sid	sname
22	dustin
28	yuppy
31	lubber
31	lubber2
44	guppy
58	rusty

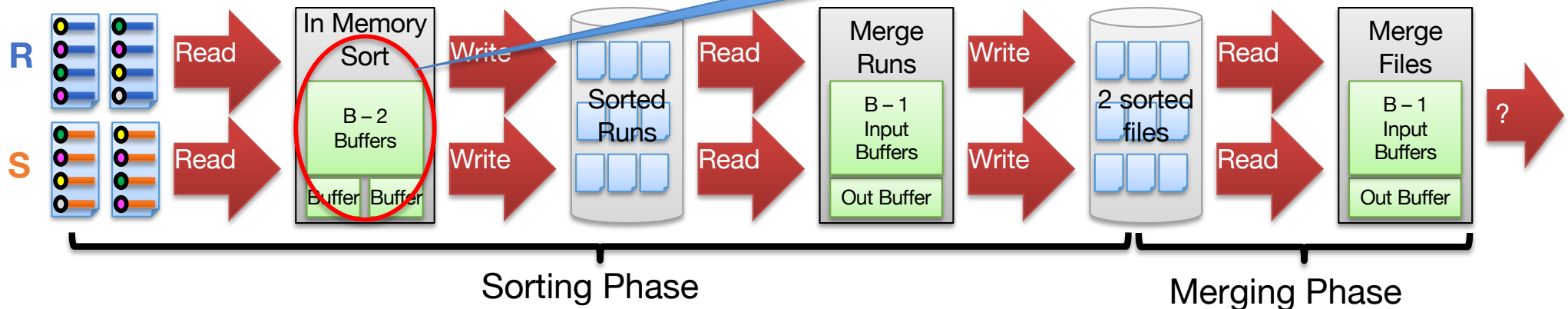
sid	bid
28	103
28	104
31	101
31	102
42	142
58	107

sid	sname	bid
28	yuppy	103
28	yuppy	104
31	lubber	101
31	lubber	102
31	lubber2	101
31	lubber2	102

What is the cost?

Cost of sort-merge join

Replacement sort
(see cowbook)



- Cost: $\text{Sort } R + \text{Sort } S + ([R] + [S])$
 - But in worst case, last term could be $[R] * [S]$ (very unlikely!)
 - Q: what is worst case?
- Question: How big does the buffer have to be to sort R and S in two passes each?
- Suppose buffer $B > \text{Sqrt}(\max([R], [S]))$:
 - Both R and S can be sorted in **2 passes**
 - $4 * 1000 + 4 * 500 + (1000 + 500) = \mathbf{7500}$

$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$

Join first, sort later?

SELECT sid, bid, sname, rname

FROM R, S

WHERE **R.sid = S.sid**

ORDER BY sid

Reserves (sid: int, bid: int, day: date, *rname*: string)

Sailors (sid: int, *sname*: string, *rating*: int, *age*: real)

Chunk NLJ

Join: $([S]/(B-2)) * [R] + [S]$

Sort: ?

Join first, sort later?

```
SELECT sid, bid, sname, rname  
FROM R, S  
WHERE R.sid = S.sid  
ORDER BY sid
```

Reserves (sid: int, bid: int, day: date, *rname*: string)

Sailors (sid: int, *sname*: string, *rating*: int, *age*: real)

Chunk NLJ

Join: $([S]/(B-2))^*[R] + [S]$

Sort: $2 * [R]^* (1 + \log_{B-1} [R]/B)$

Join first, sort later?

SELECT sid, bid, sname, rname

FROM R, S

WHERE R.sid = S.sid

ORDER BY sid

Assume $B = 102$

Join: $([S]/(B-2)) * [R] + [S] = \mathbf{5500} + \dots$

Sort: $2 * [R] * (1 + \log_{B-1} [R]/B) = \dots + \mathbf{4000}$

$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$

Join first, sort later?

```
SELECT sid, bid, sname, rname  
FROM R, S  
WHERE R.sid = S.sid  
ORDER BY sid
```

Sort-Merge

Sort R: $2*[R]*(2) = 4000$

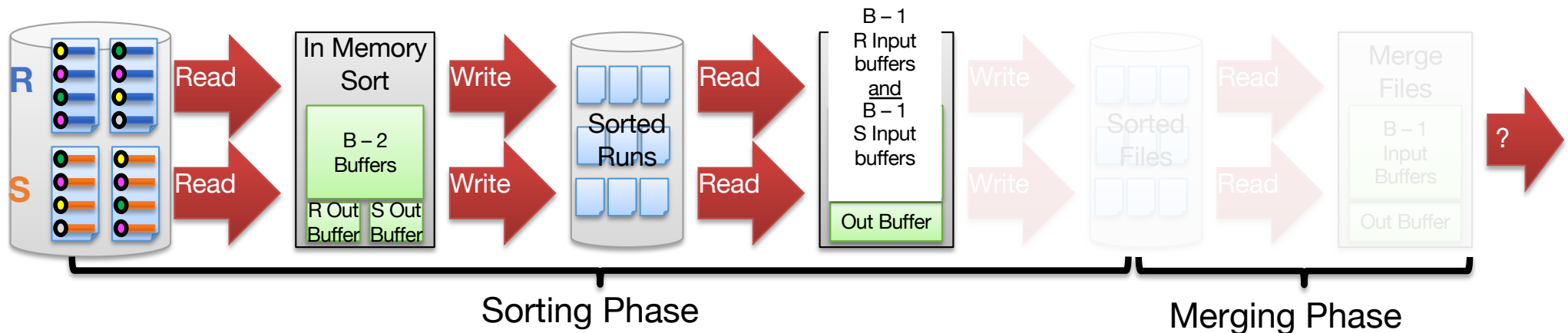
Sort S: $2*[S]*(2) = 2000$

$R + S = 1500$

Total = 7500!

$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$

Sort-merge join considerations



- An important refinement, still assuming R and S can be sorted in 2 passes (but this refinement needs 2x buffers):
 - Do the join during the final merging pass of sort
 1. Read R and write out sorted runs (pass 0)
 2. Read S and write out sorted runs (pass 0)
 3. Merge **R-runs and S-runs**, *while* finding $R \bowtie S$ matches
 - Cost = $3 * [R] + 3 * [S] = 4500$

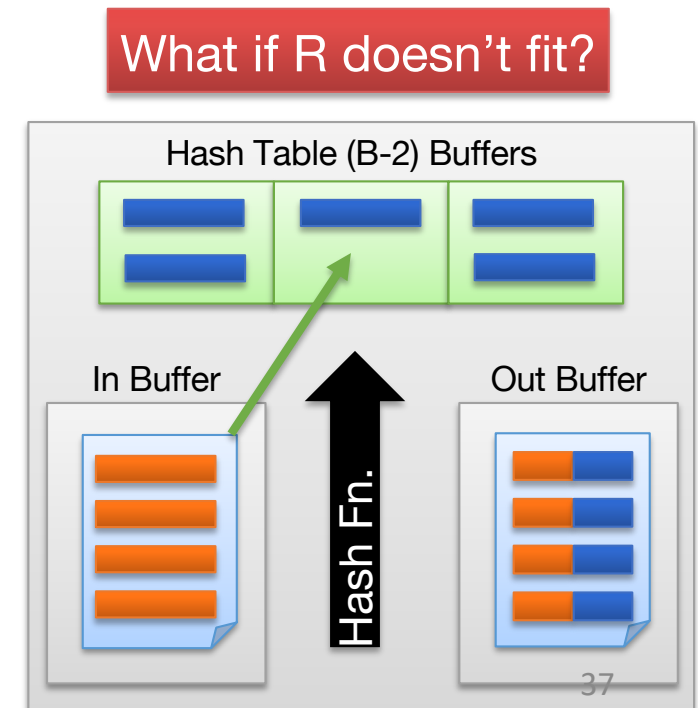
Quick quizz

When is Sort-Merge join a pretty good choice ?

- (A) One or both inputs are already sorted on the join attribute(s)
- (B) We are running an arbitrary theta-join
- (C) The output needs to be sorted on the join attributes(s)
- (D) None of the above

Naïve in-memory hash join

- Requires equality predicate:
 - Works for **equi-joins** & **natural joins**
- Assume **R** is smaller relation
 - Require **R** to fit in memory
- Simple algorithm:
 - Load all **R** into hash table
 - Scan **S** and probe **R**
- Memory requirements?
 - $R < (B-2) * \text{hash_fill}$



Properties that help

Reserves (sid: int, bid: int, day: date, *rname*: string)

Sailors (sid: int, *sname*: string, *rating*: int, *age*: real)

$$\sigma_{\text{sid}=4 \text{ or } 6} (\mathbf{R} \bowtie_{\text{sid}} \mathbf{S}) = \sigma_{\text{sid}=4} (\mathbf{R} \bowtie_{\text{sid}} \mathbf{S}) \cup \sigma_{\text{sid}=6} (\mathbf{R} \bowtie_{\text{sid}} \mathbf{S})$$

Can decompose into smaller “partial joins”

$$\sigma_{\text{sid}=4} (\mathbf{R} \bowtie_{\text{sid}} \mathbf{S}) = (\sigma_{\text{sid}=4} (\mathbf{R}) \bowtie_{\text{sid}} \sigma_{\text{sid}=4} (\mathbf{S}))$$

$$\sigma_{\text{hash}(\text{sid})=4} (\mathbf{R} \bowtie_{\text{sid}} \mathbf{S}) = (\sigma_{\text{hash}(\text{sid})=4} (\mathbf{R}) \bowtie_{\text{sid}} \sigma_{\text{hash}(\text{sid})=4} (\mathbf{S}))$$

Properties that help

What if this doesn't fit in memory?

$$\sigma_{\text{hash}(\text{sid})=4} (\mathbf{R} \bowtie_{\text{sid}} \mathbf{S}) = (\sigma_{\text{hash}(\text{sid})=4}(\mathbf{R}) \bowtie_{\text{sid}} \sigma_{\text{hash}(\text{sid})=4}(\mathbf{S}))$$

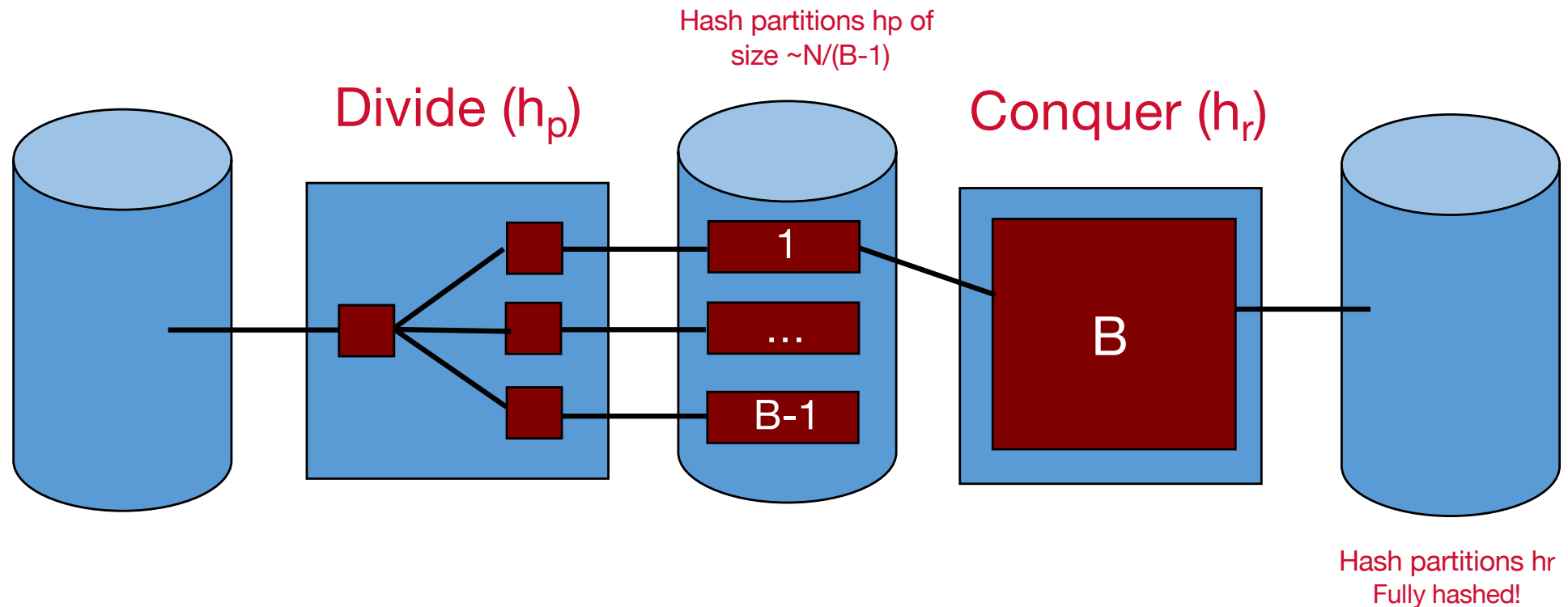
$\mathbf{R}_h \bowtie_{\text{sid}} \mathbf{S}_h$!!! a new join problem

Recursively partition with an orthogonal hash function

Grace hash join

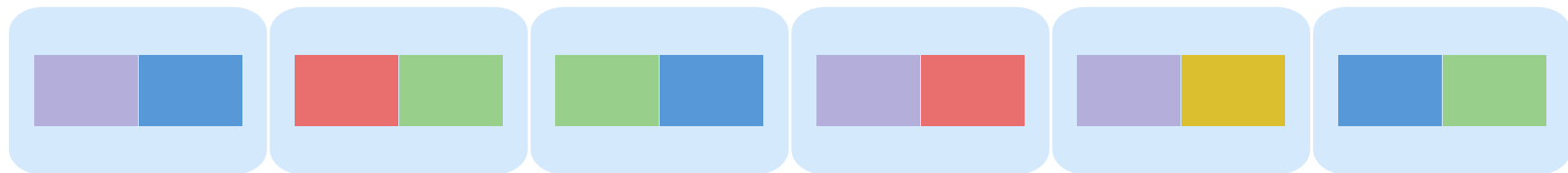
- Requires equality predicate:
 - Works for **equi-joins** & **natural joins**
- Two stages:
 - Divide* – **Partition** tuples from **R** and **S** by join key
 - all tuples for a given key in same partition
 - Conquer* – **Build & Probe** a separate hash table for each partition (like in Naïve Hash)
 - Assume **partition** of smaller relation fits in memory
 - Recurse if necessary...

Remember external hashing?



Recall external hashing: aggregating colors

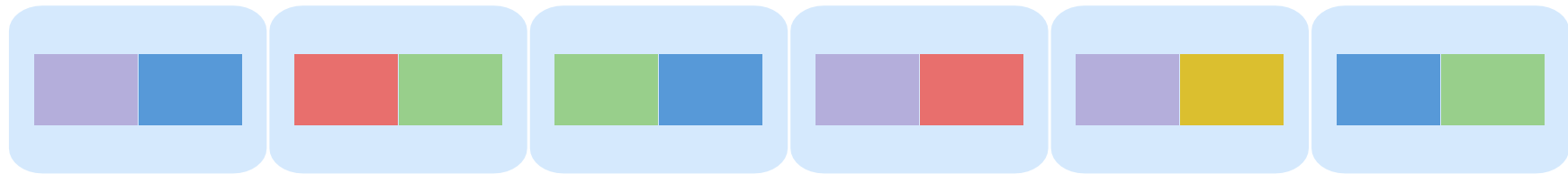
- Goal: Group squares by color
- Setup: 12 squares, each page fits 2 squares. We can hold 4 pages in memory.
- $N = 6, B = 4$



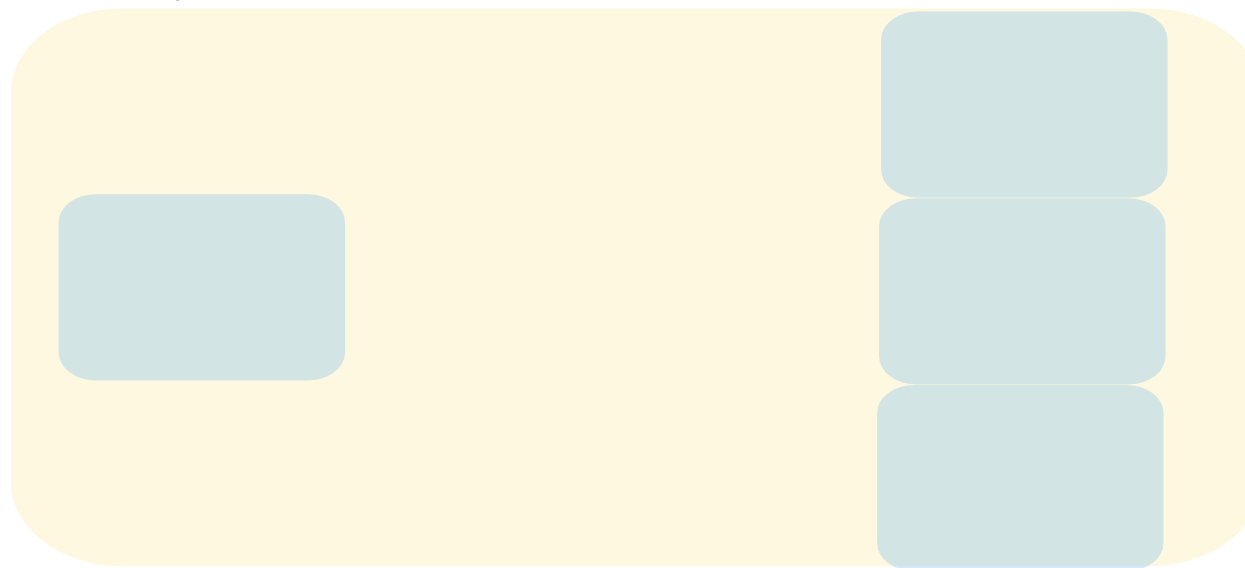
Pass 1: Divide

- Read all pages in, hash to B-1 partitions/buckets so that each group guaranteed to be in same partition.
- May not be a whole partition for each group.
- $\# \text{ I/O's} = 2N$

Pass 1: Divide



N=6, B=4



Assign colors to 3 partitions using hash function.

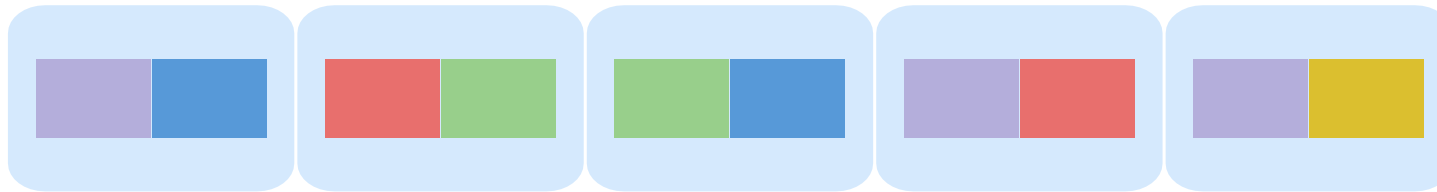
Our hash function:

{G,P} -> 1

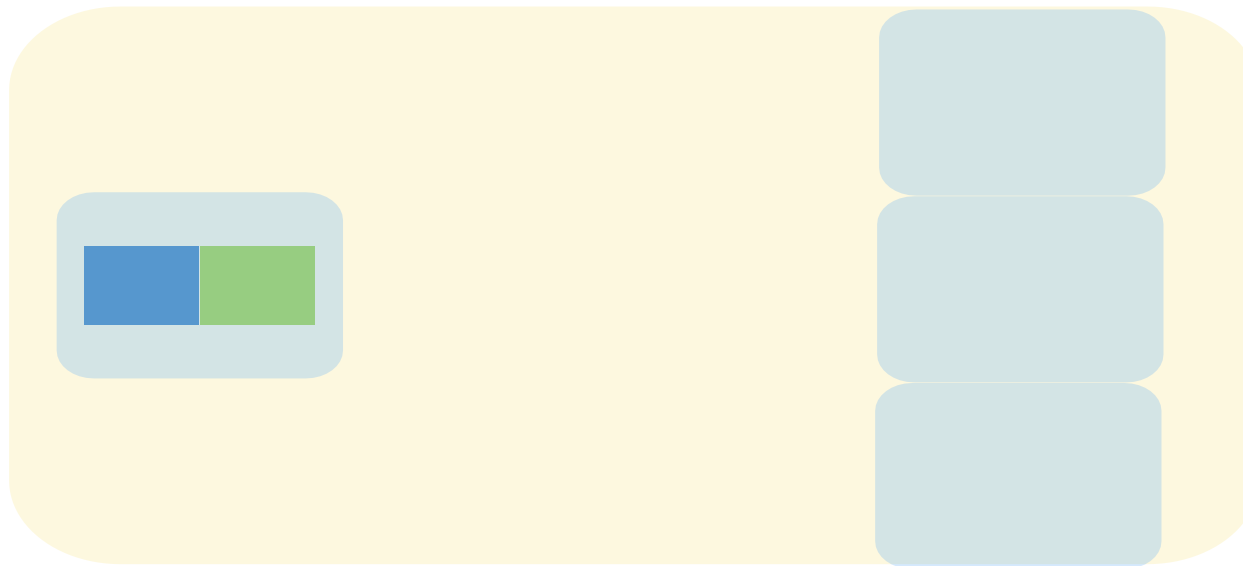
{B} -> 2

{R, Y} -> 3

Pass 1: Divide



N=6, B=4



Assign colors to 3 partitions using hash function.

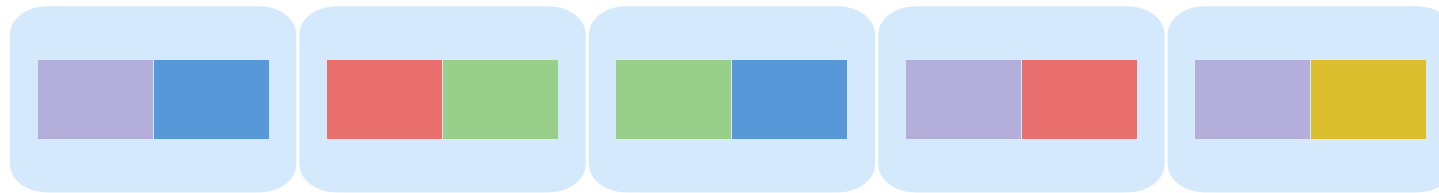
Our hash function:

{G,P} -> 1

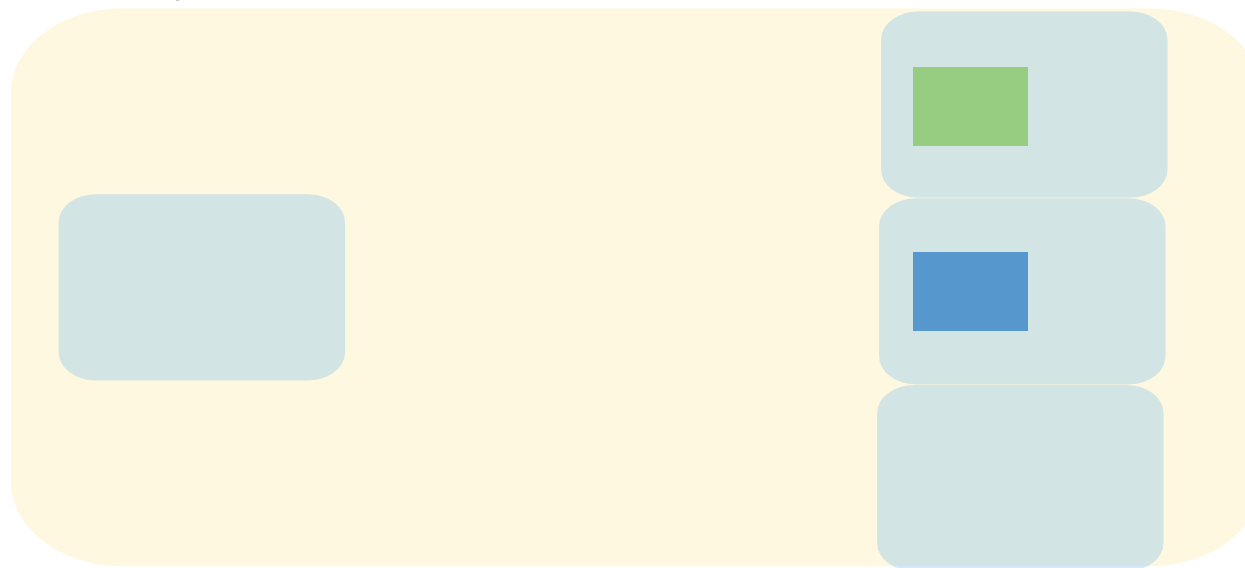
{B} -> 2

{R, Y} -> 3

Pass 1: Divide



N=6, B=4



Assign colors to 3 partitions using hash function.

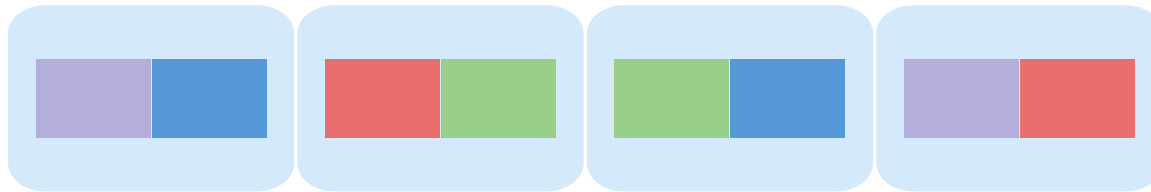
Our hash function:

{G,P} -> 1

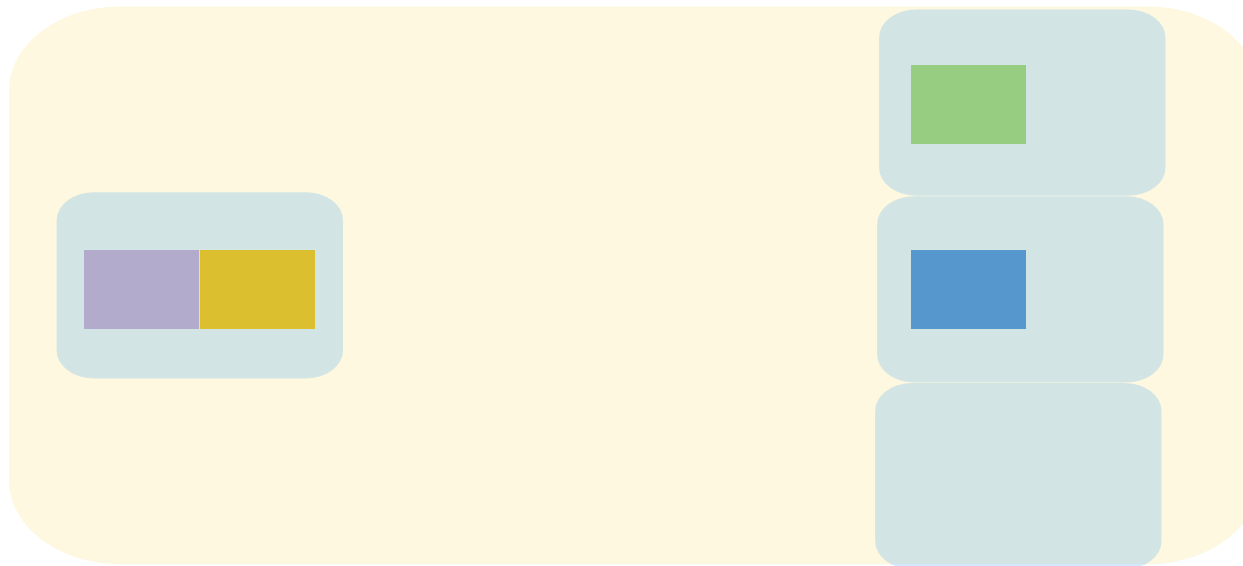
{B} -> 2

{R, Y} -> 3

Pass 1: Divide



N=6, B=4



Assign colors to 3 partitions using hash function.

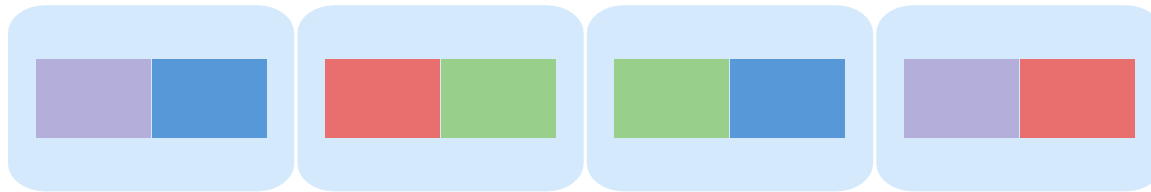
Our hash function:

{G,P} -> 1

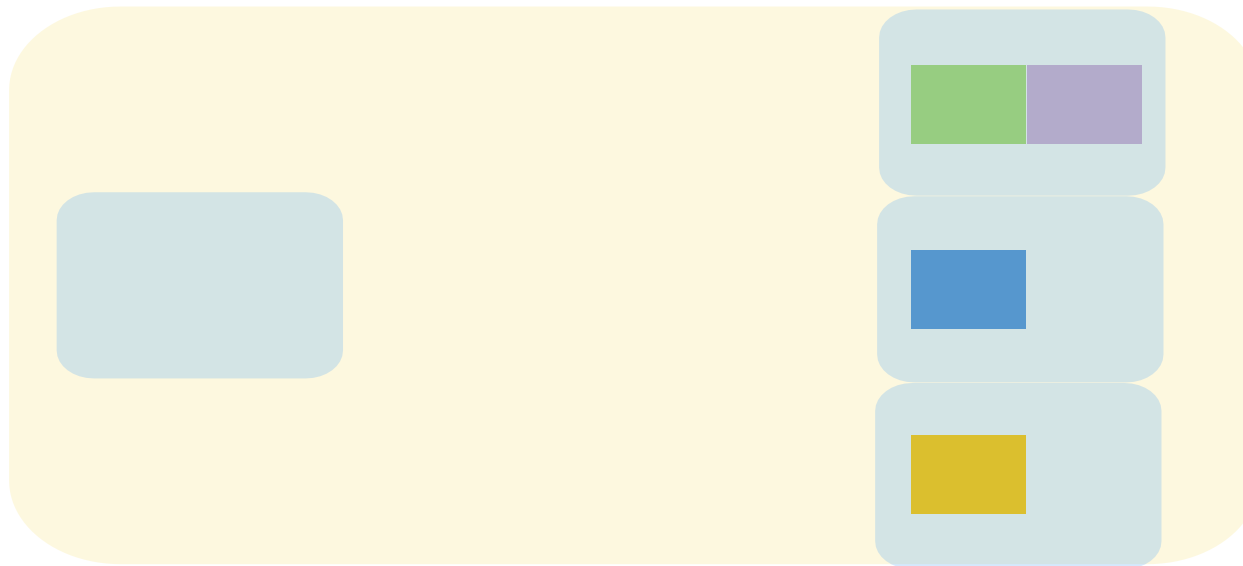
{B} -> 2

{R, Y} -> 3

Pass 1: Divide



N=6, B=4



Assign colors to 3 partitions using hash function.

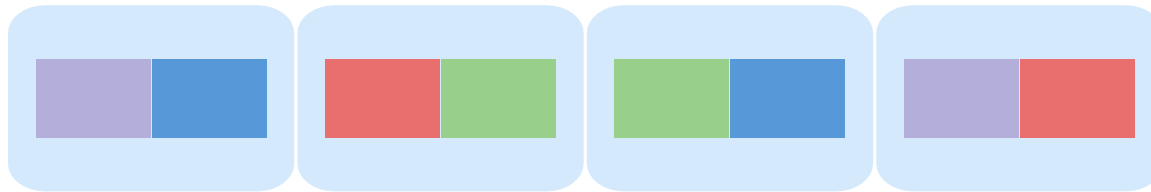
Our hash function:

{G,P} -> 1

{B} -> 2

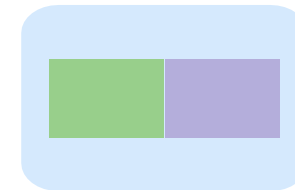
{R, Y} -> 3

Pass 1: Divide

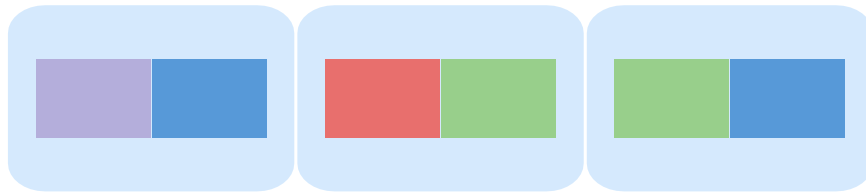


N=6, B=4

Our hash function: {G,P} $\rightarrow$ 1, {B} $\rightarrow$ 2, {R, Y} $\rightarrow$ 3

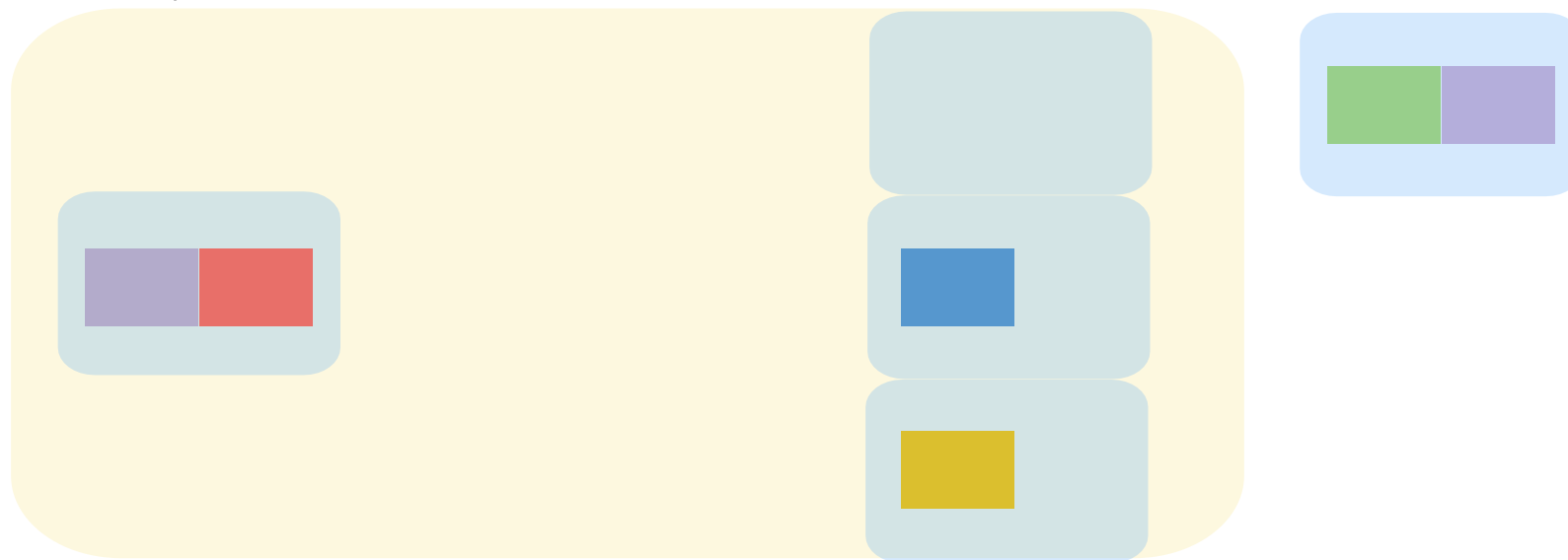


Pass 1: Divide

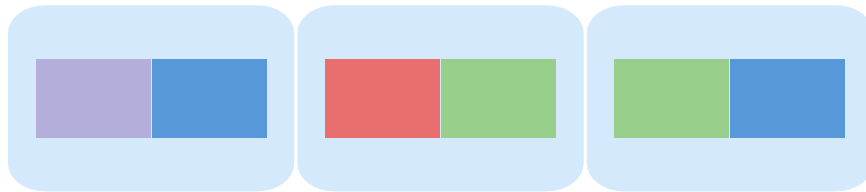


N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3

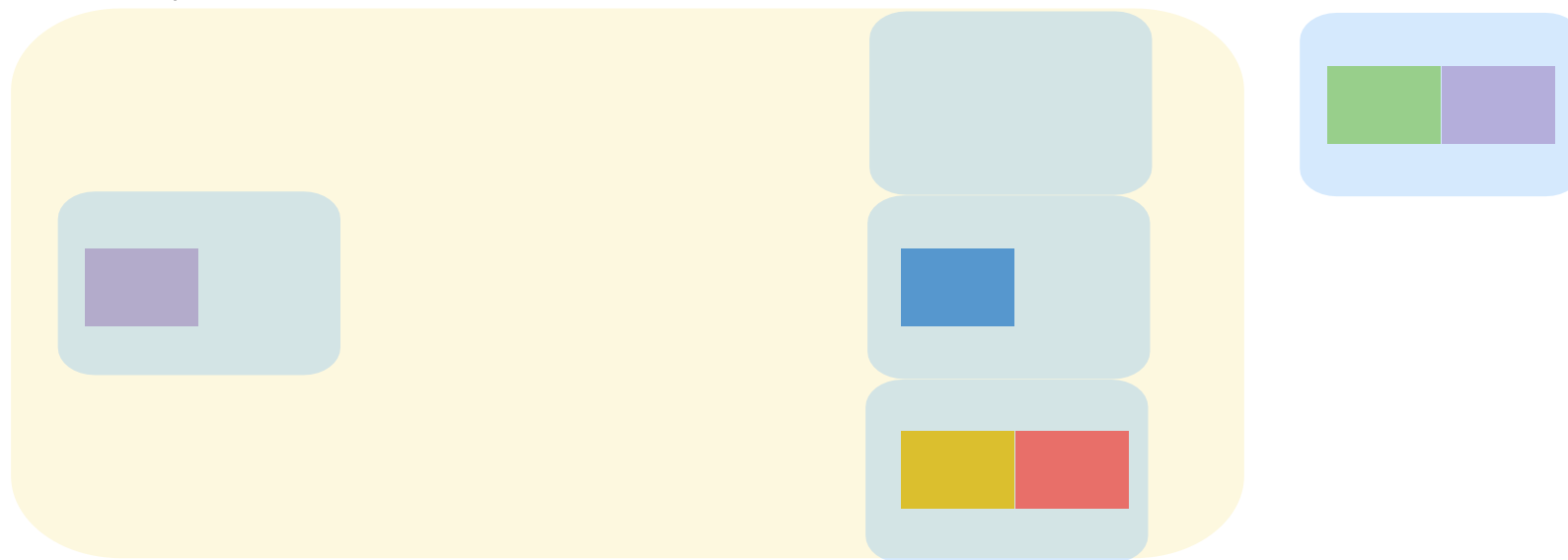


Pass 1: Divide

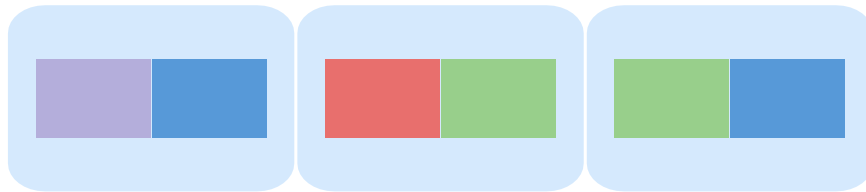


N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3

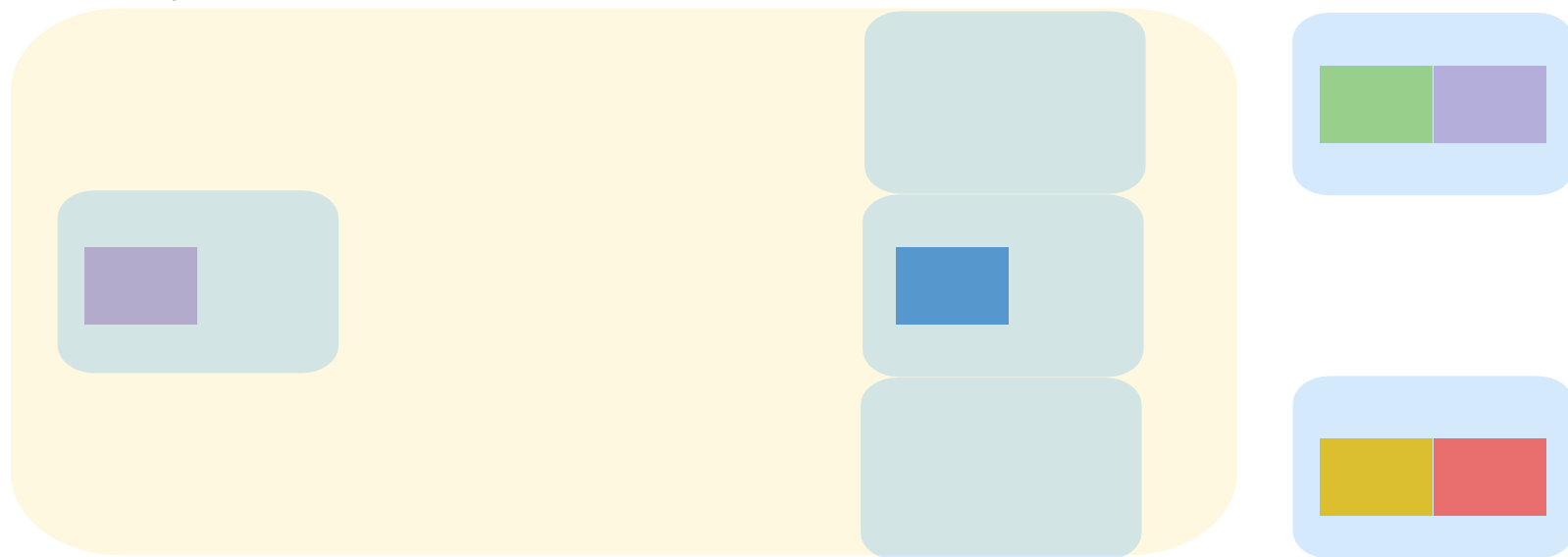


Pass 1: Divide

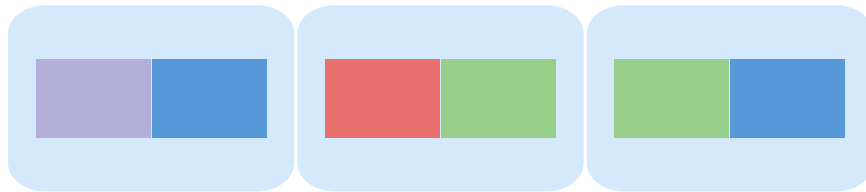


N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3

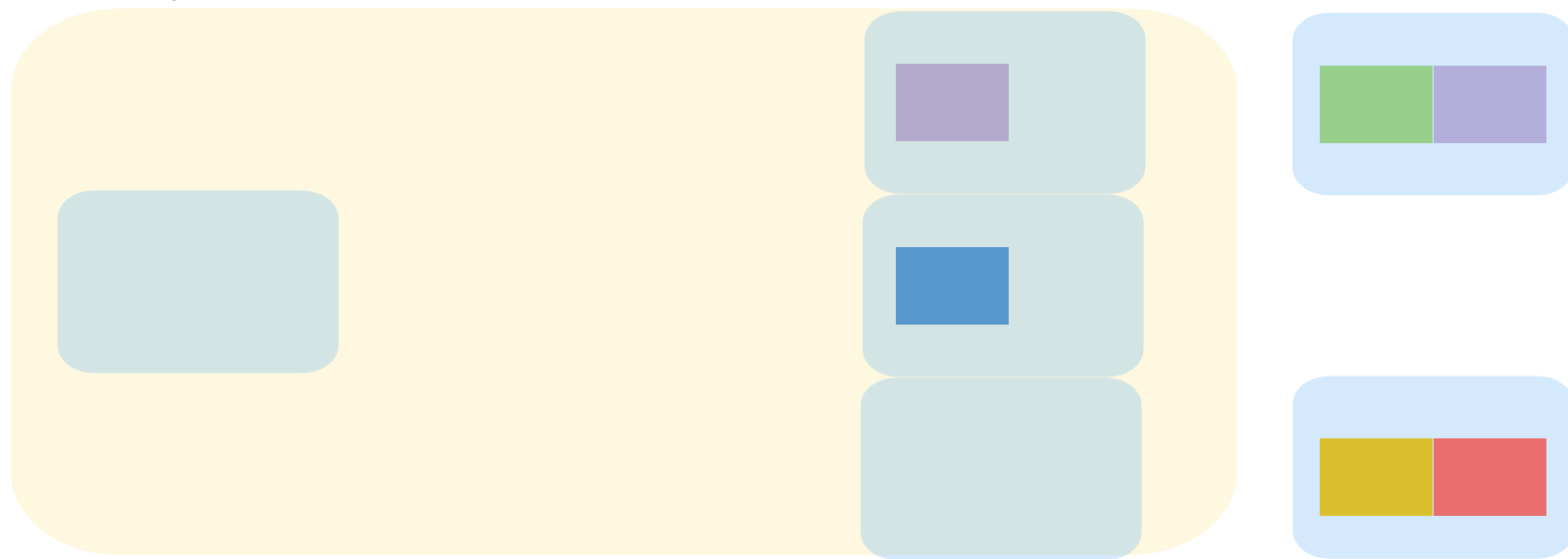


Pass 1: Divide

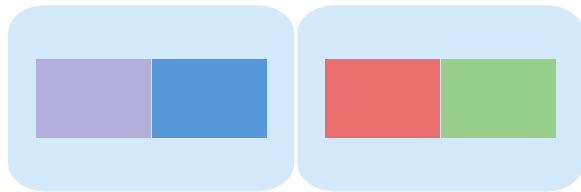


N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3

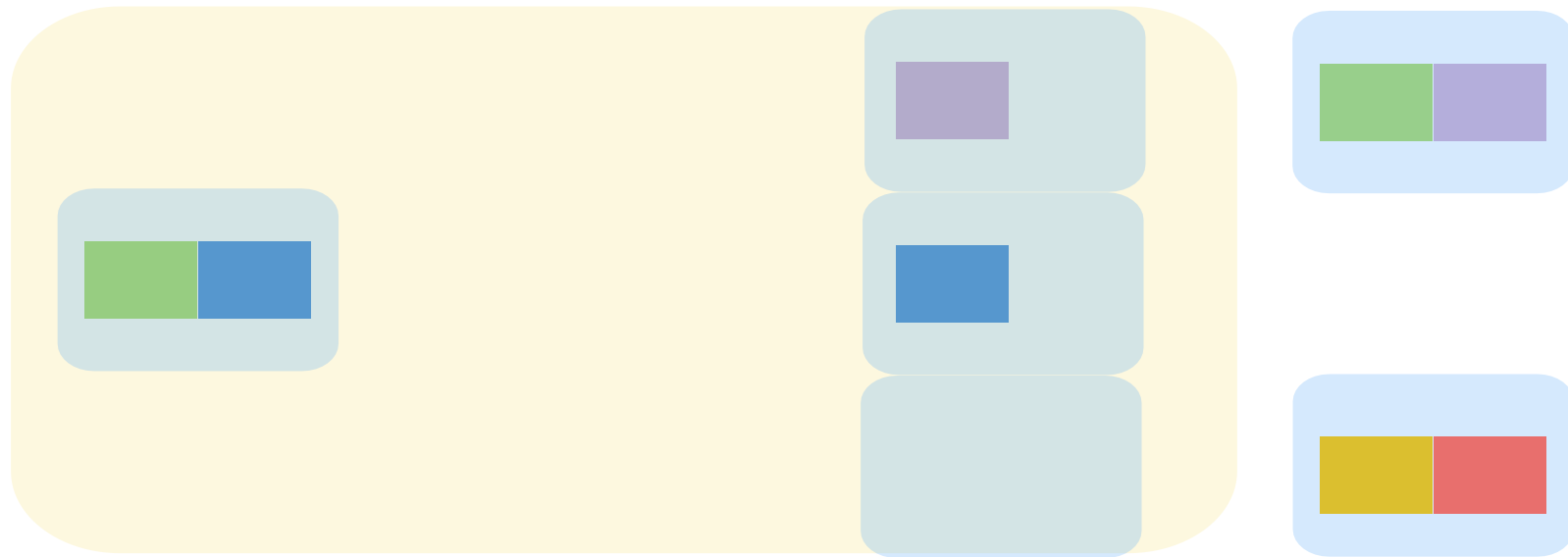


Pass 1: Divide

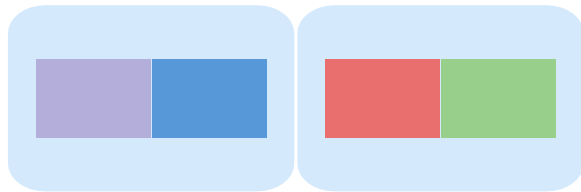


N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3

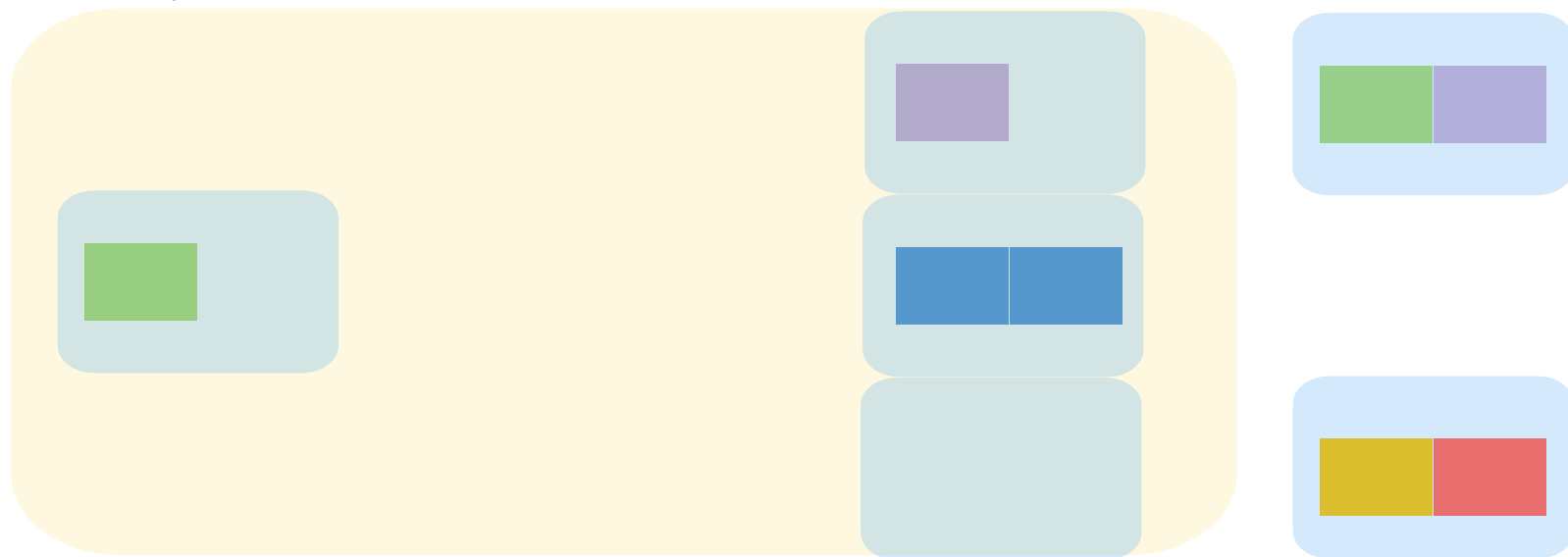


Pass 1: Divide

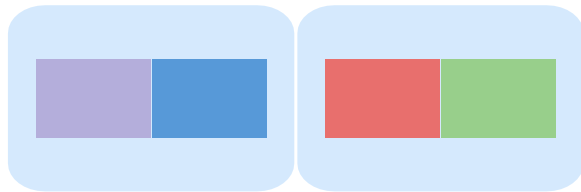


N=6, B=4

Our hash function: {G,P} $\rightarrow$ 1, {B} $\rightarrow$ 2, {R, Y} $\rightarrow$ 3

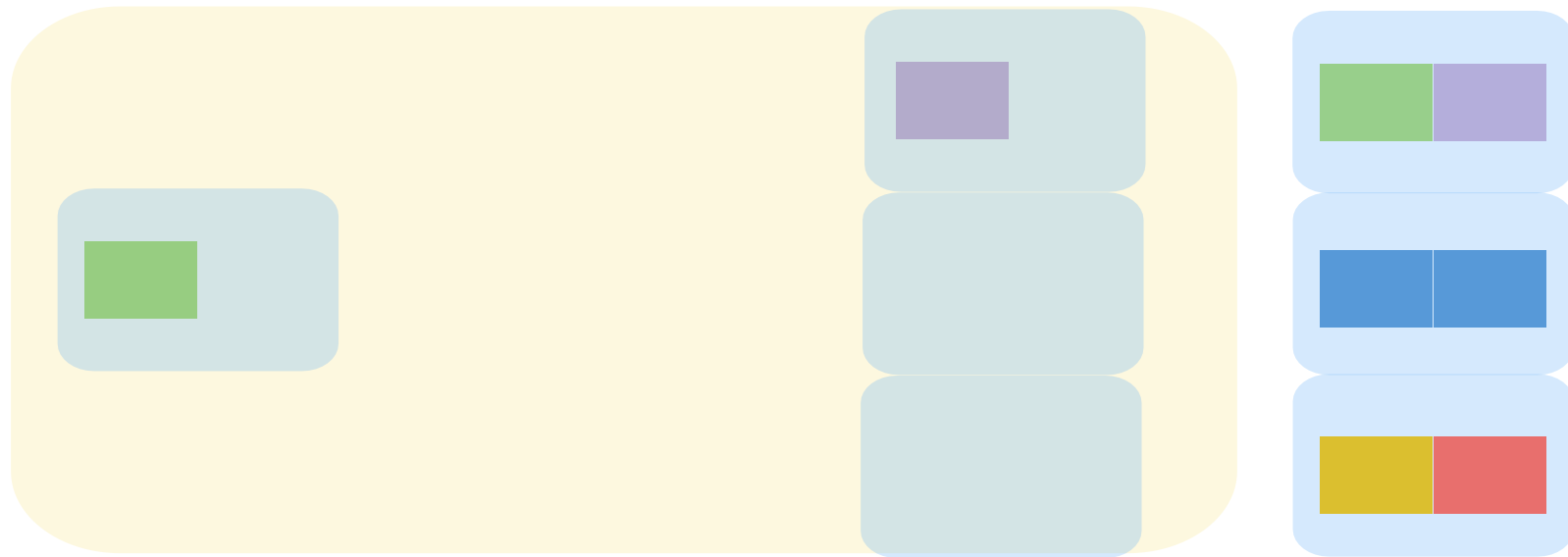


Pass 1: Divide

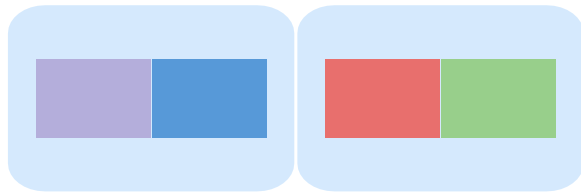


N=6, B=4

Our hash function: $\{G, P\} \rightarrow 1$, $\{B\} \rightarrow 2$, $\{R, Y\} \rightarrow 3$

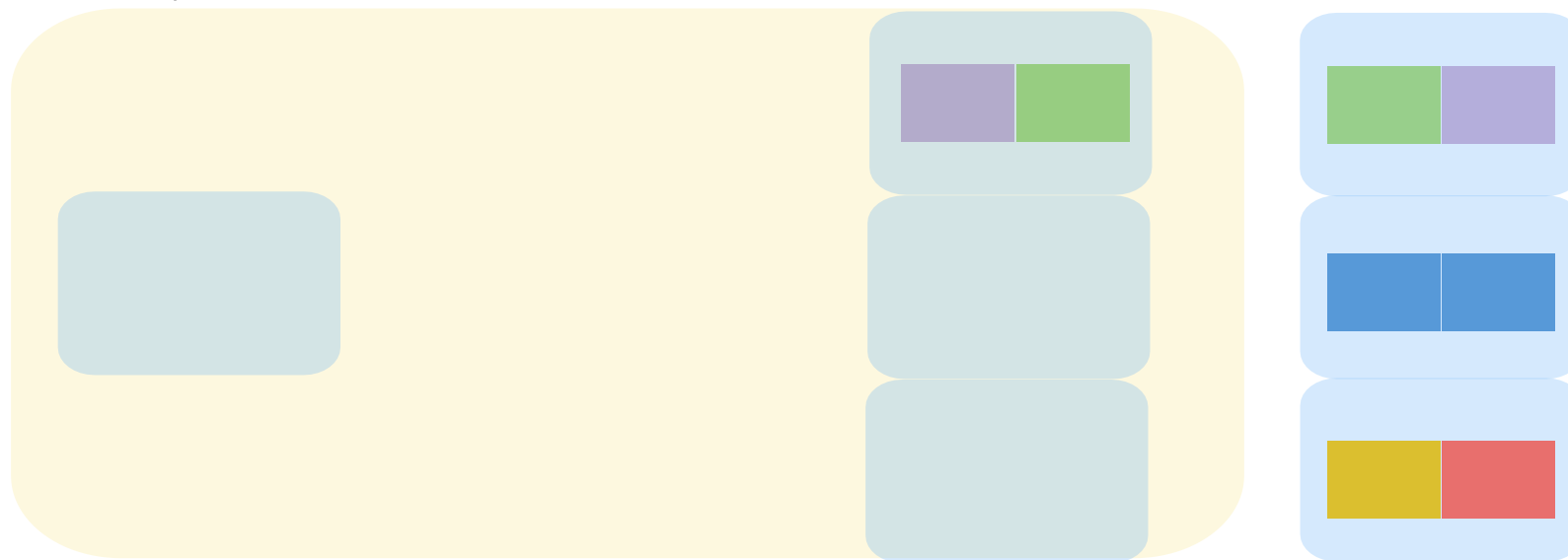


Pass 1: Divide

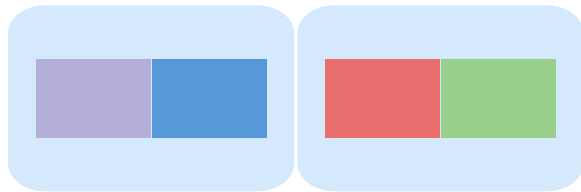


N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3

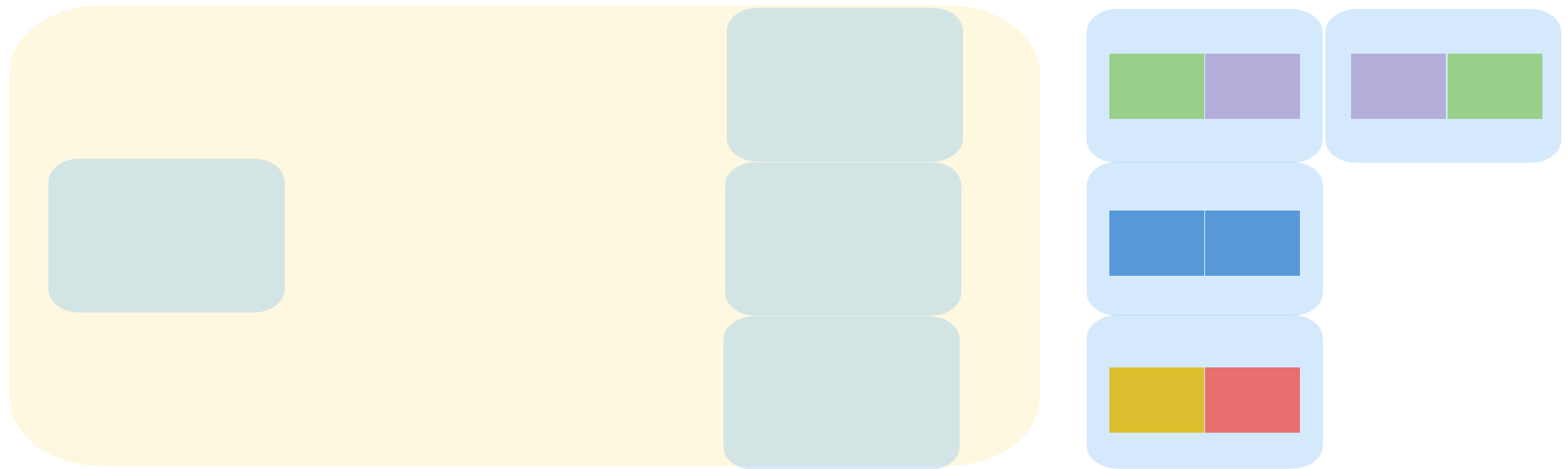


Pass 1: Divide

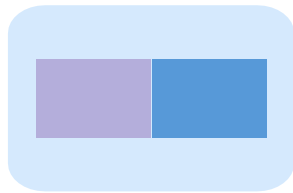


N=6, B=4

Our hash function: $\{G, P\} \rightarrow 1$, $\{B\} \rightarrow 2$, $\{R, Y\} \rightarrow 3$

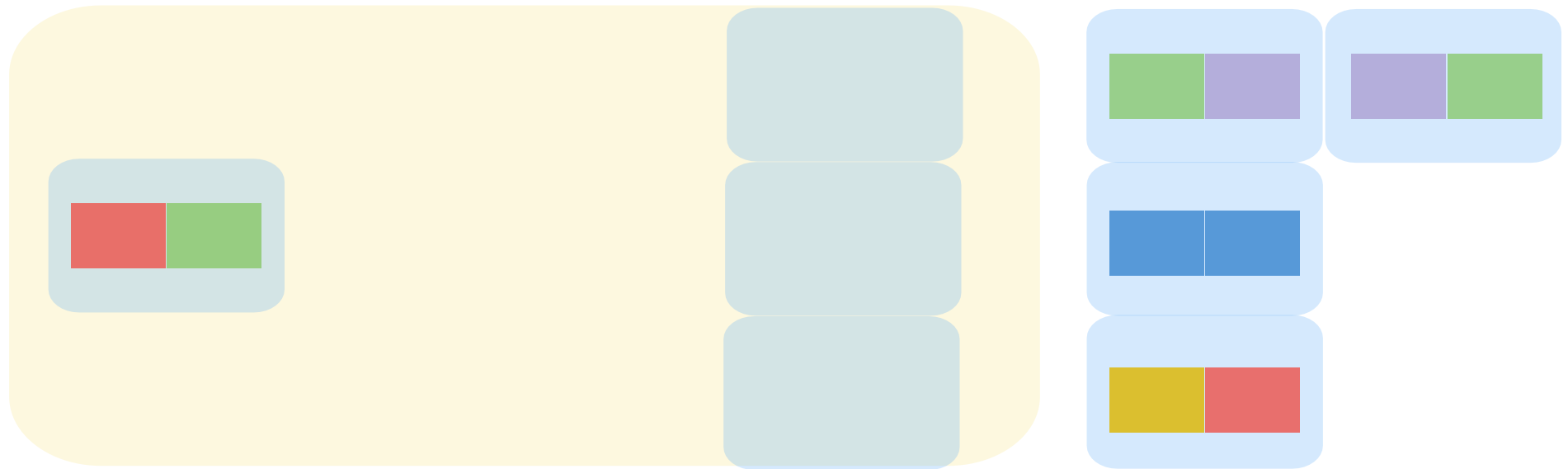


Pass 1: Divide

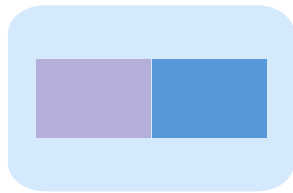


N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3

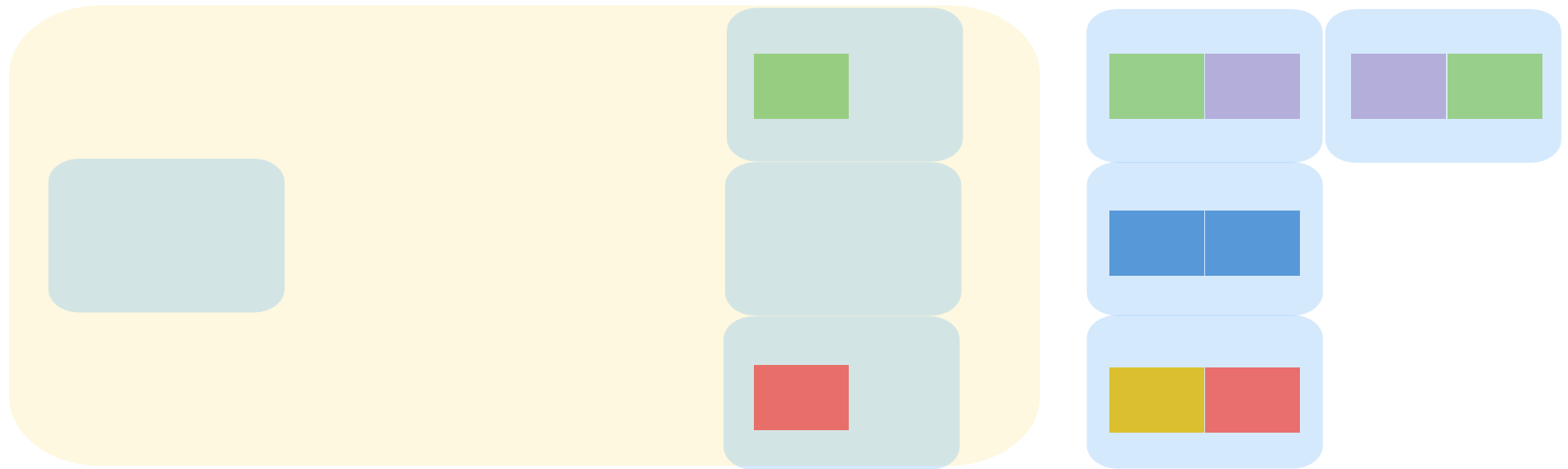


Pass 1: Divide



N=6, B=4

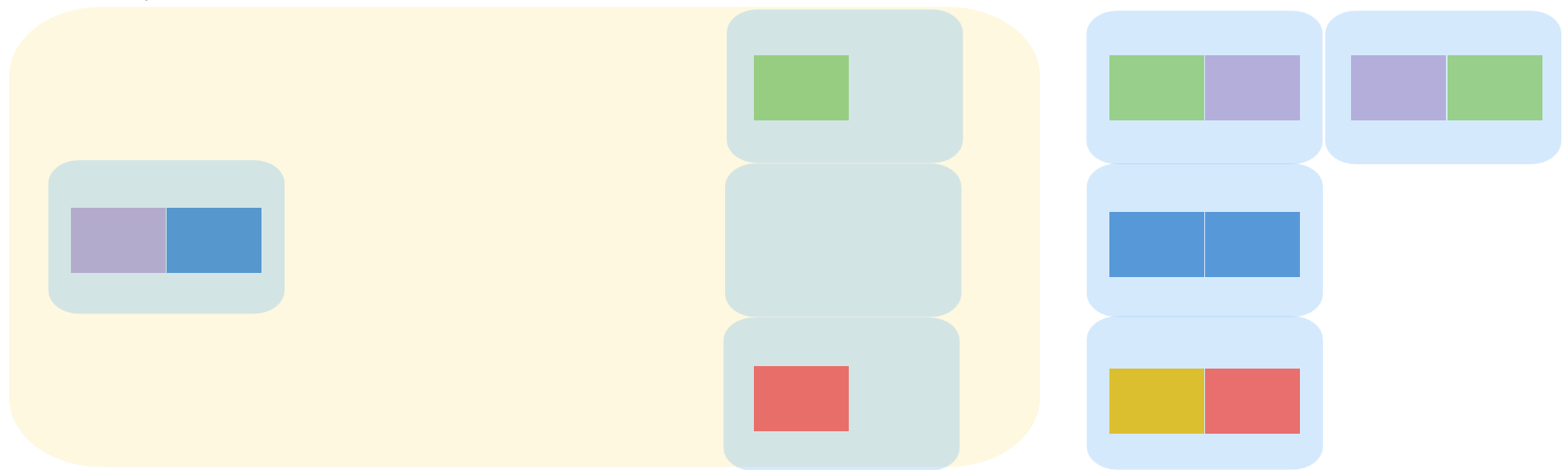
Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3



Pass 1: Divide

N=6, B=4

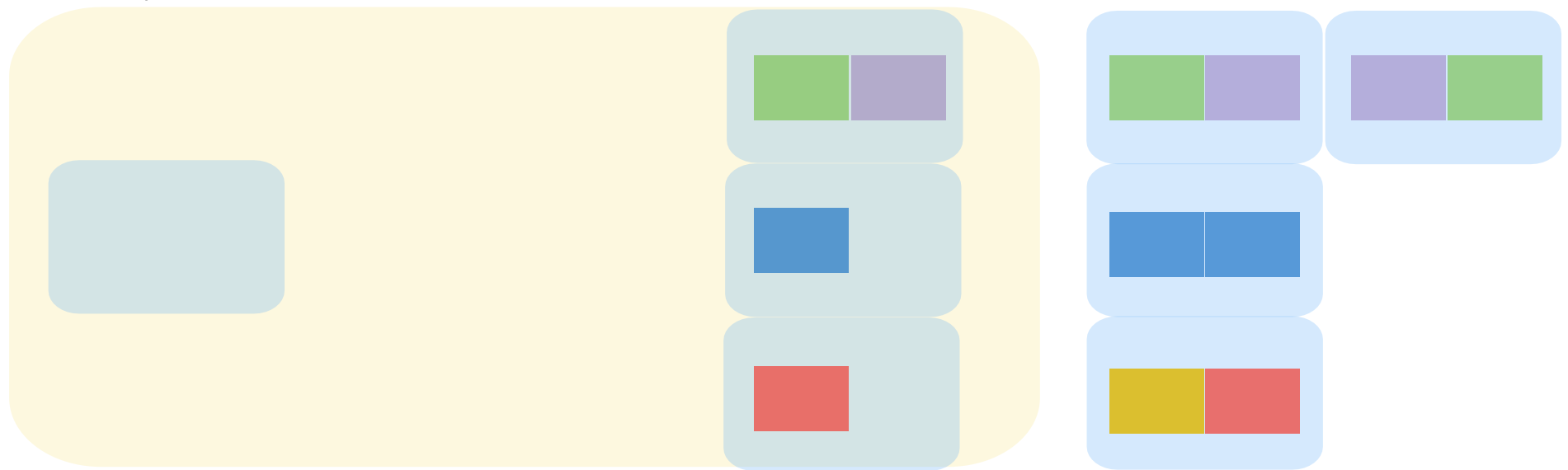
Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3



Pass 1: Divide

N=6, B=4

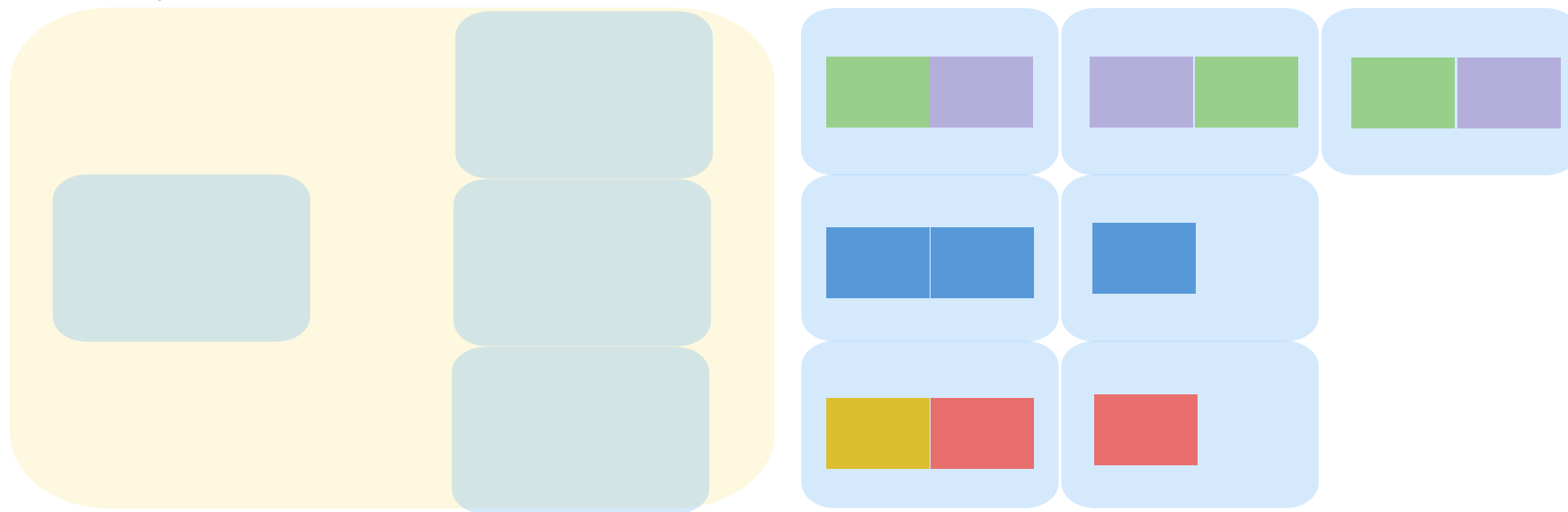
Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3



Pass 1: Divide

N=6, B=4

Our hash function: {G,P} -> 1, {B} -> 2, {R, Y} -> 3



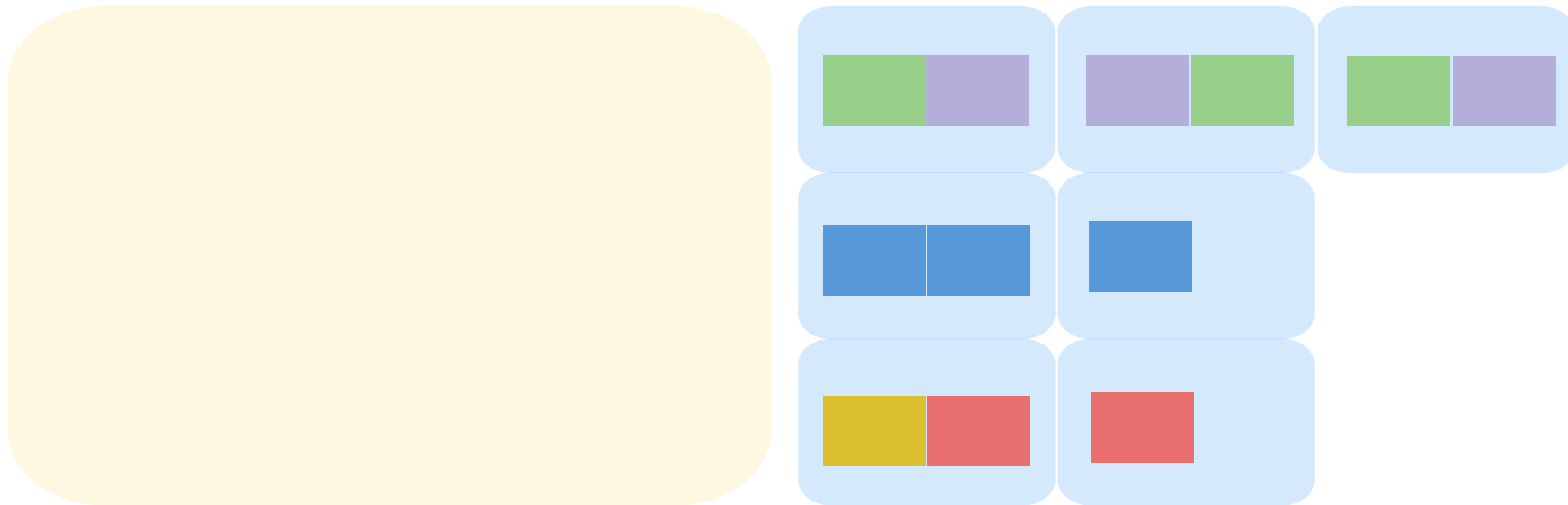
Pass 2: Conquer

- Rehash each partition
- For a partition to fit in memory, it can only have B pages
- If a partition is too large... repartition!
 - Use the partition algorithm recursively until the partition fits into memory
- $\# \text{ I/O's} = 2N$

Pass 2: Conquer

Create in-memory table for each partition.

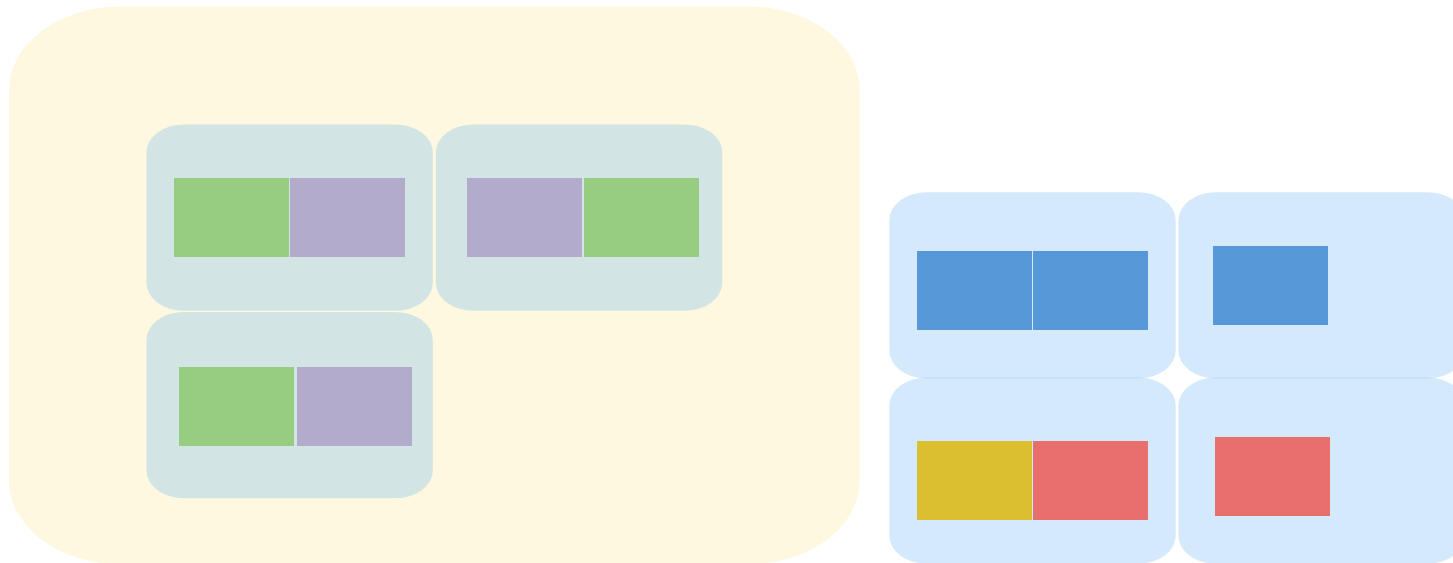
N=6, B=4



Pass 2: Conquer

Create in-memory table for each partition.

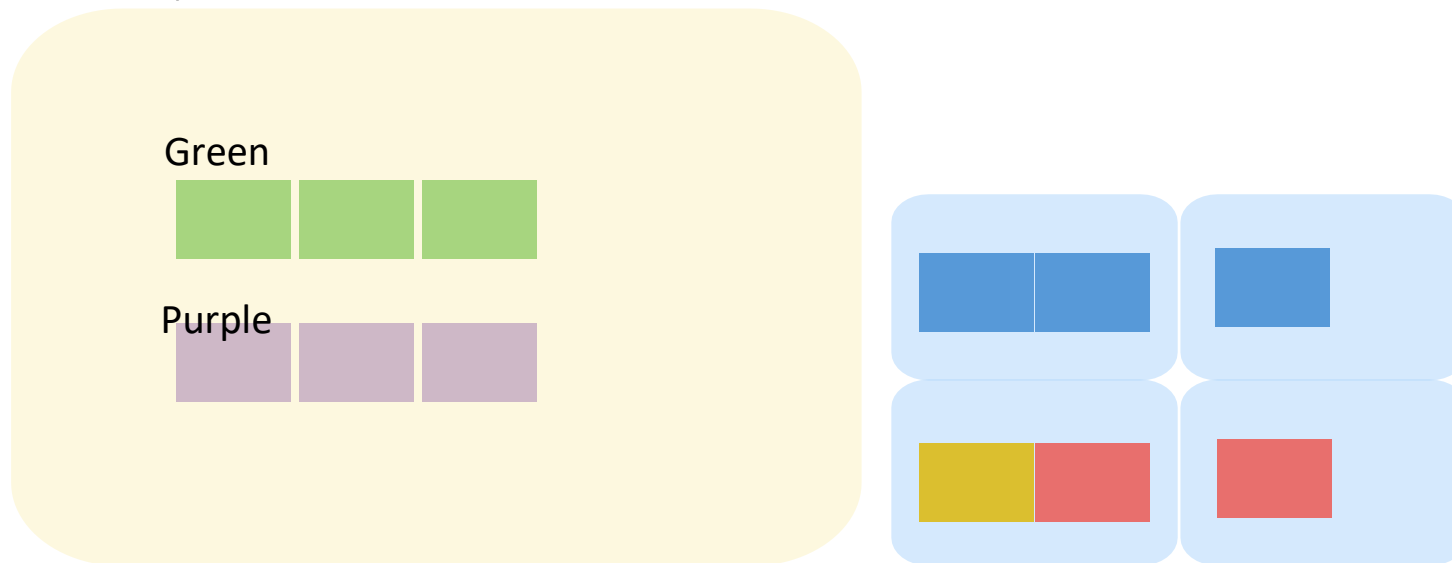
N=6, B=4



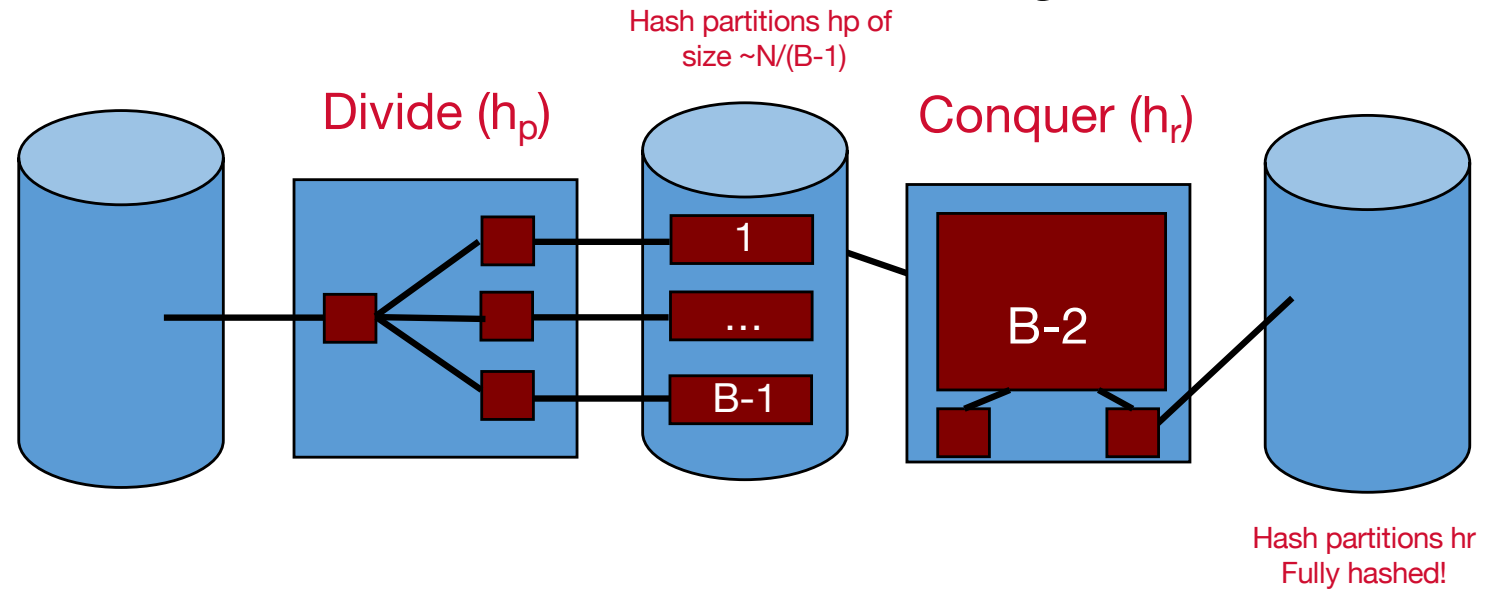
Pass 2: Conquer

Create in-memory table for each partition.

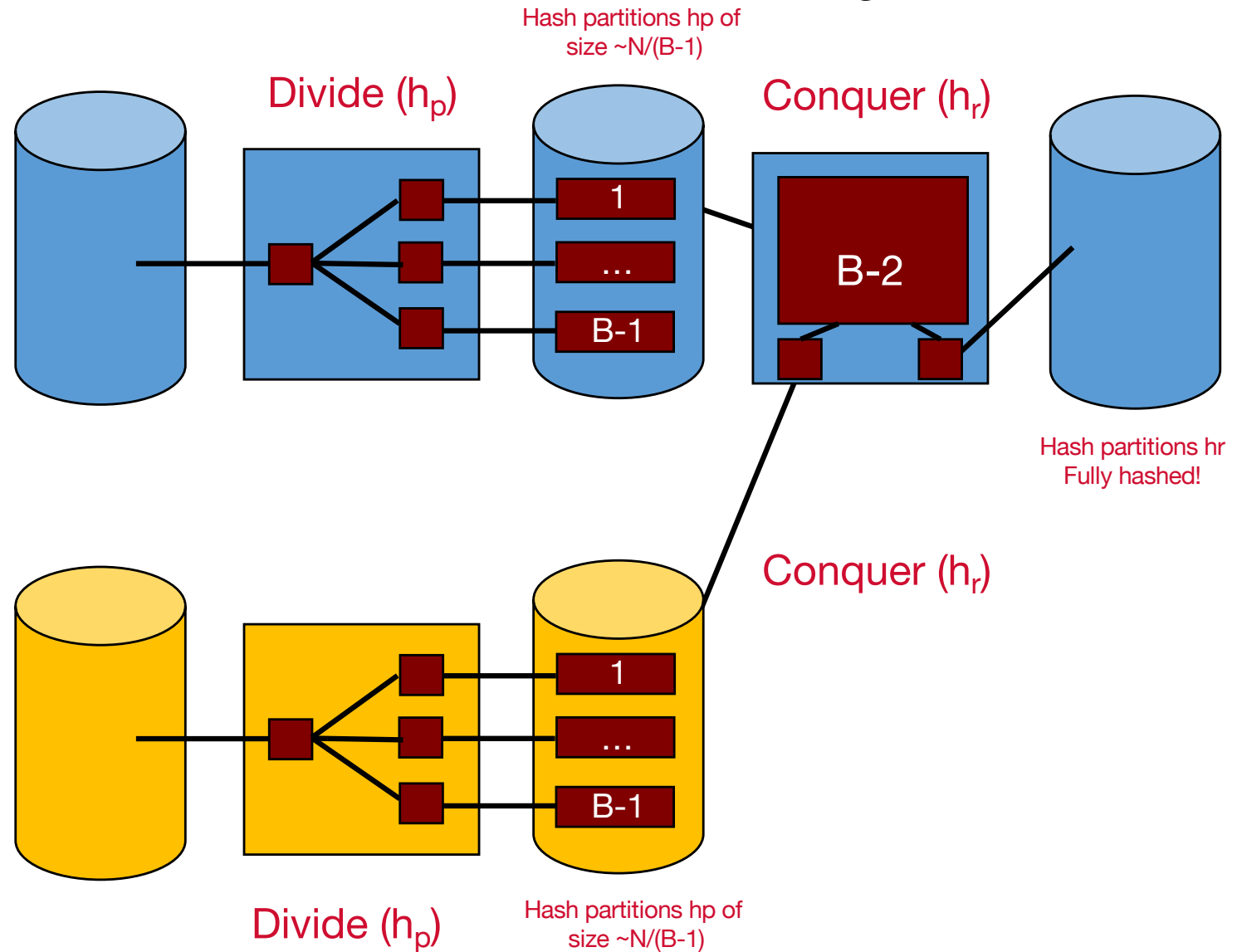
N=6, B=4



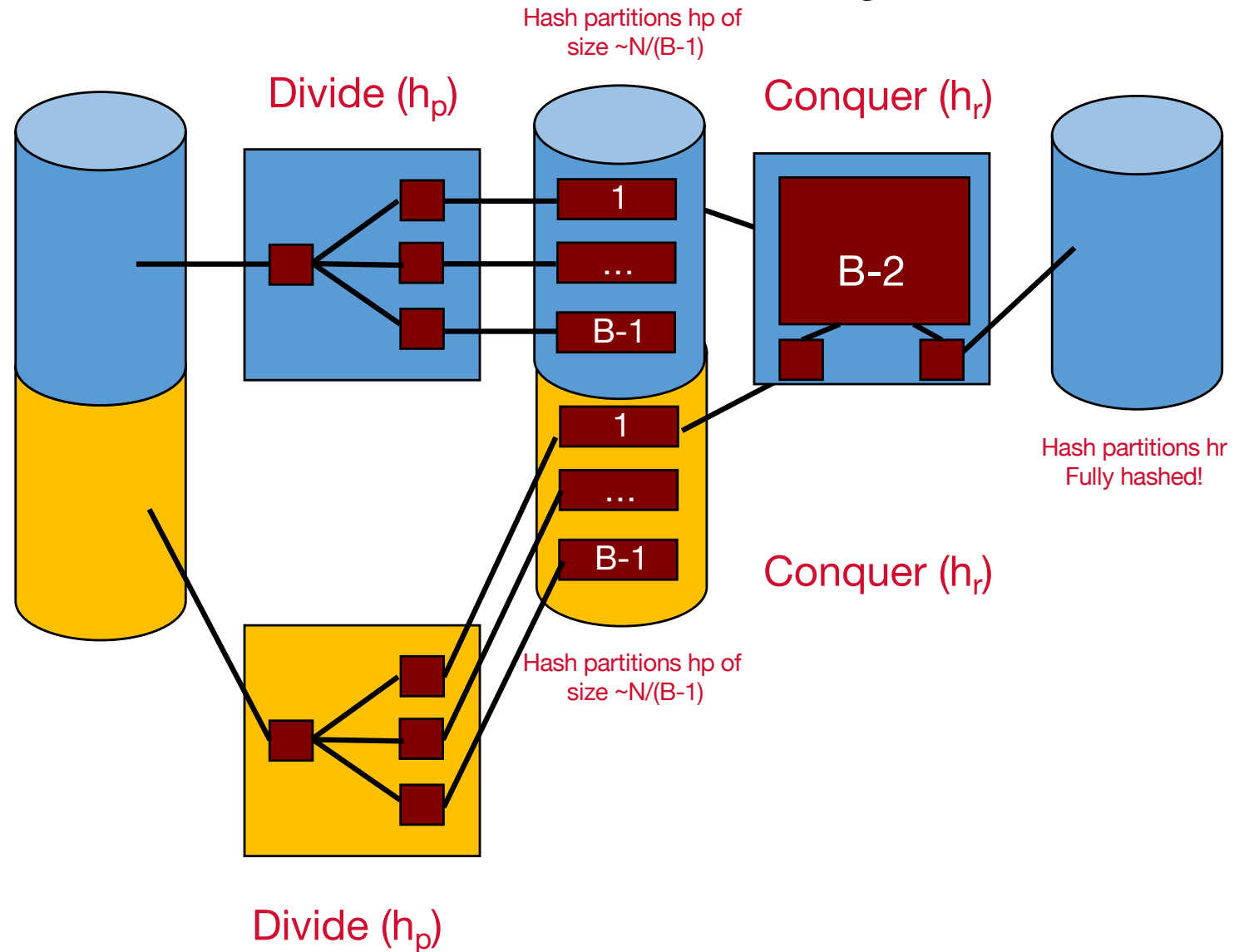
Sketch of Grace hash join



Sketch of Grace hash join

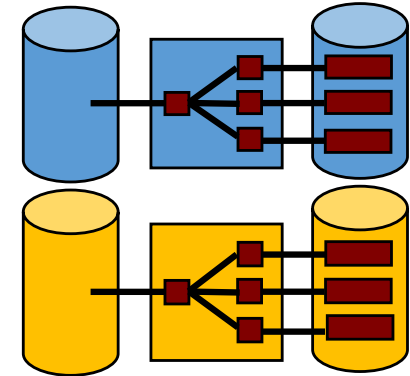


Sketch of Grace hash join



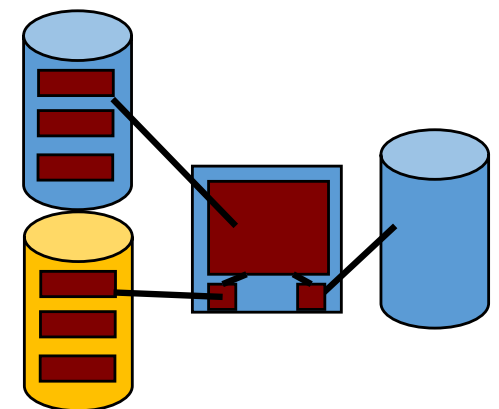
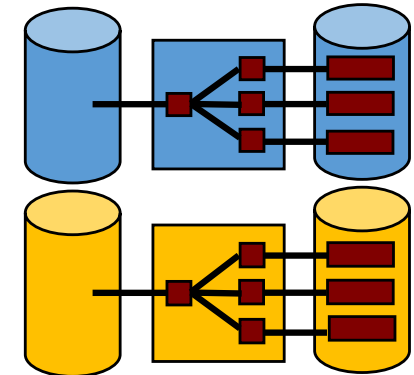
Pseudo-code of Grace hash

- For Cur in {R, S}
 - For page in Cur
 - Read page into input buffer
 - For tup on page
 - Place tup in output buf $\text{hash}_p(\text{tup.joinkey})$
 - If output buf full then flush to disk partition
 - Flush output bufs to disk partitions

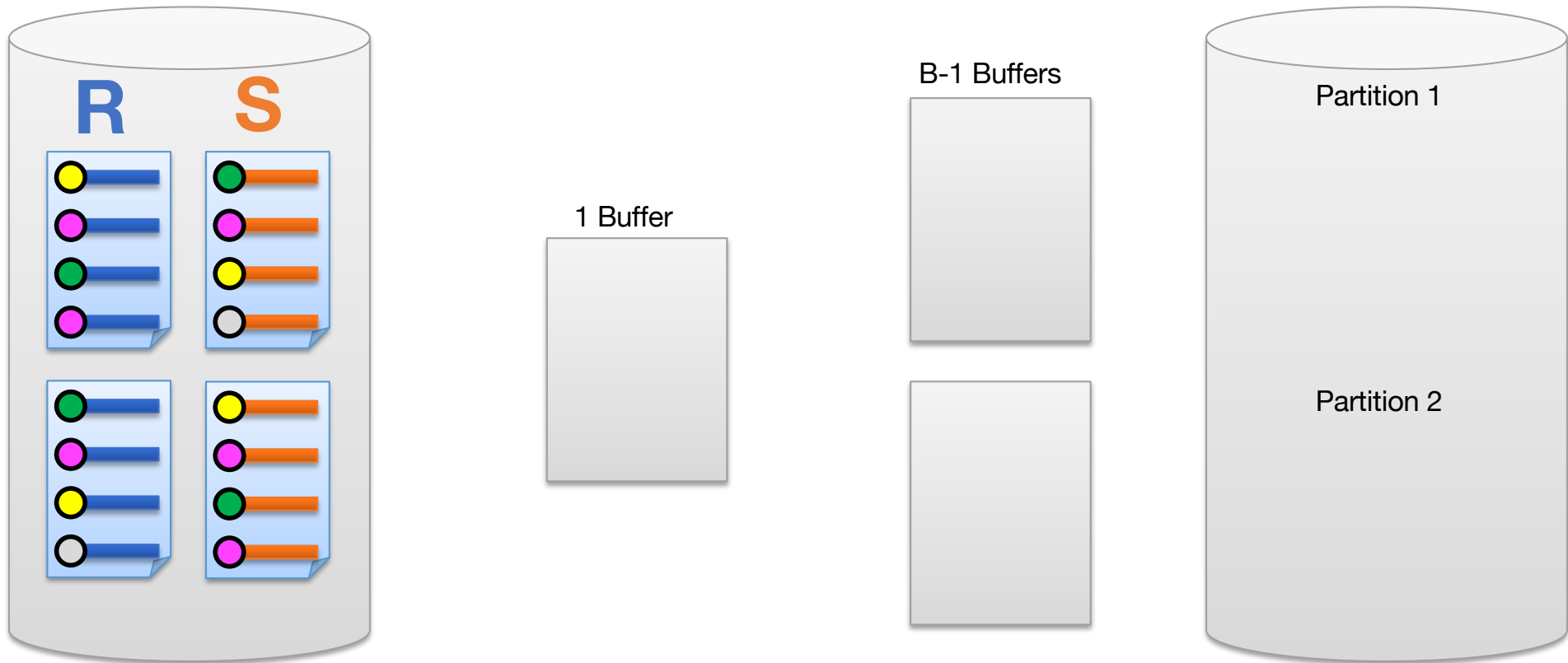


Pseudo-code of grace hash

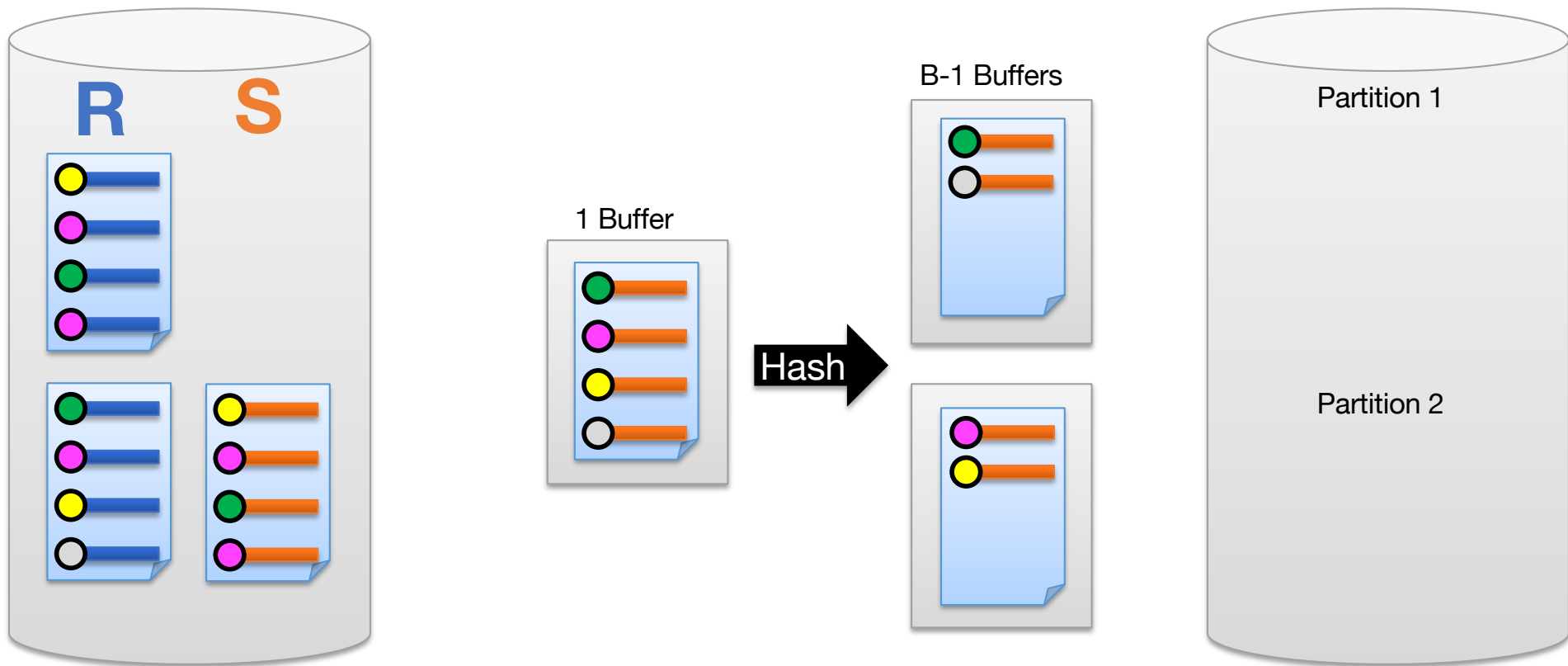
- For Cur in {R, S}
 - For page in Cur
 - Read page into input buffer
 - For tup on page
 - Place tup in output buf $\text{hash}_p(\text{tup.joinkey})$
 - If output buf full then flush to disk partition
 - Flush output bufs to disk partitions
- For i in $[0..(B-1))$
 - For page in R_i
 - For tup on page
 - Build tup in memory $\text{hash}_r(\text{tup.joinkey})$
 - For page in S_i
 - For tup on page
 - Read page into input buffer
 - Probe memory $\text{hash}_r(\text{tup.joinkey})$ for matches
 - Send all matches to output buffer
 - Flush output buffer if full



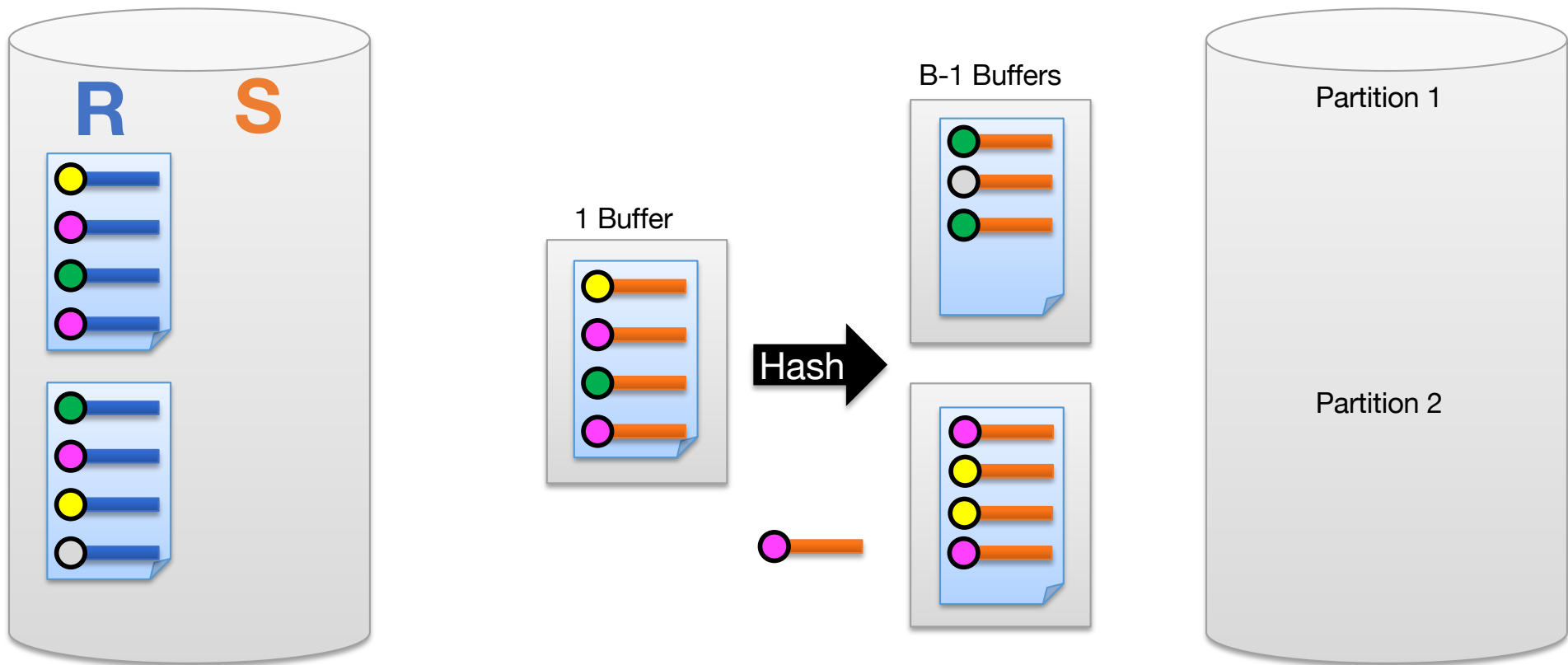
Grace hash join: *partition*



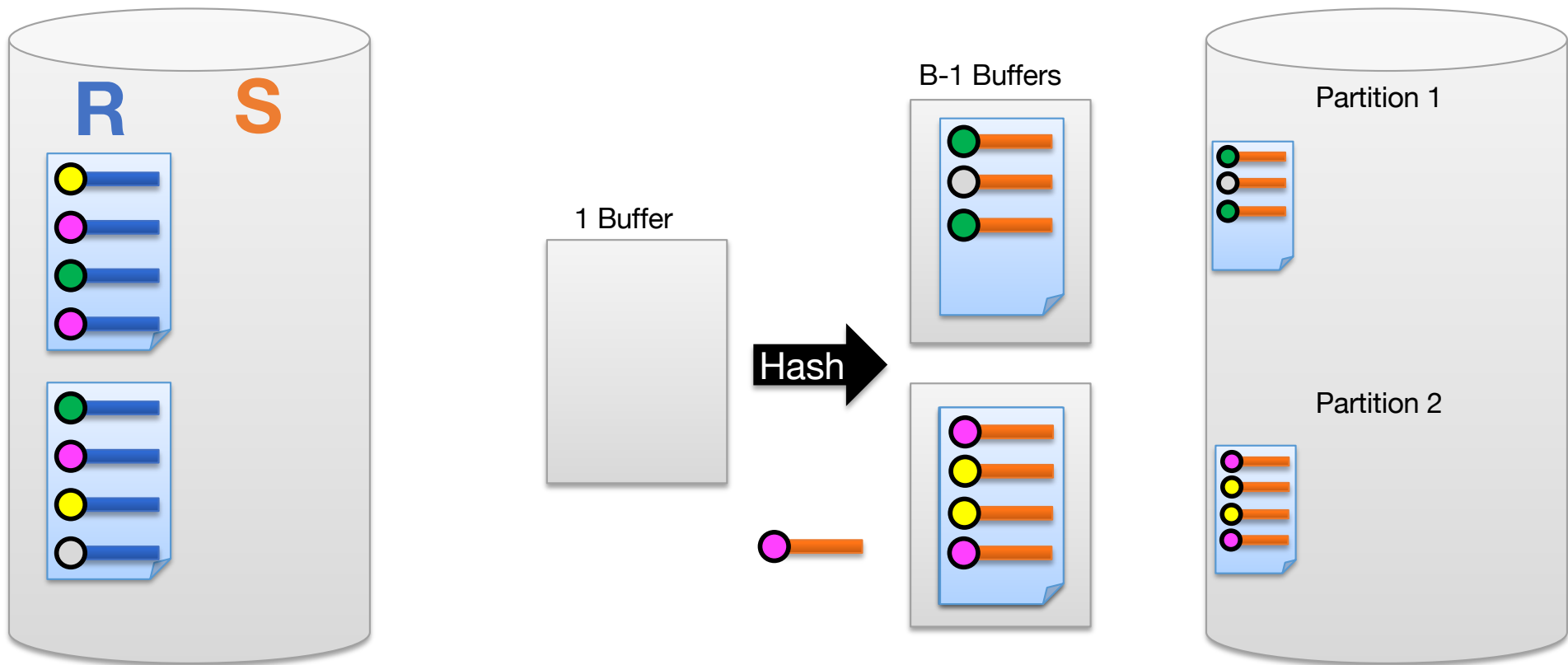
Grace hash join: *partition*



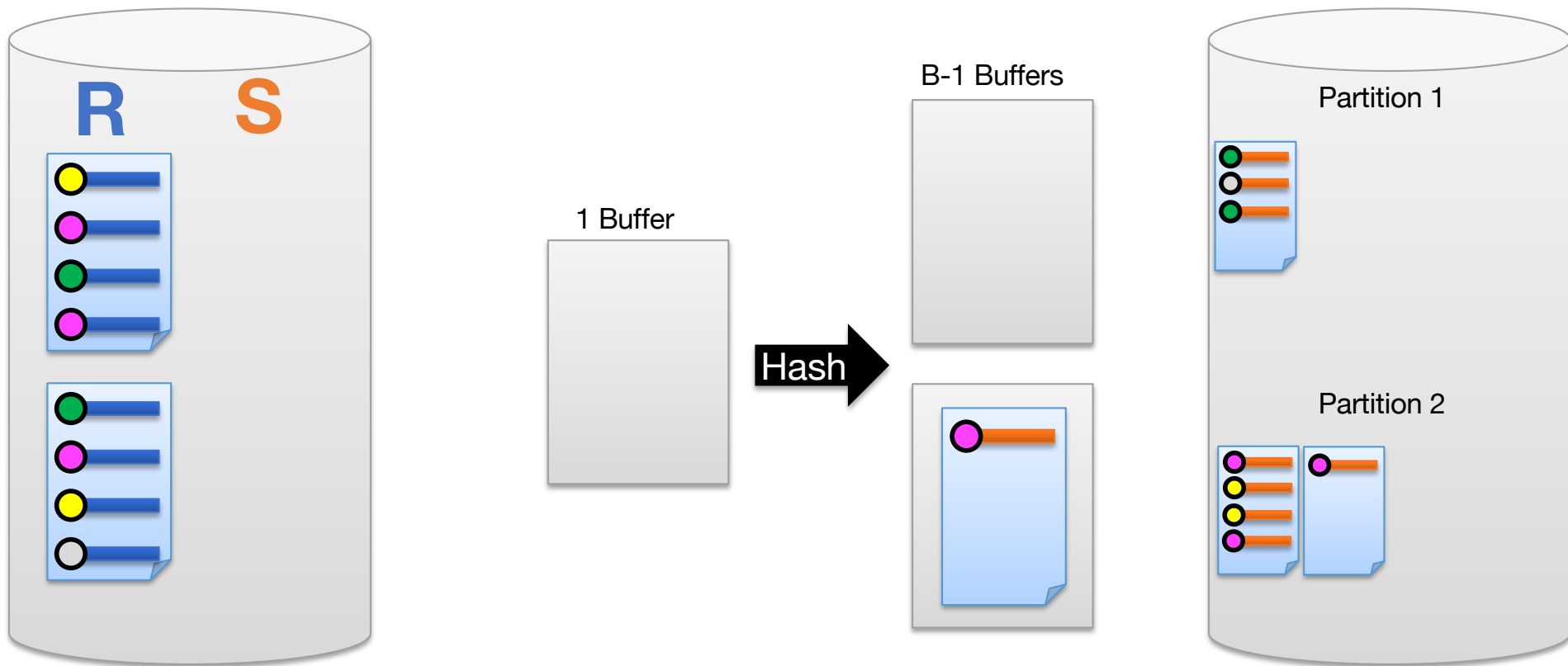
Grace hash join: *partition*



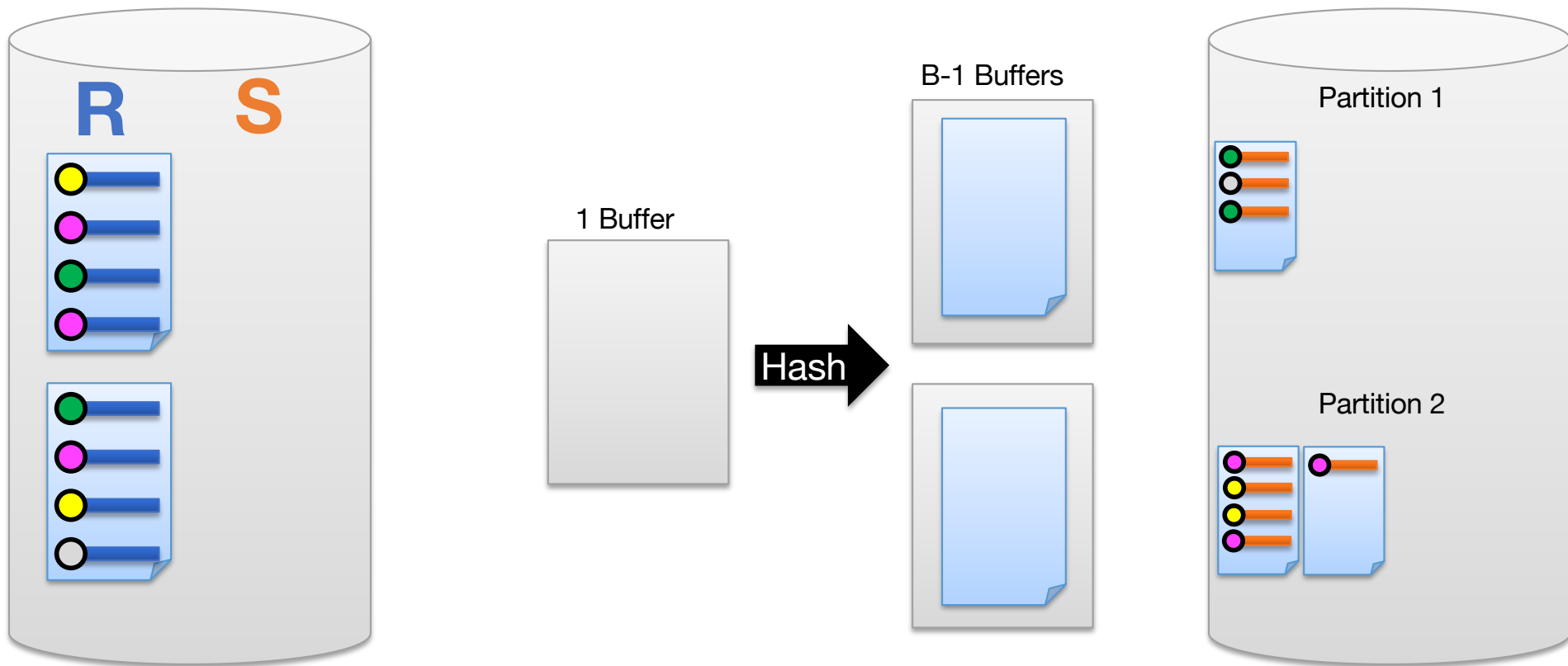
Grace hash join: *partition*



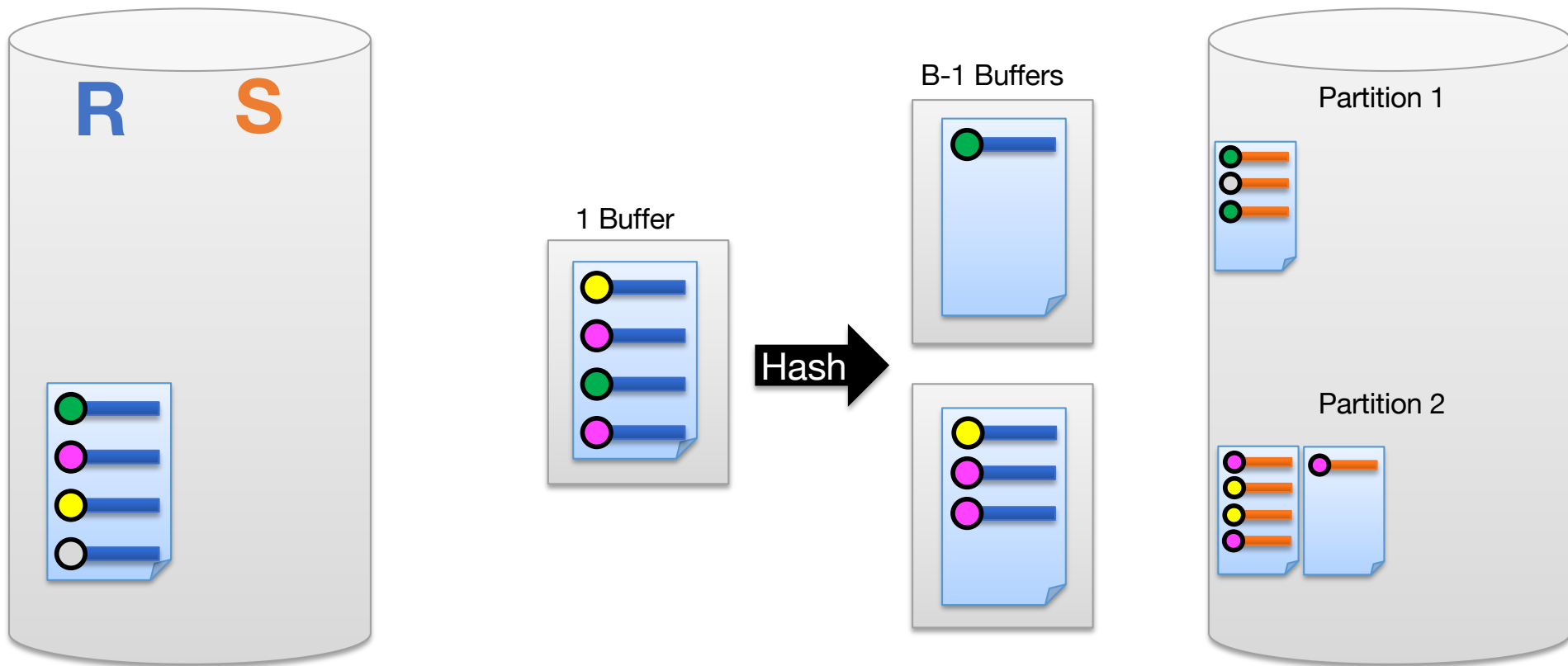
Grace hash join: *partition*



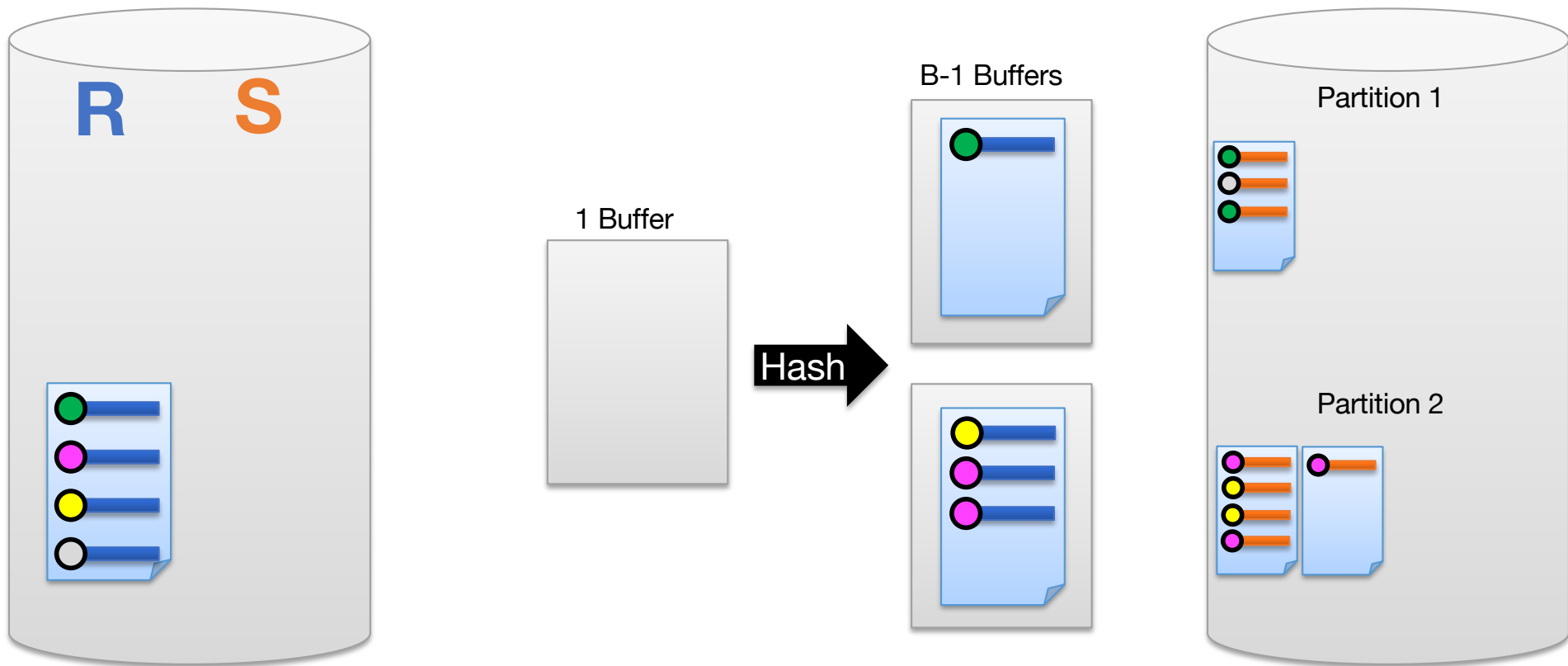
Grace hash join: *partition*



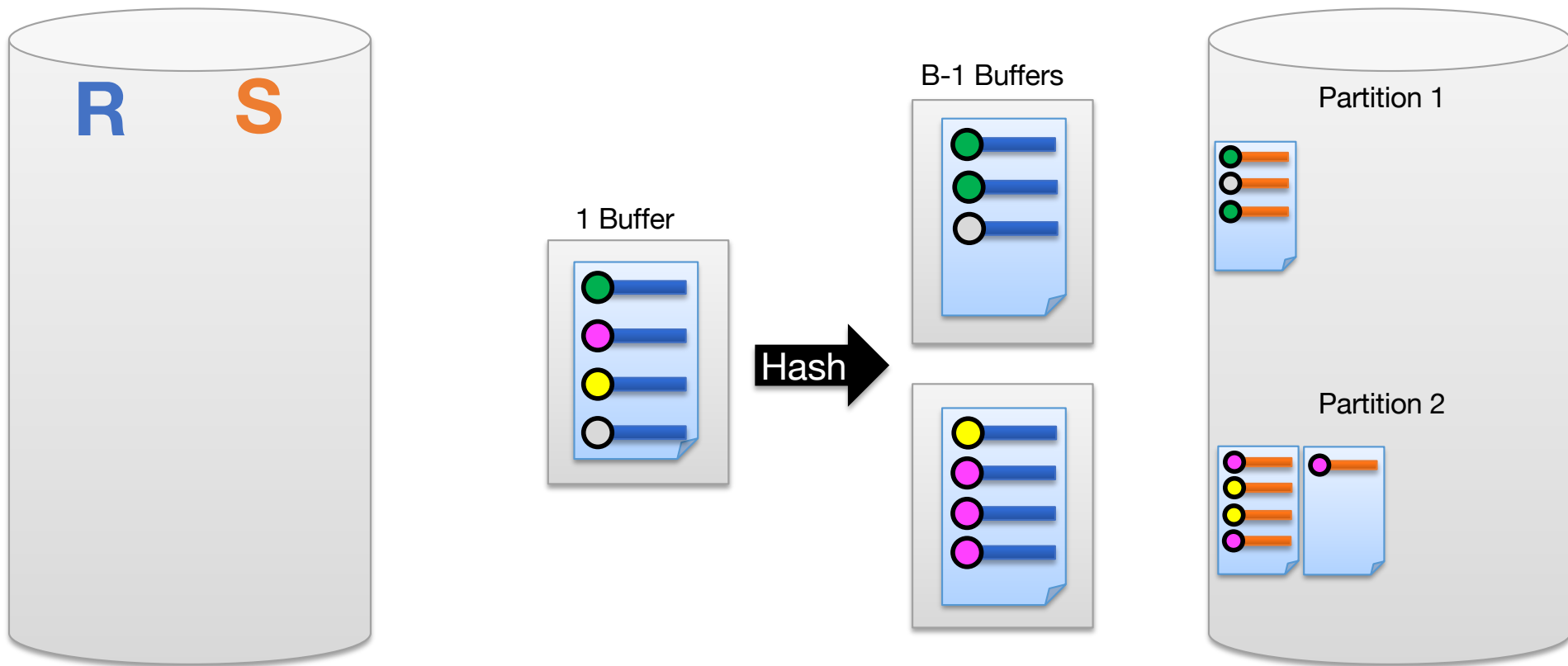
Grace hash join: *partition*



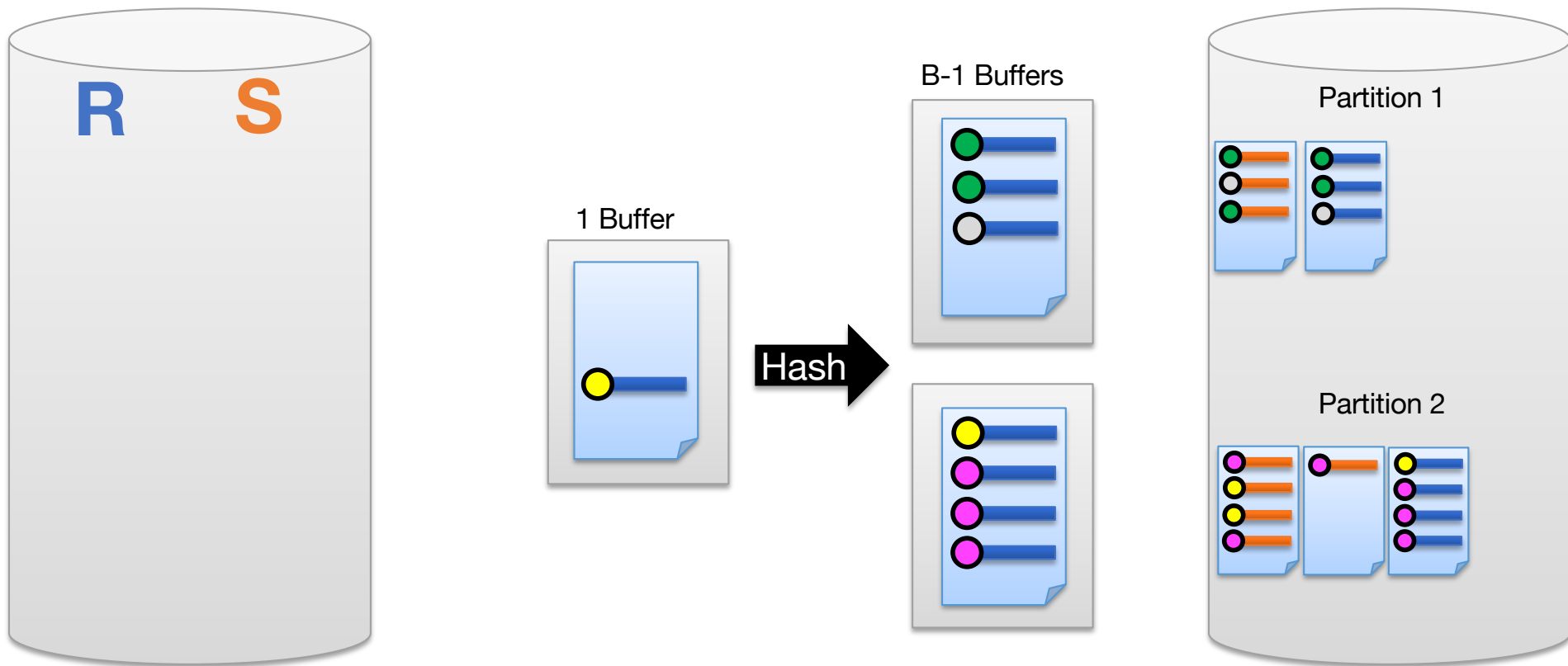
Grace hash join: *partition*



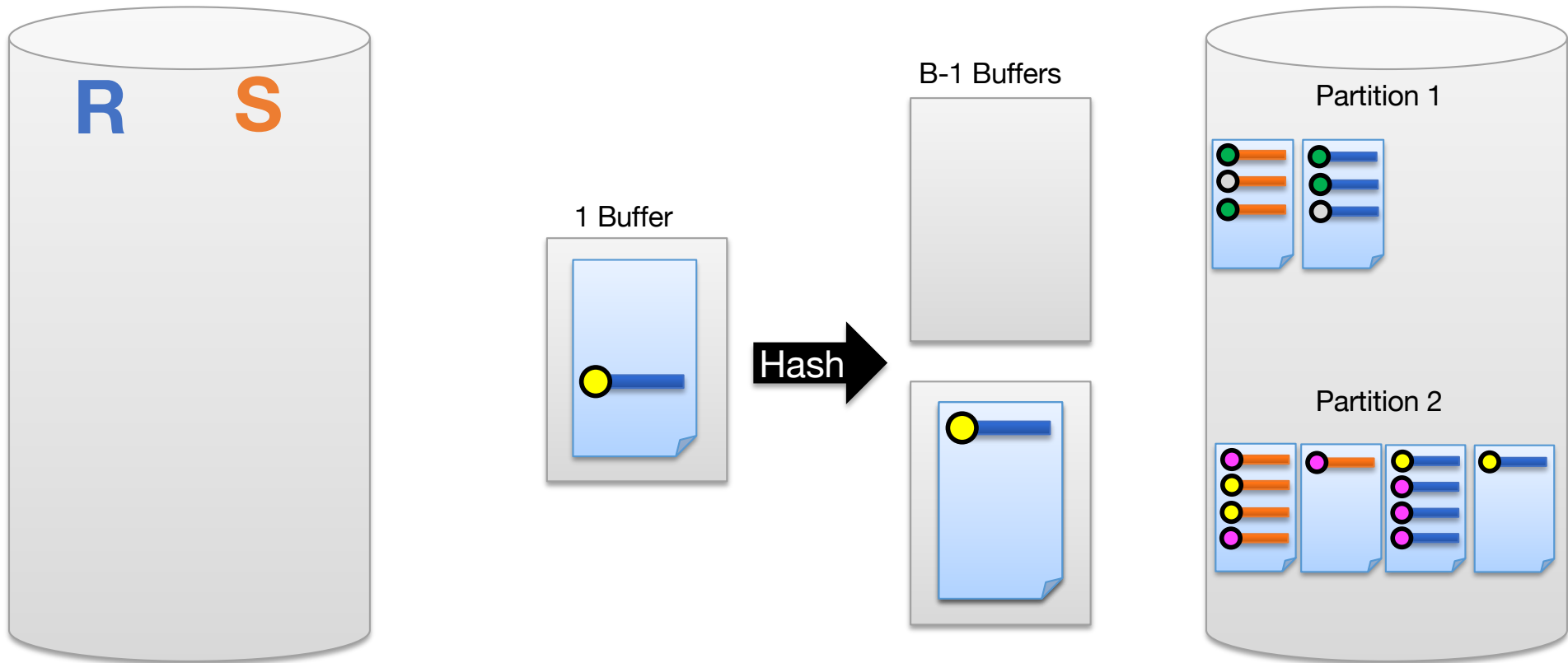
Grace hash join: *partition*



Grace hash join: *partition*

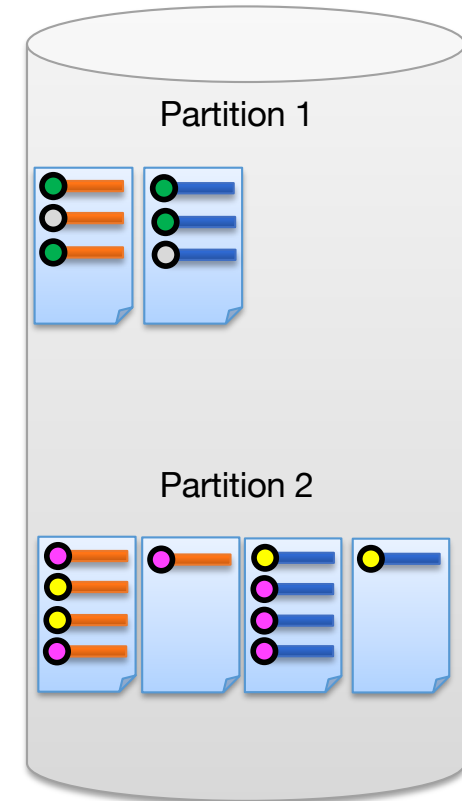


Grace hash join: *partition*

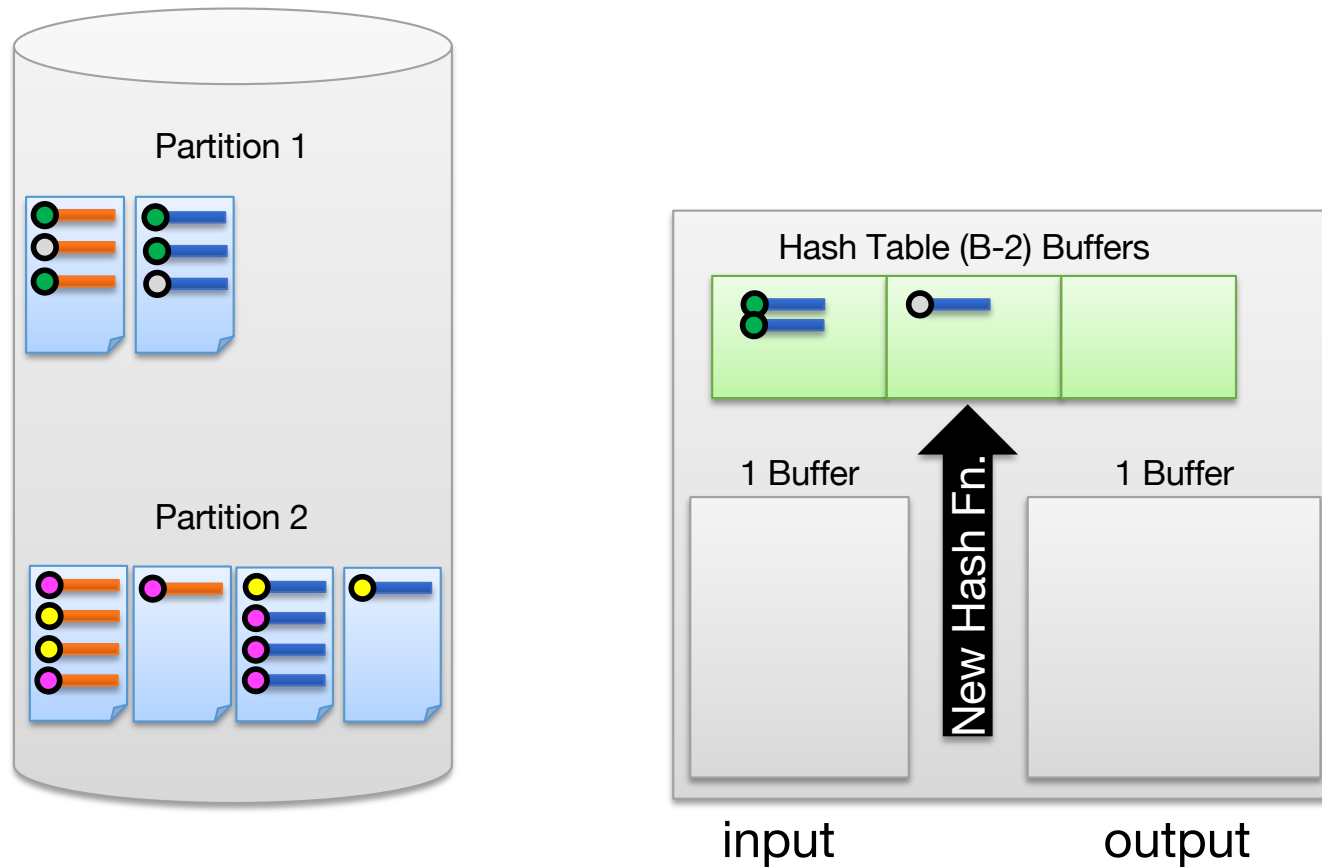


Grace hash join: *partition*

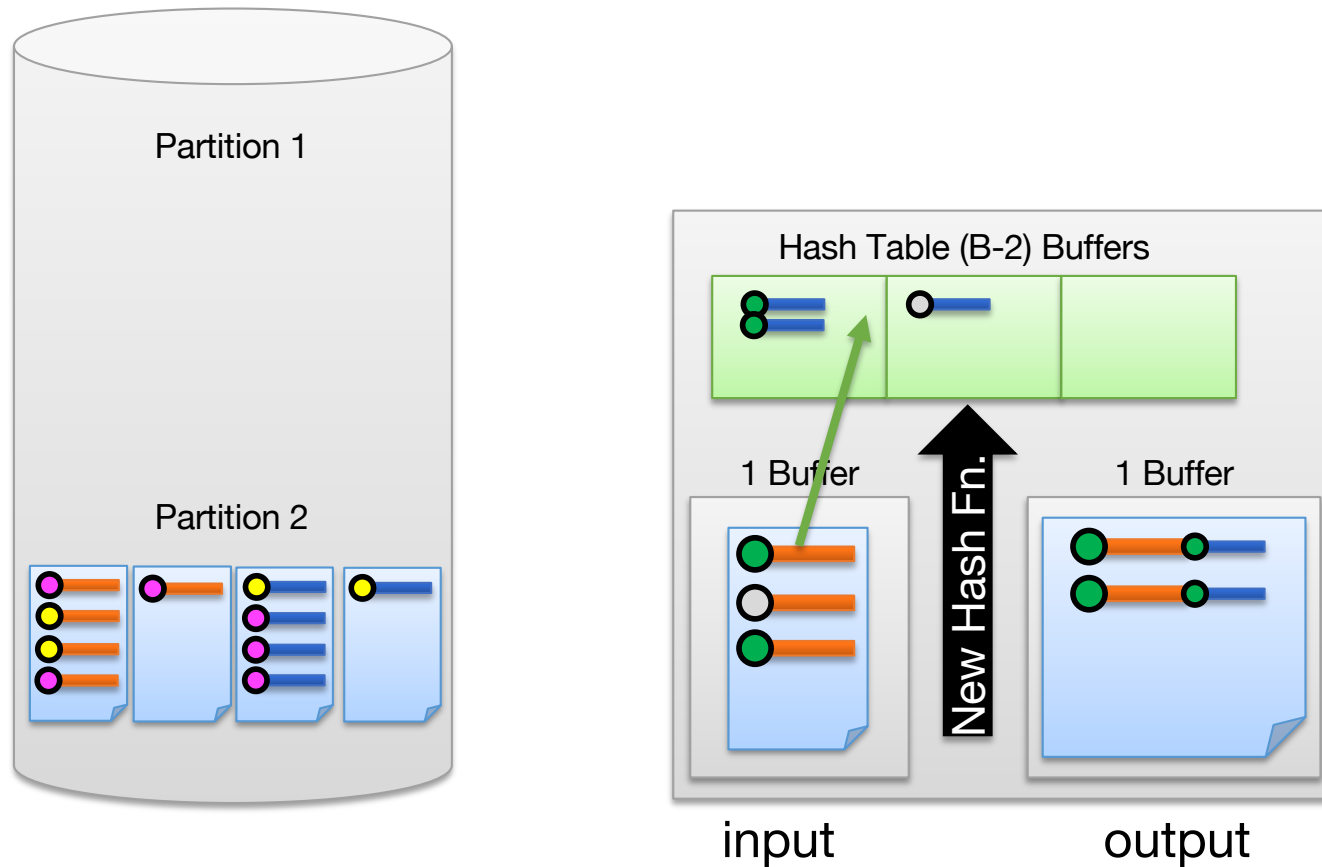
- Each key is assigned to one partition
 - e.g., **green** keys only here →
- Sensitive to key **skew**
 - **purple** key
- Each partition could be different machine



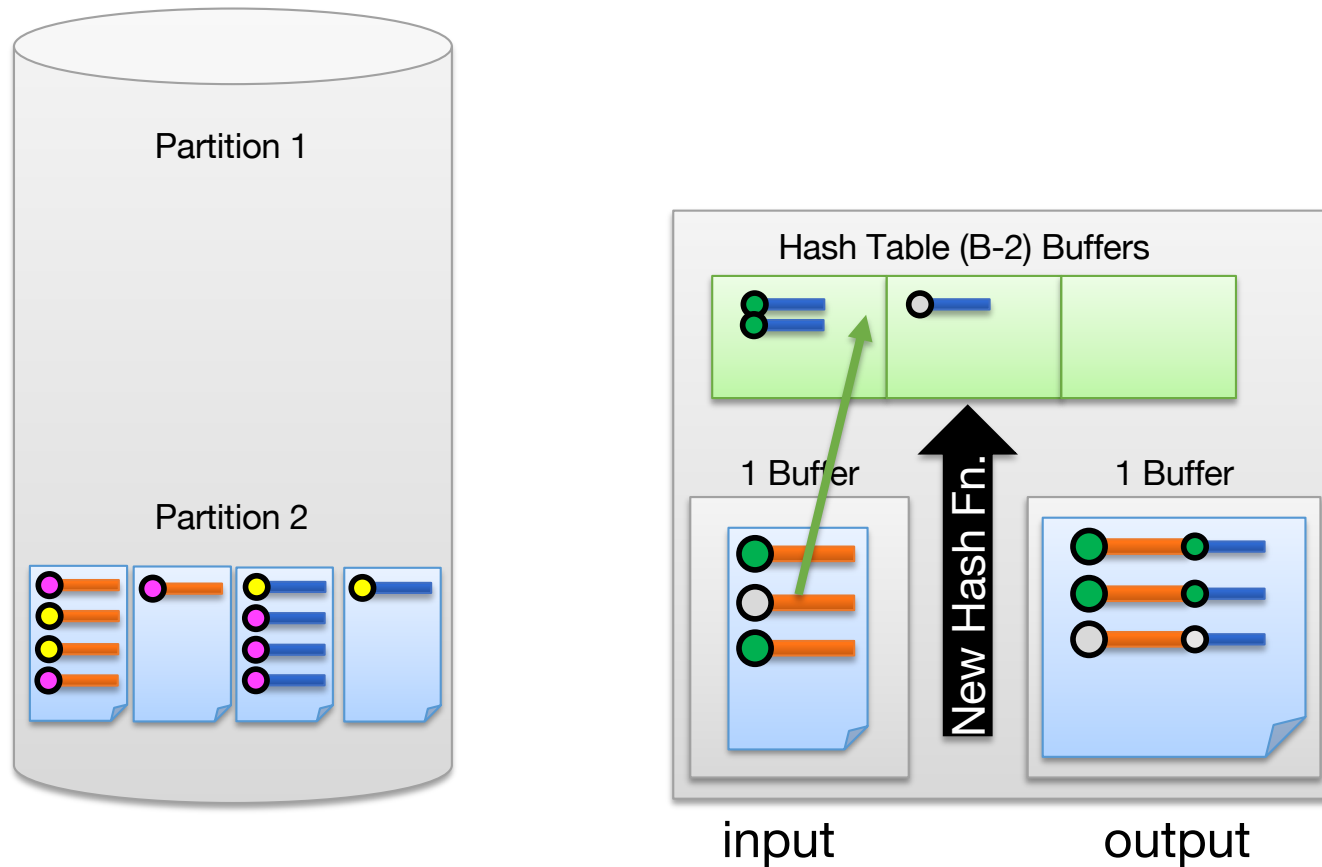
Grace hash join: build & probe



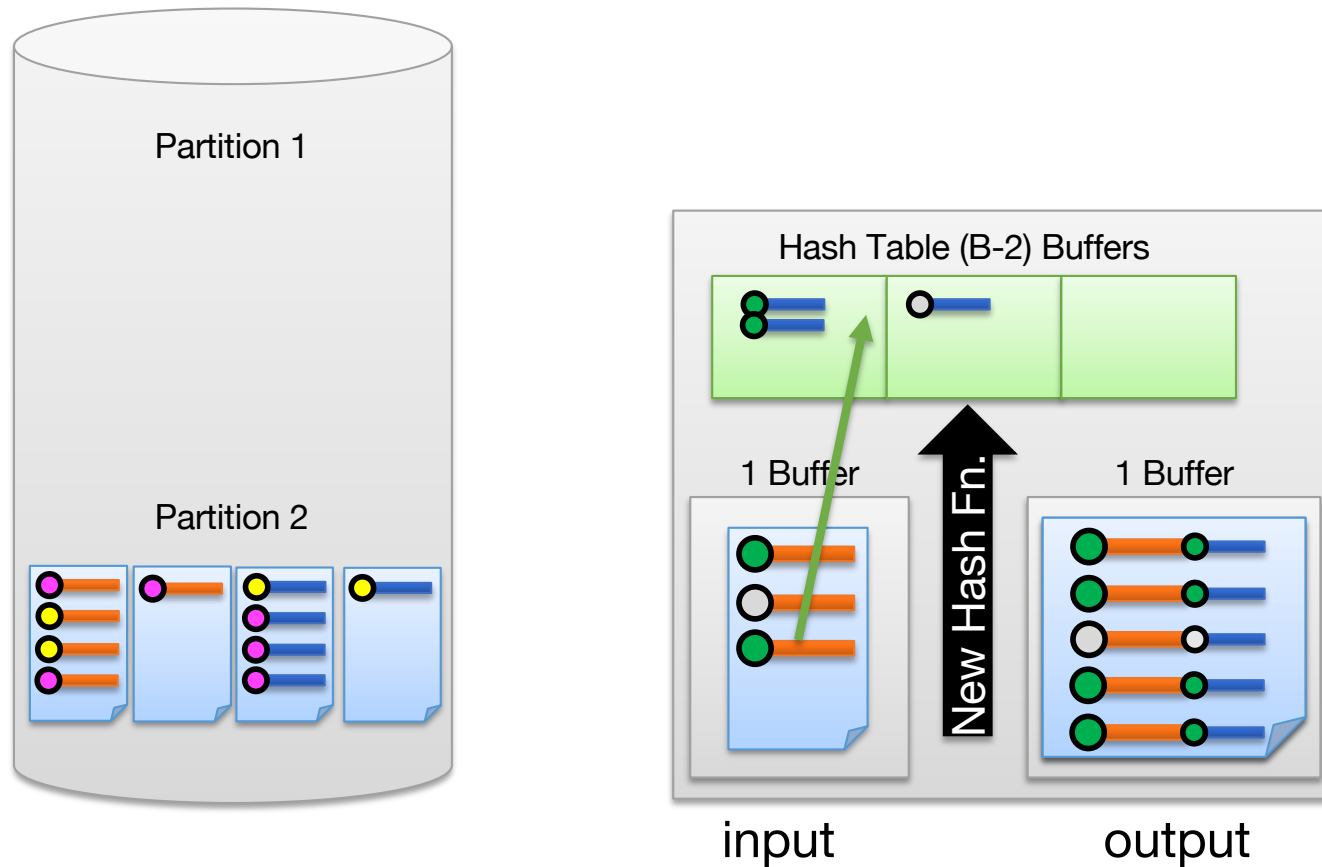
Grace hash join: build & probe



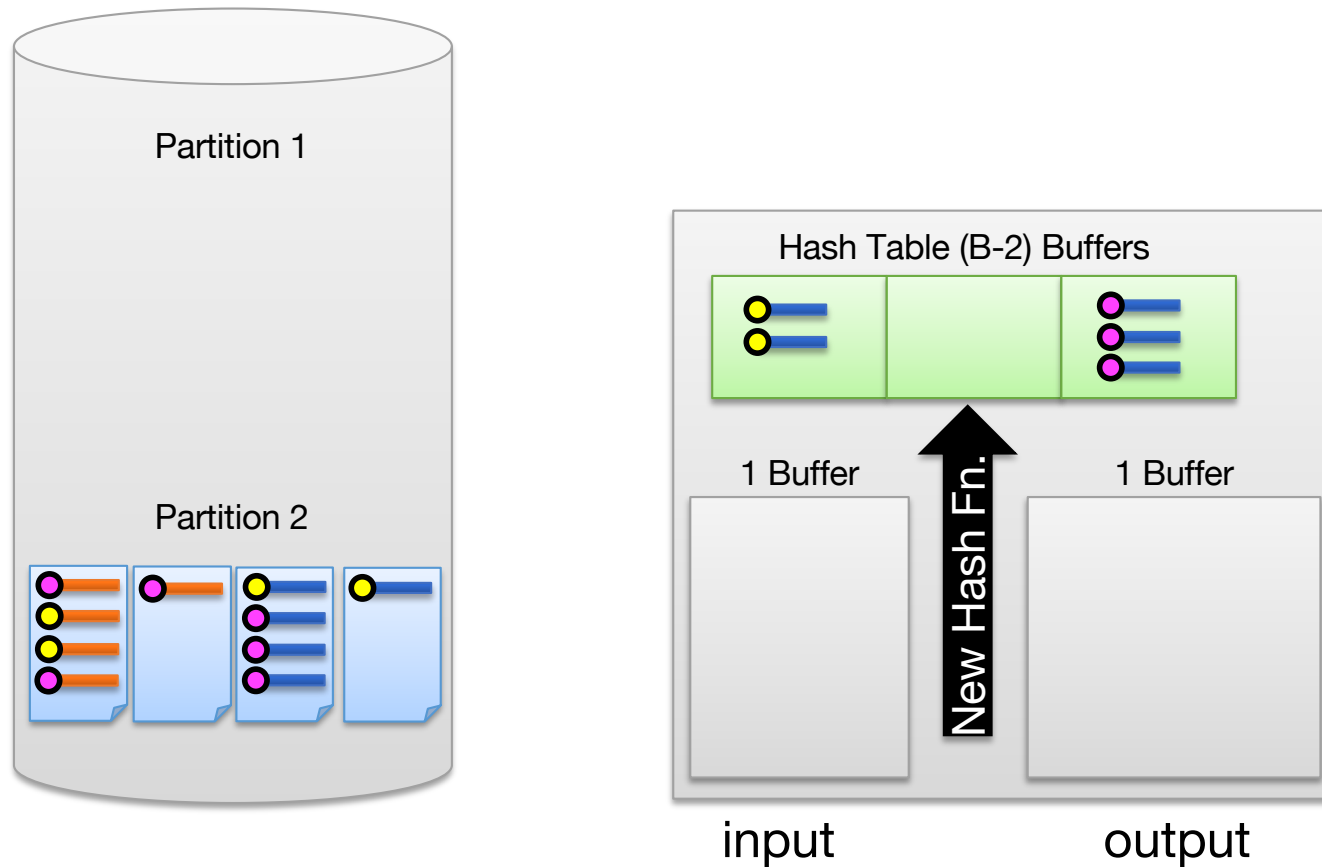
Grace hash join: build & probe



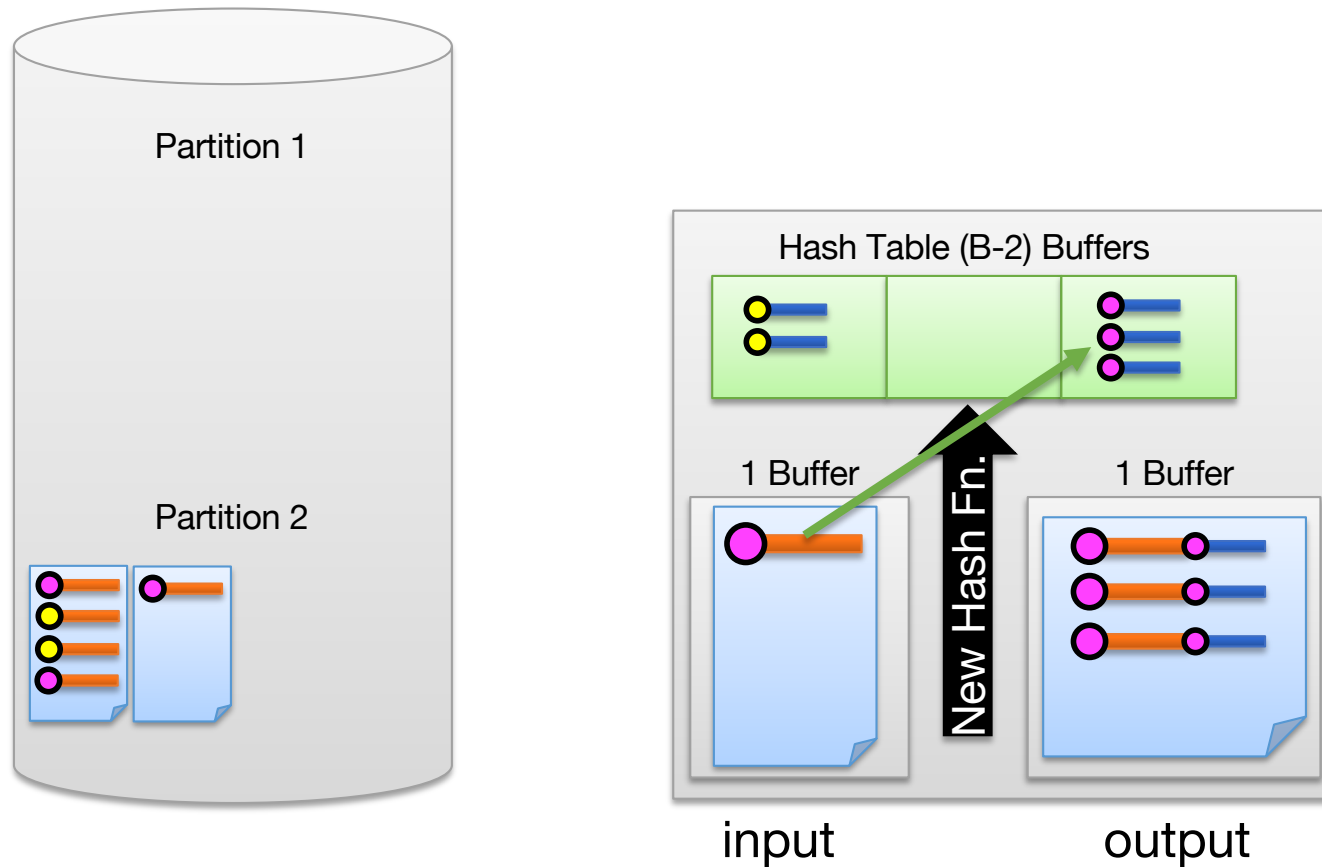
Grace hash join: build & probe



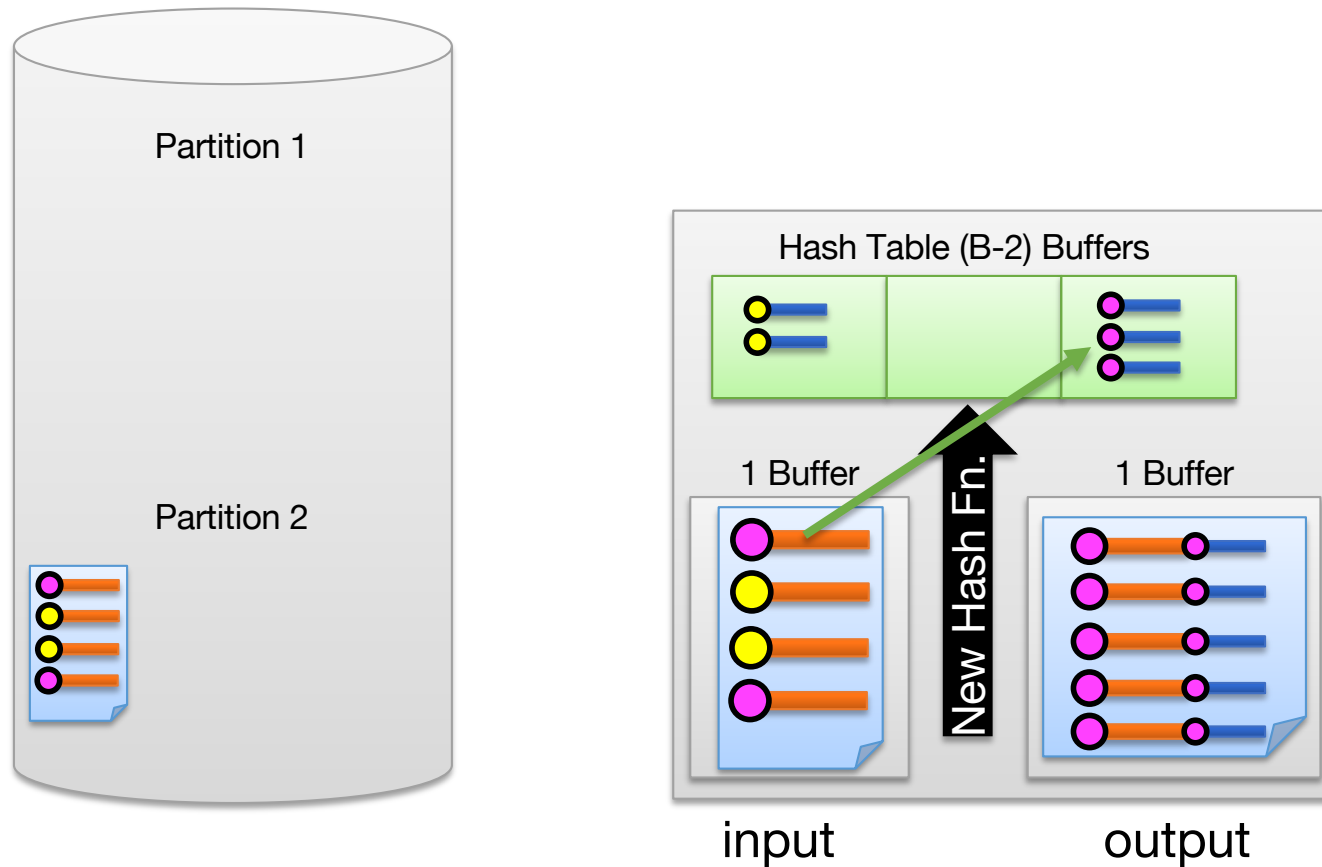
Grace hash join: build & probe



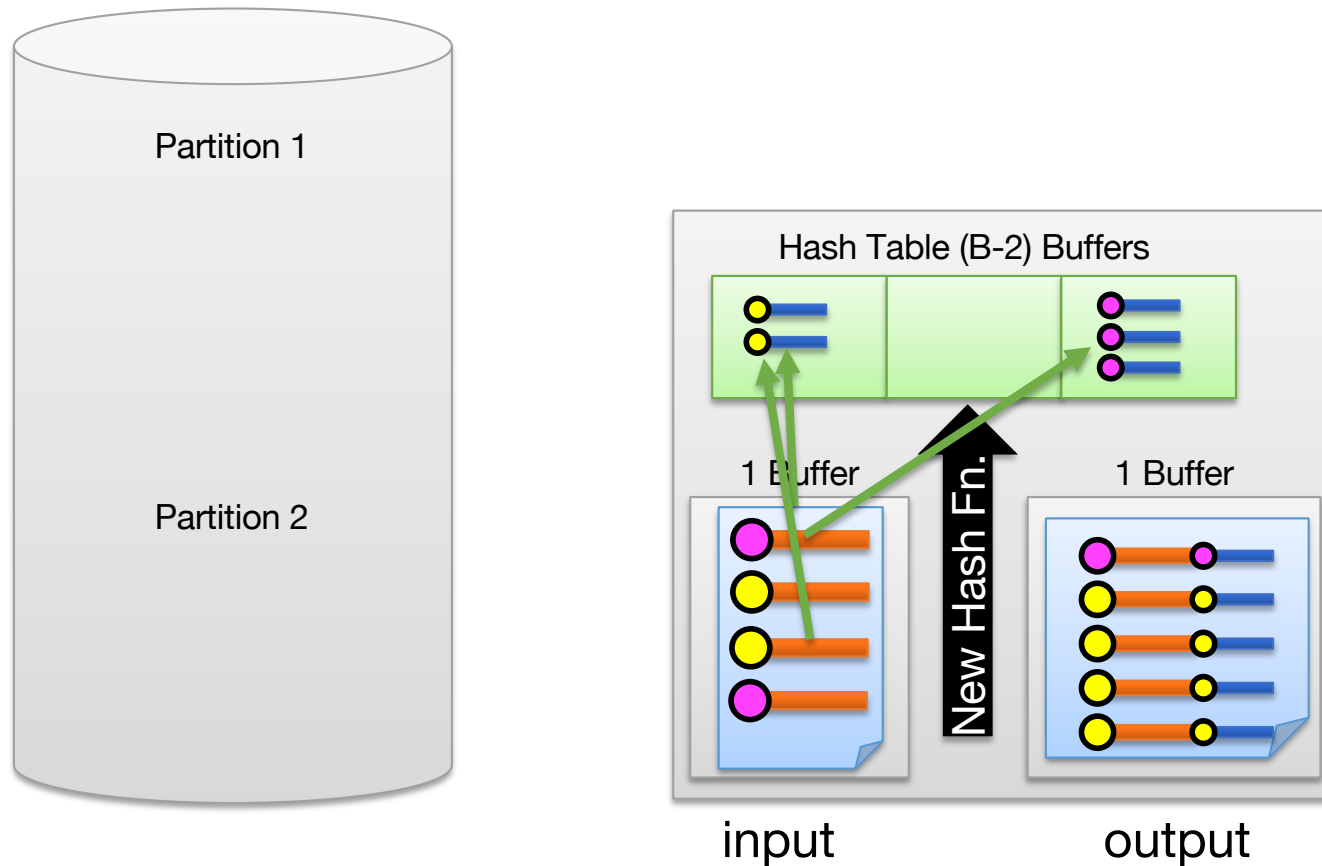
Grace hash join: build & probe



Grace hash join: build & probe

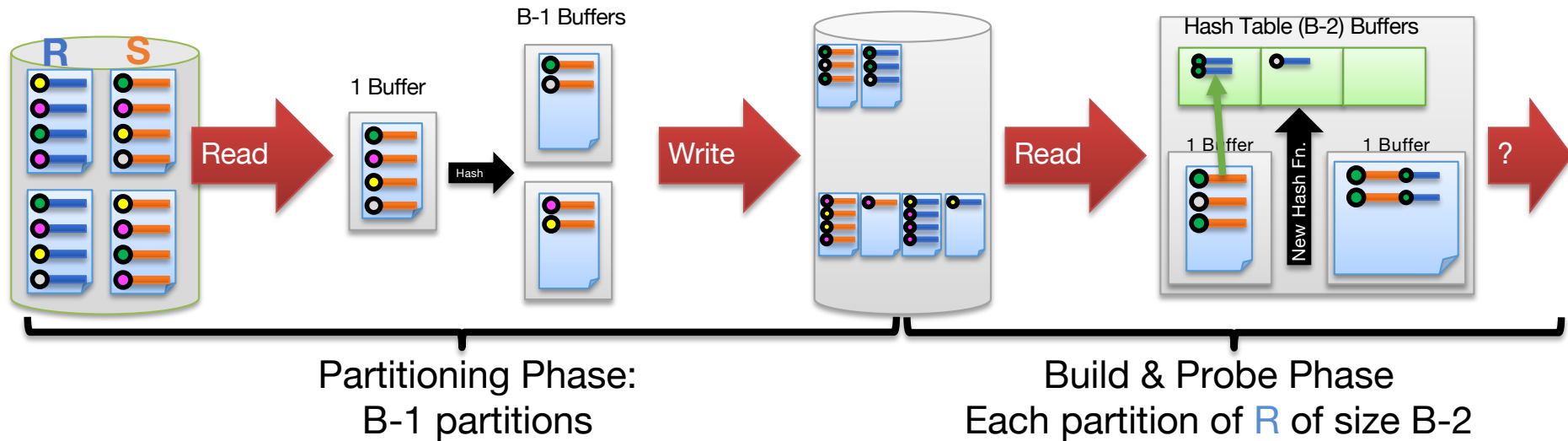


Grace hash join: build & probe



You get the idea ...

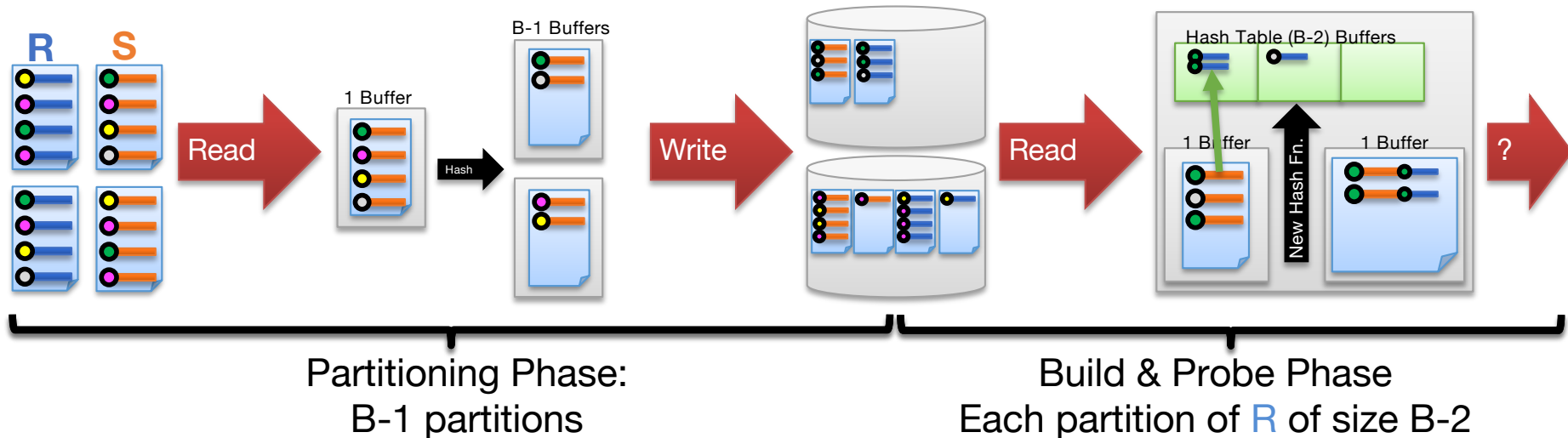
Summary of grace hash join



What is the cost?

Cost of hash

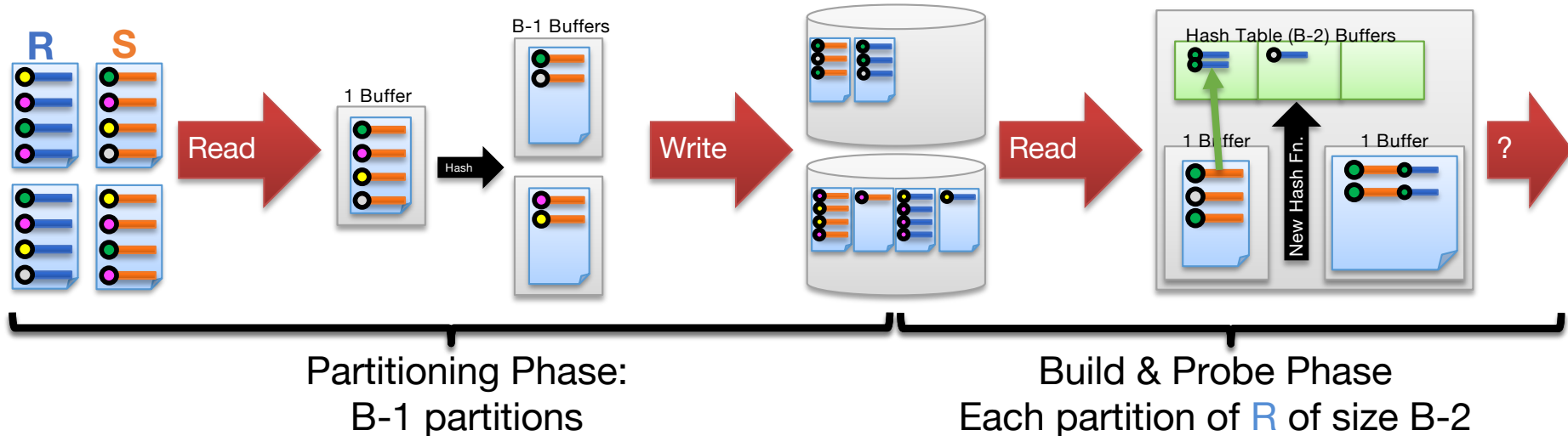
$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$



- Partitioning phase: read+write both relations
 $\Rightarrow 2([R]+[S])$ I/Os
- Matching phase: read both relations, forward output
 $\Rightarrow [R]+[S]$
- Total cost of 2-pass hash join = $3([R]+[S])$
 - $3 * (1000 + 500) = 4500$
 - 45 seconds @10ms per page, probably **much** faster, why?
 - Serial (sequential) I/O have no seek overhead $\rightarrow$ 10ms is much too high

Cost of hash

$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$

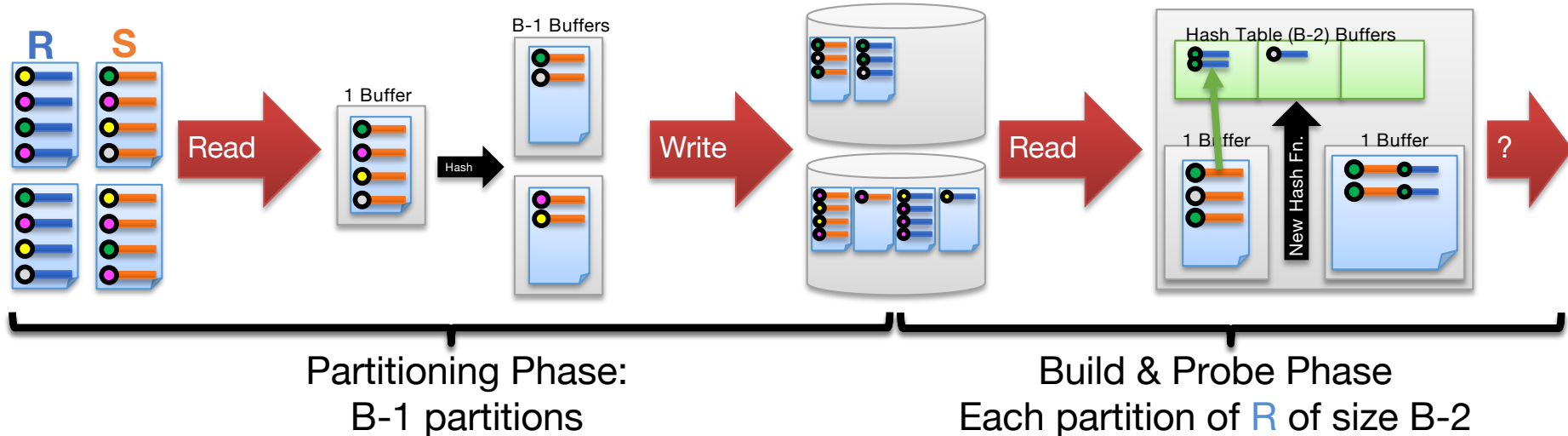


Memory Requirements?

- Build hash table on R with uniform partitioning
 - $\Rightarrow$ **Partitioning Phase** divides R into $(B-1)$ runs of size $[R] / (B-1)$
 - $\Rightarrow$ **Matching Phase** requires each $[R] / (B-1) < (B-2)$
 - $\Rightarrow R < (B-1) (B-2) \approx B^2$

Cost of hash

$[R]=1000, p_R=100, |R| = 100,000$
 $[S]=500, p_S=80, |S| = 40,000$



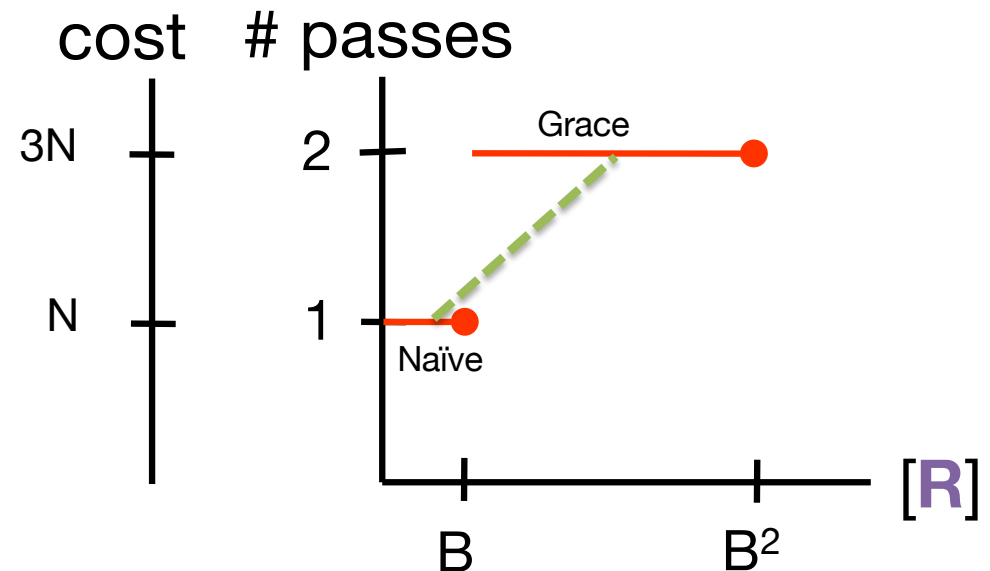
Grace Hash Join: $[R] < B^2$

Naïve Hash Join: $[R] < B$

- Put all of R in hash table

What about in $B < [R] < B^2$?

Can we we do better?



Hash join vs. sort-merge join

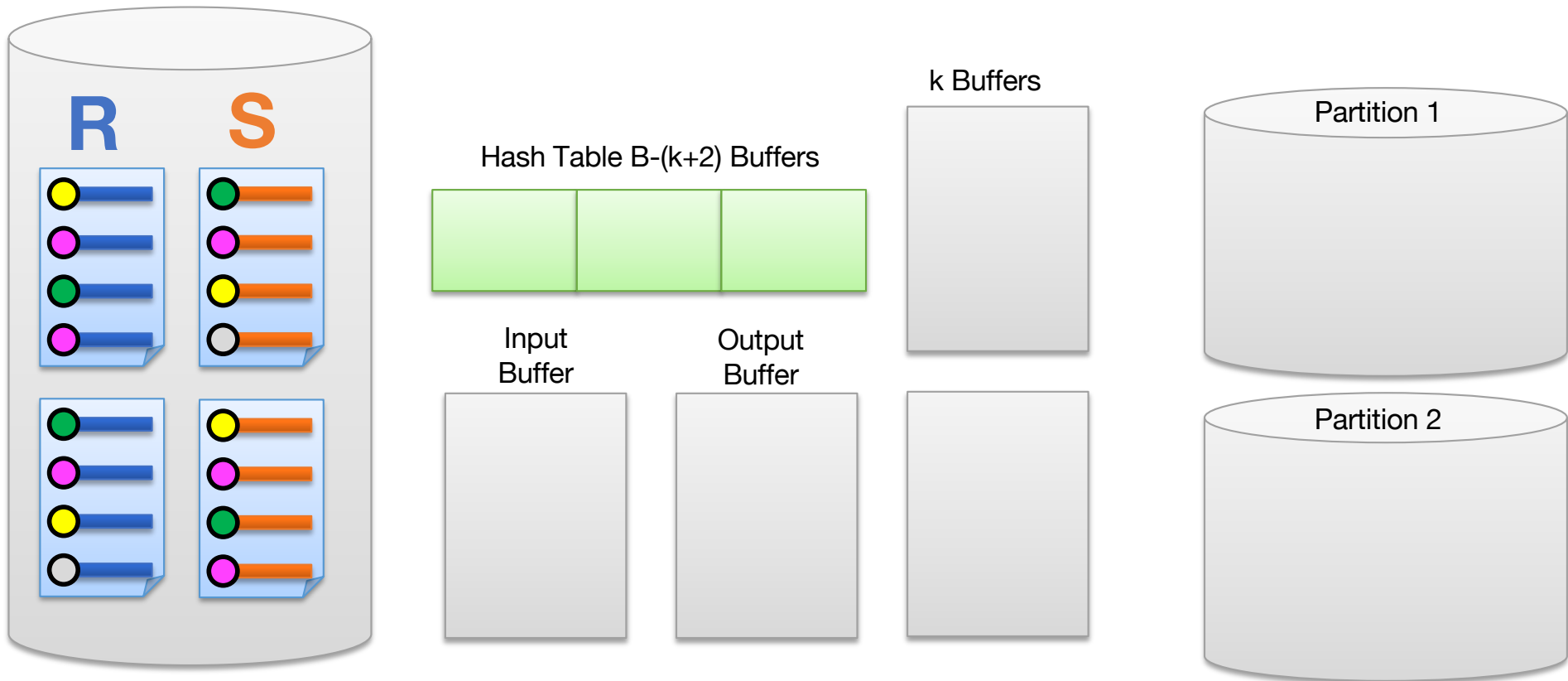
- Sorting pros:
 - Good if input already sorted, or need output sorted
 - Not sensitive to data skew or bad hash functions
 - Important for OLAP
- Hashing pros:
 - Can be cheaper due to hybrid hashing
 - For join: # passes depends on *size of smaller relation*
 - Good if input already hashed, or need output hashed
 - SIMD-NUMA make hash join faster
 - Most important operator for OLAP
 - Many OLTP systems do not have it, but INLJ is close

Summary

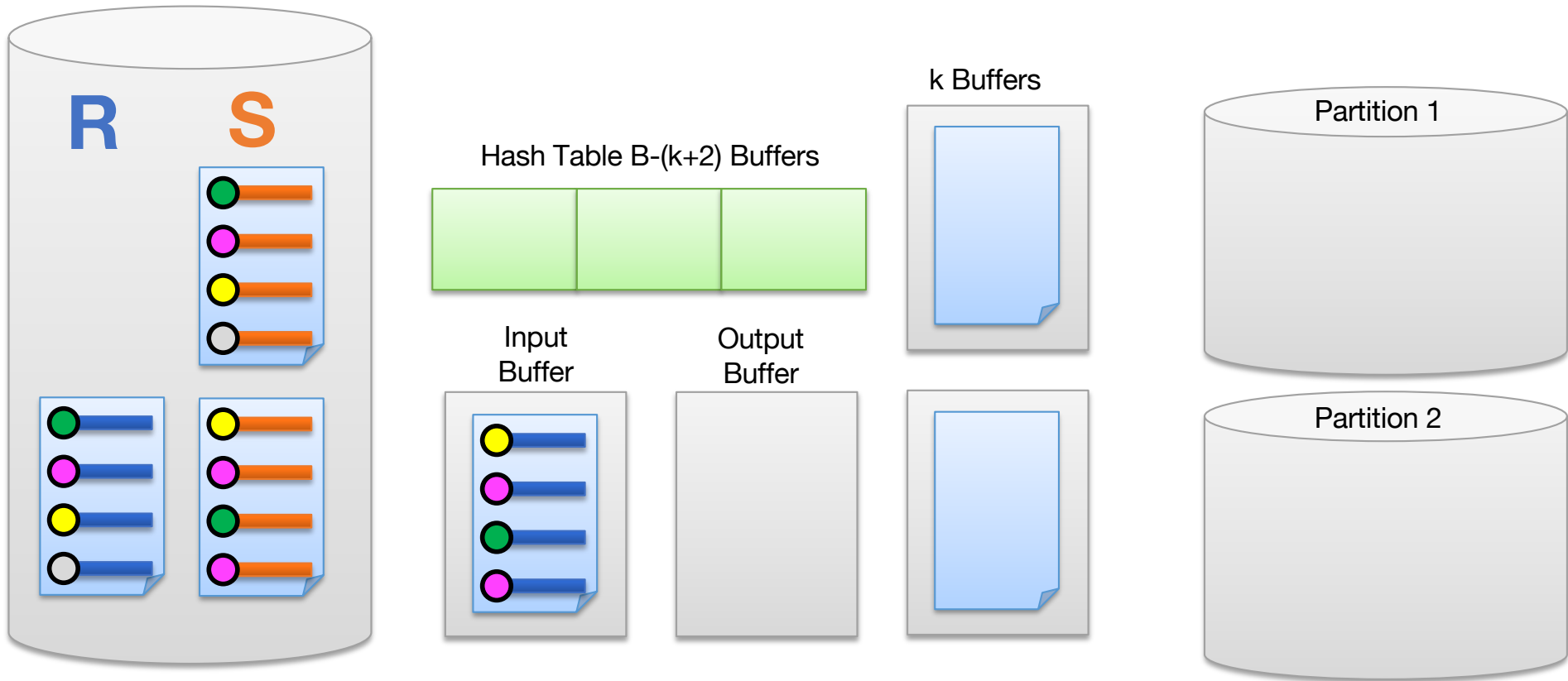
- Nested loops join (SNLJ and CNLJ)
 - Works for arbitrary Θ
 - Make sure to utilize memory in blocks
- Index nested loops join (INLJ)
 - For equi-joins
 - When you already have an index on one side
- Sort / Hash Join (SJ and HJ)
 - For equi-joins
 - No index required
- No clear winners – one may want to implement them all !
- Be sure you know the cost model for each
 - You will need it for query optimization!

Extra slides

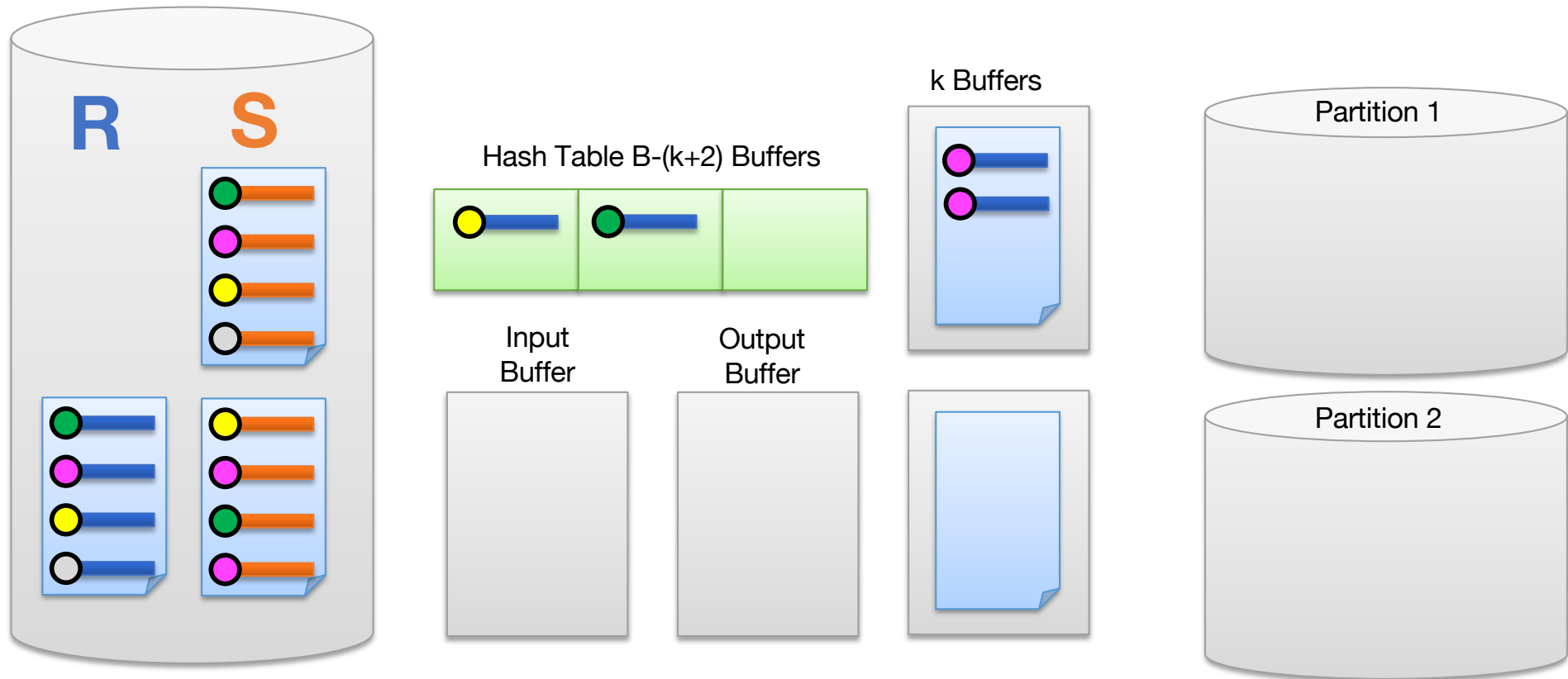
Hybrid hash join: *partition*



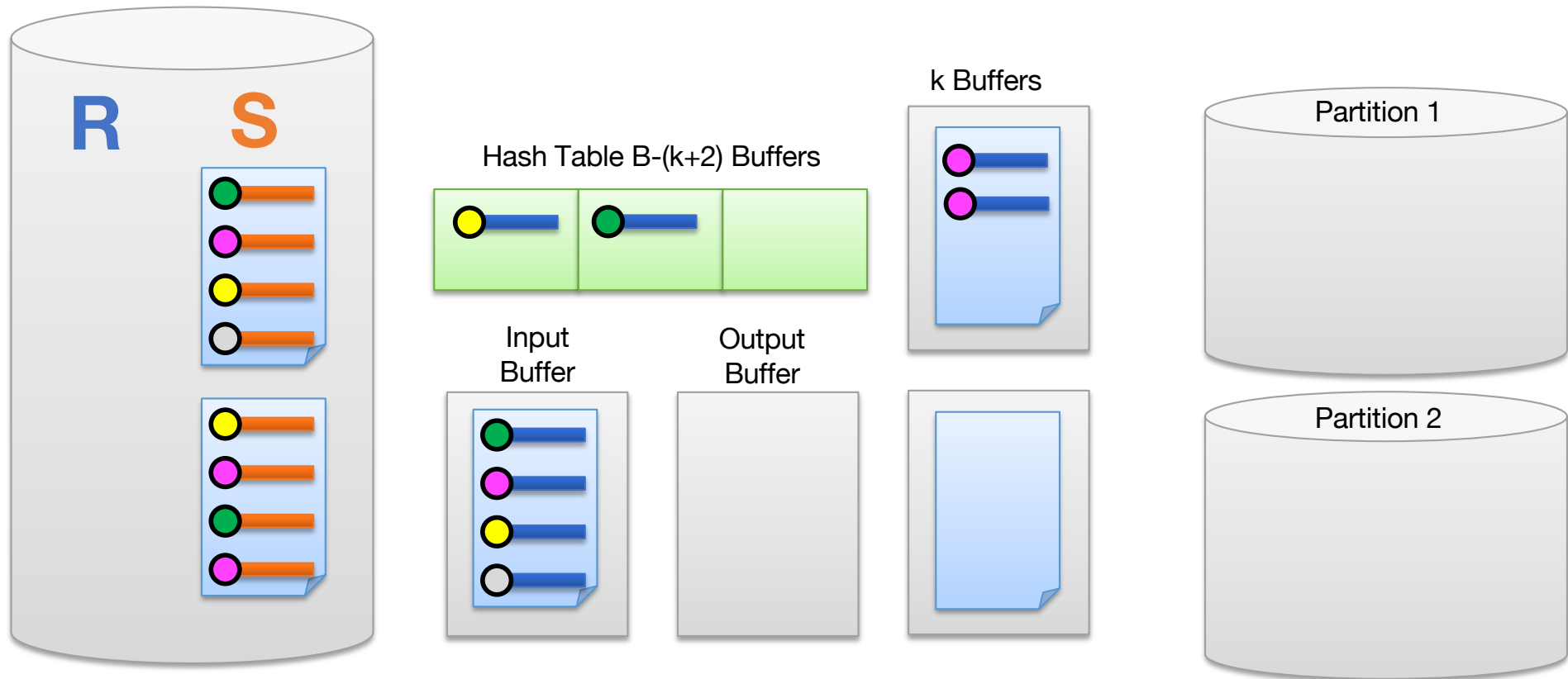
Hybrid hash join: *partition*



Hybrid hash join: *partition*

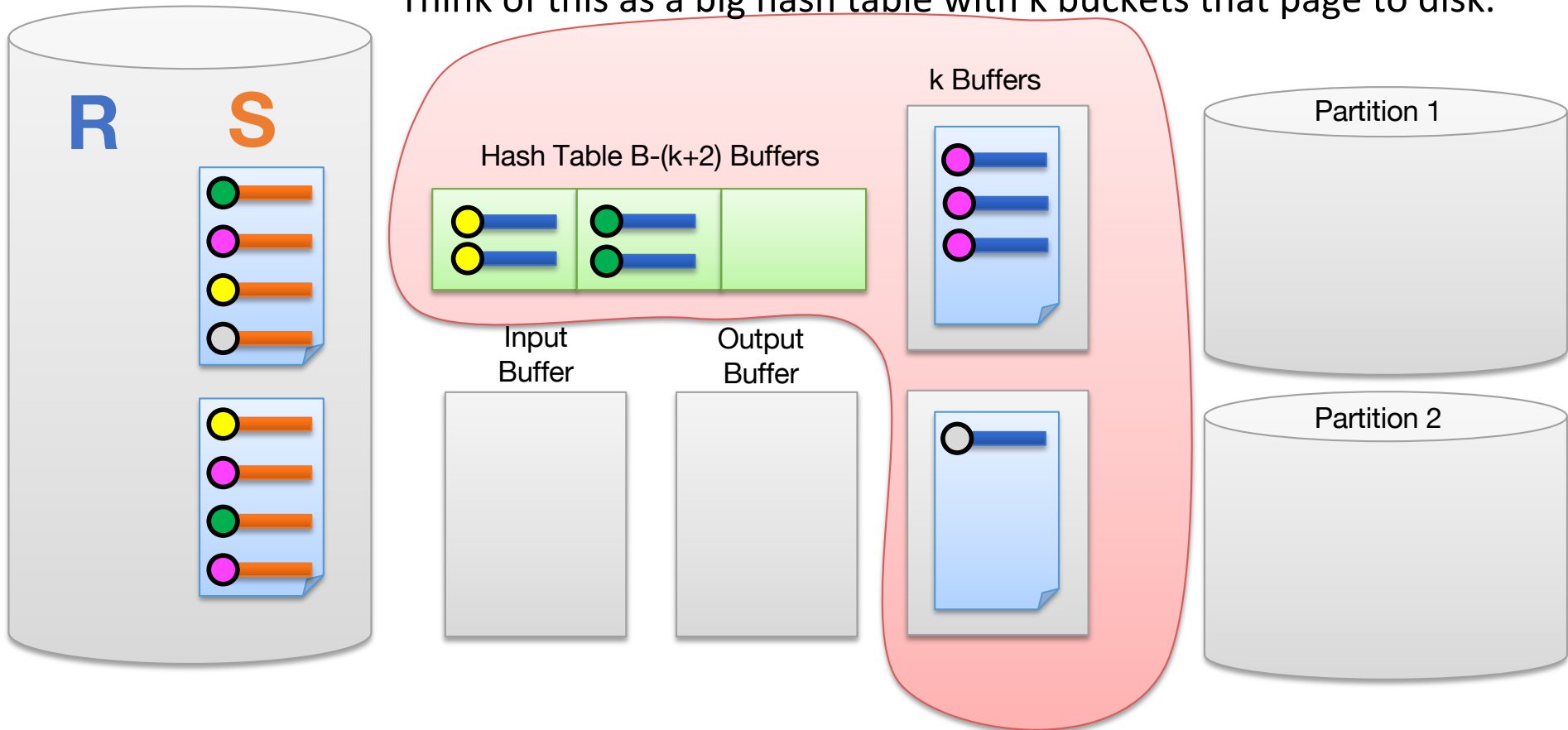


Hybrid hash join: *partition*

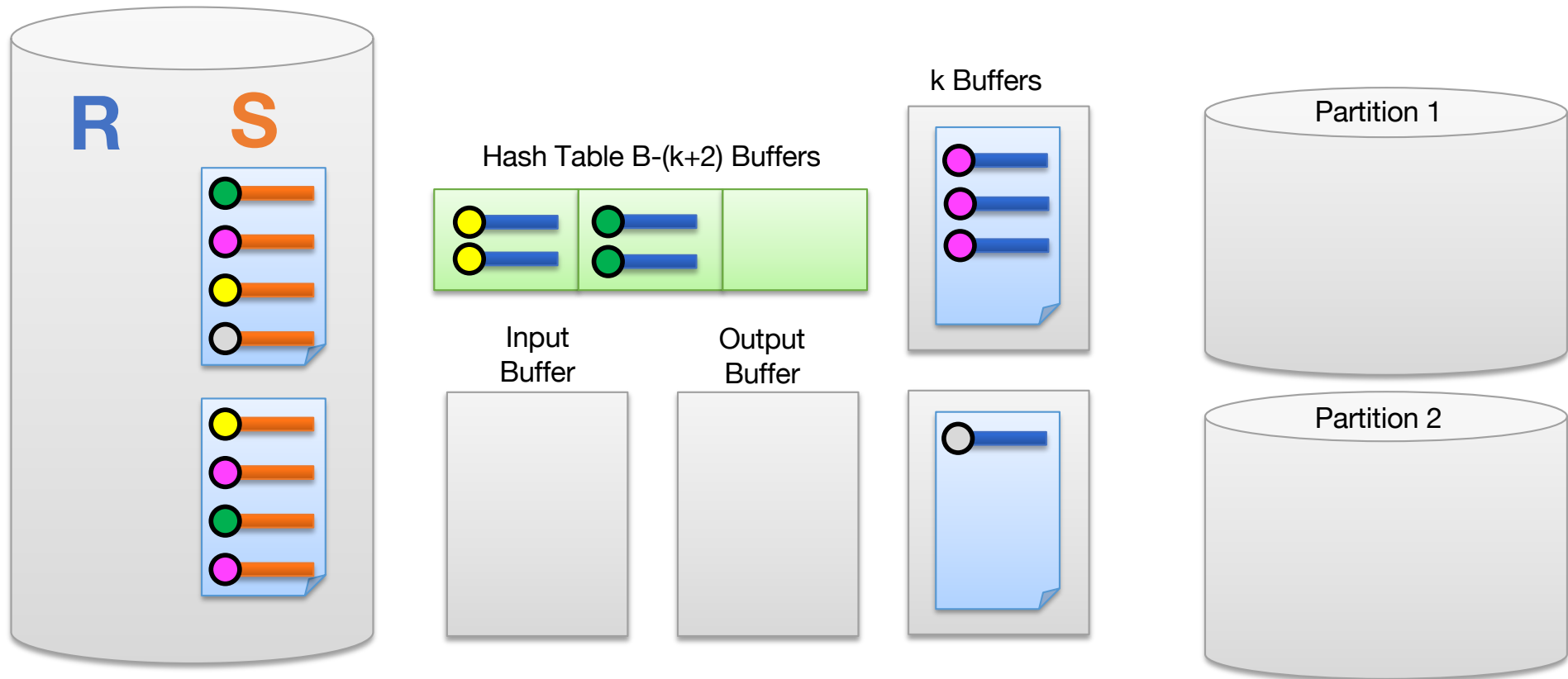


Hybrid hash join: *partition*

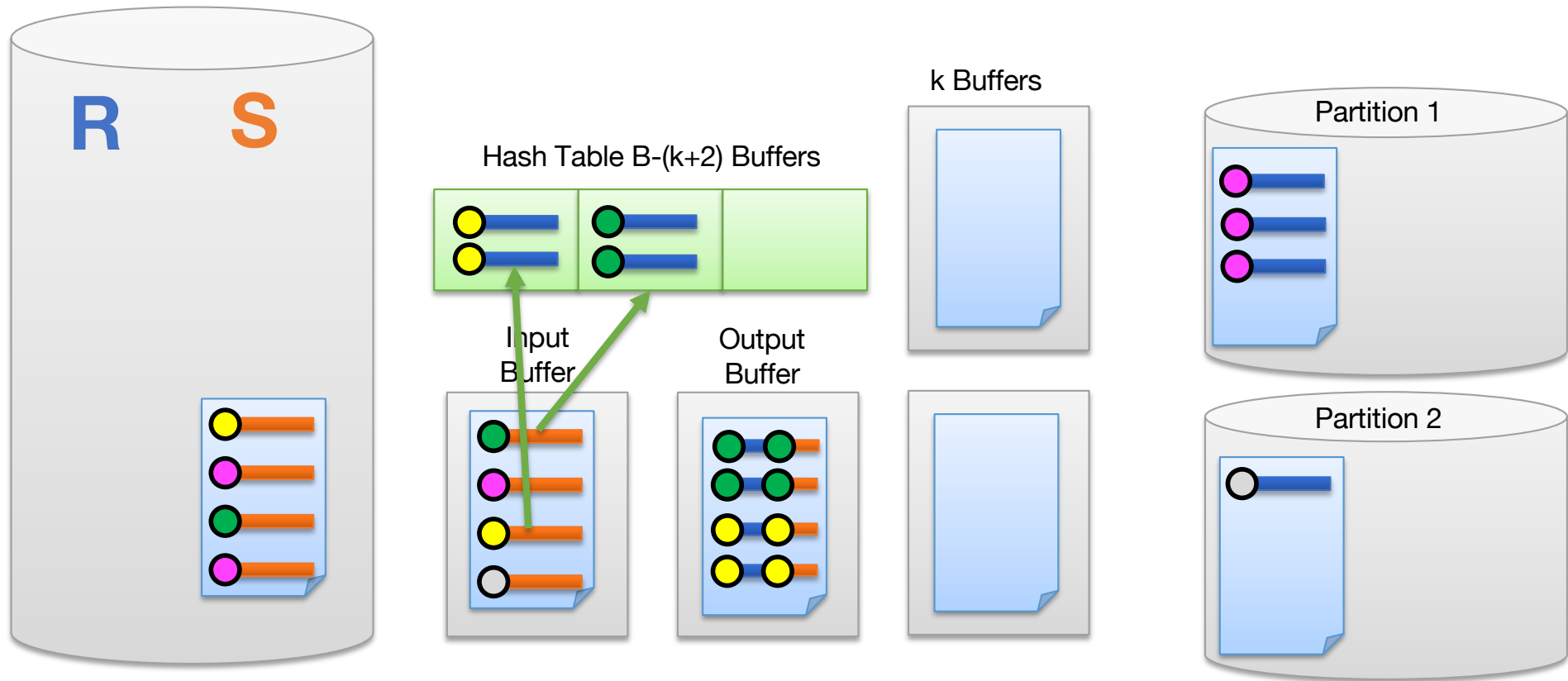
Think of this as a big hash table with k buckets that page to disk.



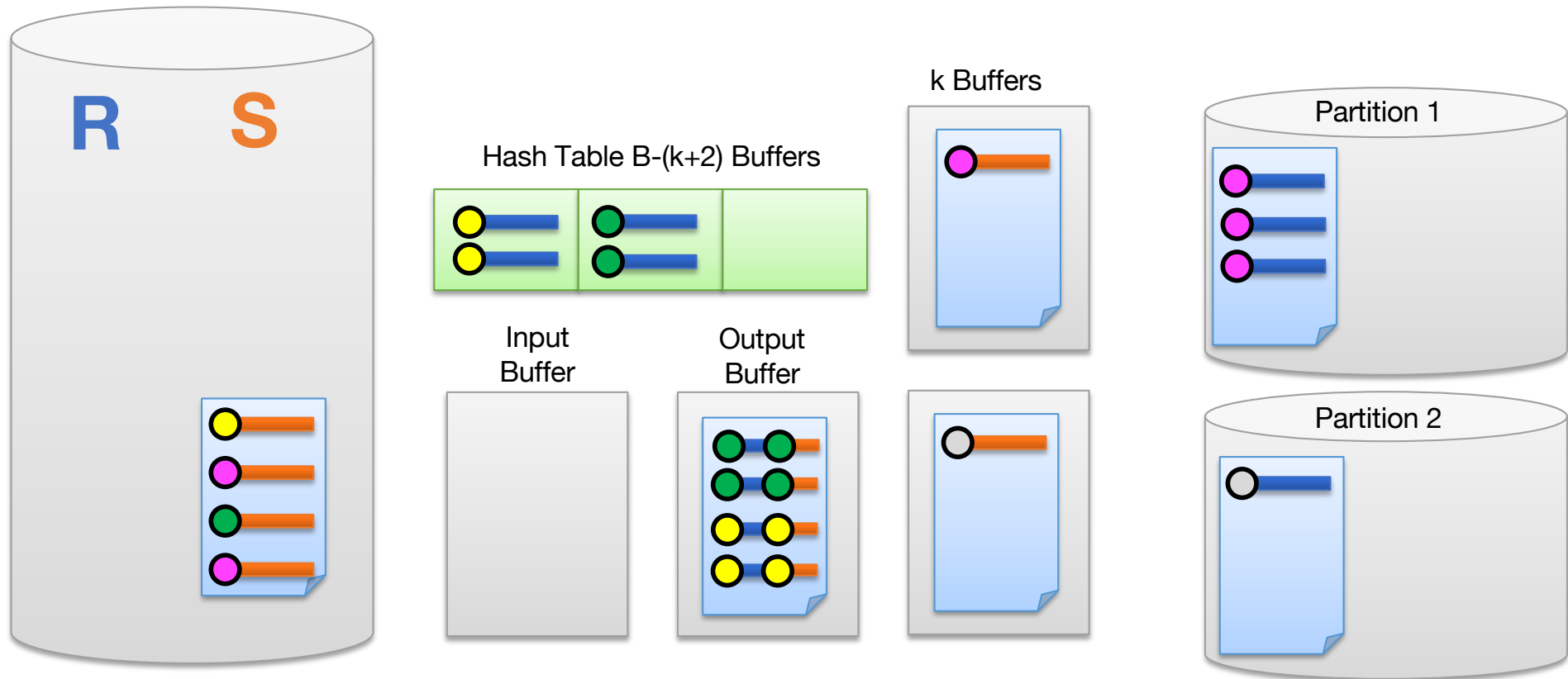
Hybrid hash join: *partition*



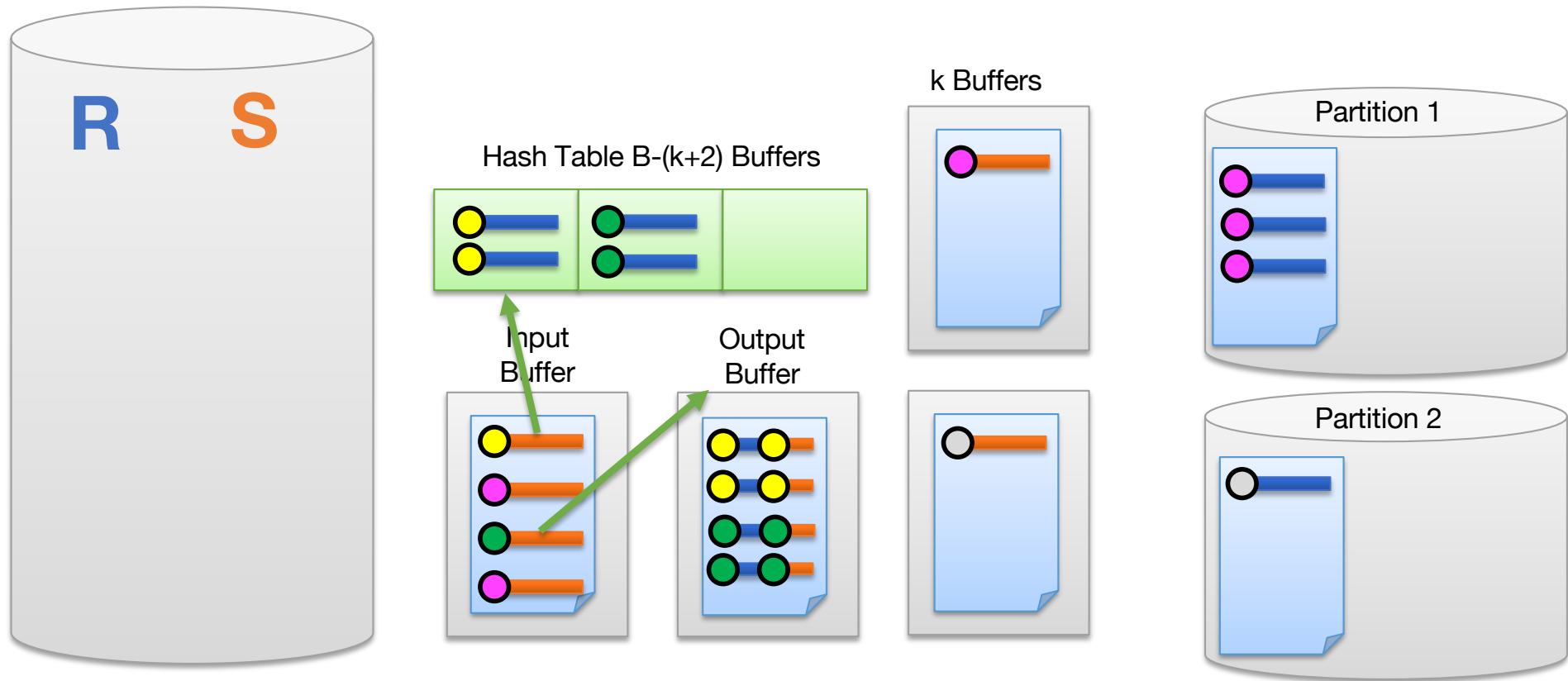
Hybrid hash join: *partition*



Hybrid hash join: *partition*

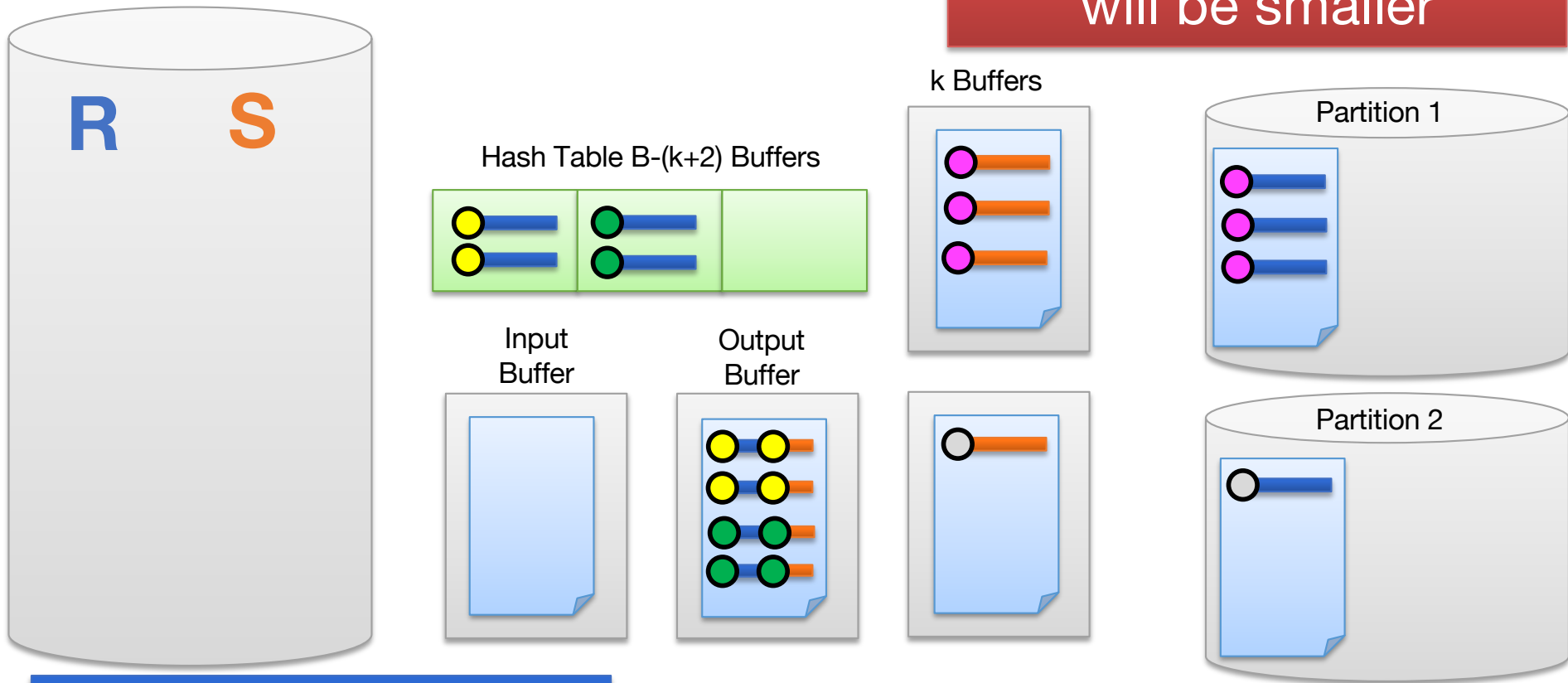


Hybrid hash join: *partition*



Hybrid hash join: *partition*

Notice that next phase will be smaller



How do we pick k ?

Amount in each "run" $\frac{([R] - (B - (k + 2)) * f)}{k} = B - 2$ Memory in build phase $\Rightarrow k = \frac{[R] - (B - 2) * f}{B - f - 2}$

If $R < (B-2)*f$ then set k to zero